TRISHA ACADEMY

DevOps Career Track · Student Notes





Kubernetes for Freshers

Complete 11-class course with live labs

Follow Ravi at QuickBite from his first pod to production

Architecture · Pods · Deployments · Services · Ingress · ConfigMaps & Secrets · Storage · StatefulSets · Scheduling · Probes & HPA · RBAC · GitOps · Helm charts · Kustomize · 20 interview Q&A per class

For students of TRISHA ACADEMY · Keep these notes for revision and interviews

Contents

Welcome and how to use these notes . . . 5

Welcome to Kubernetes . . . 5

Class 1: Why Kubernetes? Architecture and how it works inside . . . 7

1.1 The problem: containers without a manager . . . 7

1.2 What is container orchestration? . . . 8

1.3 Advantages and disadvantages . . . 8

1.4 Kubernetes architecture . . . 9

1.5 Desired state and the control loop . . . 10

1.6 What happens when you run kubectl apply . . . 11

1.7 Live lab: your first cluster . . . 12

Class 1 - Top 20 interview questions . . . 14

Class 2: Pods, YAML and kubectl . . . 16

2.1 What is a pod? . . . 16

2.2 Imperative vs declarative . . . 17

2.3 Anatomy of a YAML manifest . . . 17

2.4 Multi-container patterns . . . 18

2.5 Pod lifecycle . . . 19

2.6 Labels, selectors and annotations . . . 20

2.7 Live lab: the kubectl toolbox . . . 20

Class 2 - Top 20 interview questions . . . 22

Class 3: ReplicaSets and Deployments . . . 24

3.1 From pods to ReplicaSets . . . 24

3.2 Rolling updates . . . 25

3.3 Rollout history and rollback . . . 26

3.4 Release strategies . . . 27

3.5 Live lab: deploy, update, break, roll back . . . 27

Class 3 - Top 20 interview questions . . . 29

Class 4: Services and cluster networking . . . 31

4.1 Why we need Services . . . 31

4.2 Ports explained . . . 32

4.3 Service types . . . 32

4.4 Cluster DNS . . . 33

4.5 How pod networking works . . . 33
4.6 Live lab: Services in action . . . 34
Class 4 - Top 20 interview questions . . . 35

Class 5: Ingress, Gateway API and Namespaces . . . 37

5.1 Why Ingress? . . . 37
5.2 Gateway API - the next generation . . . 38
5.3 Namespaces . . . 39
5.4 Live lab: one door, two apps . . . 40
Class 5 - Top 20 interview questions . . . 42

Class 6: ConfigMaps and Secrets . . . 44

6.1 Separate config from code . . . 44
6.2 Creating ConfigMaps . . . 45
6.3 Consuming config in a pod . . . 46
6.4 Secrets . . . 46
6.5 Immutable config and the Downward API . . . 47
6.6 Live lab . . . 48
Class 6 - Top 20 interview questions . . . 49

Class 7: Storage, PersistentVolumes and StatefulSets . . . 51

7.1 The disappearing data problem . . . 51
7.2 PersistentVolume and PersistentVolumeClaim . . . 52
7.3 StorageClass and dynamic provisioning . . . 53
7.4 StatefulSets . . . 53
7.5 Live lab . . . 55
Class 7 - Top 20 interview questions . . . 56

Class 8: Workload types and scheduling . . . 58

8.1 Choosing a workload controller . . . 58
8.2 How scheduling works . . . 59
8.3 Requests, limits and QoS . . . 61
8.4 Live lab: a 3-node cluster . . . 61
Class 8 - Top 20 interview questions . . . 64

Class 9: Health checks, autoscaling, monitoring and troubleshooting . . . 66

9.1 Health probes . . . 66
9.2 Autoscaling . . . 68
9.3 Observability . . . 69

9.4 Troubleshooting playbook . . . 69
9.5 Live lab . . . 70
Class 9 - Top 20 interview questions . . . 71

Class 10: Security, GitOps and production Kubernetes . . . 73

10.1 Who can do what? (RBAC) . . . 73
10.2 Securing pods . . . 74
10.3 NetworkPolicies . . . 75
10.4 CI/CD and GitOps . . . 76
10.5 Kubernetes in production . . . 77
10.6 Live lab: security and GitOps . . . 77
10.7 Final project: WordPress + MySQL . . . 78
Class 10 - Top 20 interview questions . . . 80

Class 11: Helm charts in depth, Kustomize and cluster essentials . . . 82

11.1 Why Helm? . . . 82
11.2 Using public charts . . . 83
11.3 Anatomy of a chart . . . 84
11.4 The template language . . . 86
11.5 Values for many environments . . . 87
11.6 Dependencies, hooks and tests . . . 88
11.7 Debug, package and share . . . 89
11.8 Helm in CI/CD and GitOps . . . 89
11.9 Kustomize - the template-free alternative . . . 89
11.10 Other important concepts . . . 91
11.11 Live lab: your own Helm chart . . . 93
Class 11 - Top 20 interview questions . . . 95

kubectl cheat sheet . . . 97

Concept coverage map . . . 99

Everyday-example glossary . . . 99

Final checklist and next steps . . . 101

Final checklist - can you do these without notes? . . . 101

Welcome to Kubernetes

Docker taught you how to pack an application into a container. That is great for one laptop. But real companies run hundreds of containers on dozens of servers, and they need someone to start them, restart them when they crash, spread them across machines, give them stable addresses, and update them without downtime. **Kubernetes** is that someone.

These notes go with eleven live classes of 60-90 minutes each. Every class has the same shape:

- **A story** - what problem Ravi faced at QuickBite that day.
- **Concepts with everyday examples** - restaurants, offices, malls, trains.
- **Diagrams** - what happens inside the cluster.
- **Live lab** - commands you type on your own laptop, with the output you should expect.
- **Common mistakes and a recap**.
- **20 interview questions with model answers**.

> **Ravi's story**
> Ravi joined **QuickBite**, a food-delivery start-up, as a fresher DevOps engineer. QuickBite already ships its apps as Docker containers - a menu service, an orders service, a payment service and a MySQL database. On his first Friday night sale, the orders container crashed at 9 PM and nobody noticed for 40 minutes. His manager said: "Next month we move to Kubernetes. You are going to learn it." Follow Ravi through these eleven classes.

Course map

Class	Topic	Main concepts
1	Why Kubernetes and its architecture	orchestration, control plane, nodes, desired state, minikube
2	Pods, YAML and kubectl	pod, multi-container, init containers, labels, selectors, annotations
3	ReplicaSets and Deployments	self-healing, scaling, rolling update, rollback, blue-green, canary
4	Services and cluster networking	ClusterIP, NodePort, LoadBalancer, DNS, kube-proxy, CNI
5	Ingress, Gateway API and Namespaces	host/path routing, TLS, namespaces, quotas, contexts
6	ConfigMaps and Secrets	env, envFrom, volumes, base64, immutable, Downward API
7	Storage and StatefulSets	volumes, PV, PVC, StorageClass, access modes, StatefulSet
8	Workloads and scheduling	DaemonSet, Job, CronJob, affinity, taints, requests/limits, QoS
9	Health, scaling and troubleshooting	probes, HPA, VPA, PDB, metrics, logs, debugging
10	Security, GitOps and production	RBAC, NetworkPolicy, securityContext, Argo CD, managed K8s
11	Helm charts, Kustomize and cluster essentials	charts, templates, values, hooks, Kustomize, CRDs, kubeadm, etcd backup

Set up your lab before Class 1

You need Docker Desktop (from the Docker course) and about 8 GB of free RAM. We use **minikube**, which runs a real Kubernetes cluster inside a Docker container on your laptop. Open PowerShell as Administrator:

PowerShell - one-time setup

```
winget install Kubernetes.kubectl
winget install Kubernetes.minikube
winget install Helm.Helm

# close and reopen PowerShell, then:
kubectl version --client
minikube version
minikube start --driver=docker --cpus=2 --memory=4096
kubectl get nodes
```

Expected output of the last command:

```
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   45s   v1.3x.x
```

Lab tips

- Stop the cluster at the end of each class with `minikube stop`; start it again with `minikube start`. Your work is kept.
- If things get badly broken, `minikube delete` and then `minikube start` gives you a fresh cluster in two minutes.
- Linux and Mac users: install the same tools with your package manager; every kubectl command in these notes is identical.

How to read the notes

Box	Meaning
Purple - Ravi's story	The real-world problem behind the topic.
Gold - Everyday example	A non-technical comparison to make the idea stick.
Green - Try it yourself	Something extra to type and observe.
Red - Common mistake	What freshers get wrong in labs and interviews.
Teal - Did you know?	A fun fact or bit of history.
Blue - Remember	The short summary you should memorise.

K8S · CLASS 01 · 60-90 minutes

Why Kubernetes? Architecture and how it works inside

By the end you can explain why companies use Kubernetes, name every component of a cluster, and run your own cluster on your laptop.

What you will learn today

- The problem with running containers by hand
- What container orchestration means, and where Kubernetes came from
- Advantages and disadvantages of Kubernetes
- Control plane and worker node components - what each one does
- Desired state and the reconciliation (control) loop
- What happens step by step when you run `kubectl apply`
- Lab: start minikube, explore the cluster, watch self-healing

Part	Topic	Time
1	Story and the problem with plain Docker	10 min
2	Orchestration, history, pros and cons	15 min
3	Architecture and the apply flow	25 min
4	Live lab	25 min
5	Recap and interview questions	10 min

1.1 The problem: containers without a manager

Ravi's story

QuickBite runs 12 containers on 3 servers using `docker run`. On Friday night the orders container ran out of memory and died. Docker did not move it anywhere, nobody was paged, and customers saw "Something went wrong" for 40 minutes. Then traffic tripled and Ravi had to SSH into each server and start more containers by hand. On Monday a new release needed to go out - he stopped the old containers and started new ones, causing 3 minutes of downtime.

Plain Docker is excellent at running a container on **one** machine. As soon as you have many machines and many containers, you need answers to these questions:

Question	With plain Docker	With Kubernetes
A container crashes at 2 AM. Who restarts it?	You, after you wake up	Kubernetes, within seconds
A whole server dies. What happens to its containers?	They are gone until you recreate them	They are recreated on healthy servers
Traffic triples. How do we add copies?	SSH and docker run, one by one	`kubectl scale` or automatic autoscaling

Question	With plain Docker	With Kubernetes
Which server has free CPU for a new container?	You check manually	The scheduler decides
How do containers find each other when IPs change?	Hard-coded IPs, links	Services with stable DNS names
How do we release a new version?	Stop old, start new (downtime)	Rolling update, zero downtime
Where do passwords and config live?	Env files on each server	ConfigMaps and Secrets

1.2 What is container orchestration?

Orchestration means automatically deploying, scaling, networking, healing and updating containers across a group of machines. You describe *what* you want ("3 copies of the orders app, version 2.1, reachable on port 80") and the orchestrator figures out *how* to make it happen and keeps it that way.

Everyday example
Think of a large restaurant chain. The **head office** does not cook. It decides how many cooks each branch needs, replaces a cook who calls in sick, opens extra counters on festival days, and rolls out the new menu branch by branch so no branch is ever closed. Kubernetes is the head office; your servers are the branches; containers are the cooks.

The name **Kubernetes** is Greek for *helmsman* - the person who steers a ship. That is why the logo is a ship's wheel. People shorten it to **K8s** (K, then 8 letters, then s).

Did you know?
Google ran its own internal orchestrator called **Borg** for over a decade, launching billions of containers a week. In 2014 Google open-sourced a fresh system built on those lessons: Kubernetes. In 2015 it was donated to the Cloud Native Computing Foundation (CNCF). Today every major cloud offers it: Amazon EKS, Azure AKS and Google GKE.

Other orchestrators exist - Docker Swarm, HashiCorp Nomad, Amazon ECS - but Kubernetes has become the industry standard, which is why it appears in almost every DevOps job description.

1.3 Advantages and disadvantages

Advantage	What it means in real life
Self-healing	Crashed containers restart; pods on a dead node are recreated elsewhere.
Horizontal scaling	Go from 3 to 30 copies with one command, or automatically on CPU load.
Zero-downtime releases	Rolling updates replace pods gradually; one-command rollback.
Service discovery and load balancing	Apps call each other by name; traffic is spread across copies.
Declarative configuration	Everything is YAML in Git - reviewable, repeatable, auditable.
Efficient use of servers	The scheduler packs containers onto nodes based on CPU and memory.
Portable	The same YAML runs on a laptop, on-premises, on AWS, Azure or GCP.
Huge ecosystem	Helm, Argo CD, Prometheus, Istio and thousands of CNCF tools.

Disadvantage	Why it matters
Steep learning curve	Many new objects and ideas; that is why this course has eleven classes.
Operational complexity	Upgrades, networking, storage and security need skilled people.
Overkill for small apps	A single website on one server does not need a cluster.
Resource overhead	The control plane and agents use CPU and memory themselves.
Cost if misconfigured	Over-sized requests or forgotten load balancers waste cloud money.
Debugging has more layers	A failure could be in the app, pod, node, network or config.

Remember
Use Kubernetes when you have many services, need high availability, scale often, or release frequently. For one small app, plain Docker or a PaaS is simpler.

1.4 Kubernetes architecture

A Kubernetes **cluster** is a group of machines (called **nodes**) split into two roles:

- **Control plane** - the brain. It stores the desired state, makes decisions and exposes the API. In production there are usually 3 control-plane nodes for high availability.
- **Worker nodes** - the muscle. They actually run your containers.

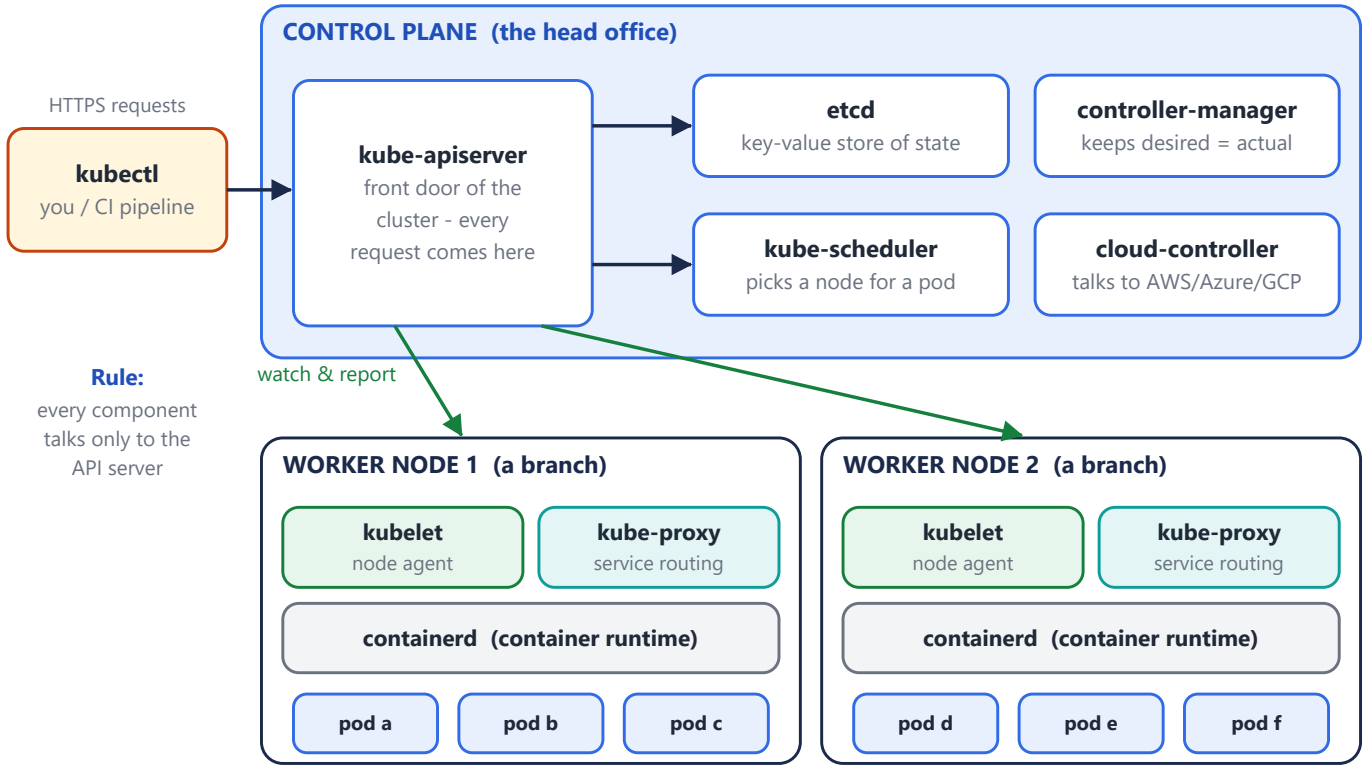


Figure 1.1 - Kubernetes architecture: one control plane managing worker nodes

Control plane components

Component	Job	Everyday example
kube-apiserver	The only entry point. Validates every request (from kubectl, nodes, controllers) and reads/writes etcd.	The reception desk - nobody walks into the office without passing it.
etcd	A distributed key-value database that stores the whole cluster state: every pod, service, secret and config.	The master register book of the head office.
kube-scheduler	Watches for new pods with no node and picks the best node based on resources, rules and affinity.	HR deciding which branch a new cook joins.
kube-controller-manager	Runs many controllers (Deployment, ReplicaSet, Node, Job...) that keep actual state equal to desired state.	Supervisors who keep checking each counter is staffed.
cloud-controller-manager	Talks to the cloud provider: creates load balancers, attaches disks, notices deleted VMs.	The office admin who deals with the landlord.

Worker node components

Component	Job	Everyday example
kubelet	Agent on every node. Receives pod specs from the API server, asks the runtime to start containers, runs health checks, reports status.	The branch manager who reports to head office.
kube-proxy	Programs network rules (iptables/IPVS) so traffic to a Service reaches the right pods.	The telephone operator forwarding calls to the right desk.
Container runtime	Actually runs containers. Usually containerd or CRI-O, talking to the kubelet through the Container Runtime Interface (CRI).	The kitchen equipment the cooks use.
CNI plugin	Gives every pod an IP and connects pods across nodes (Calico, Flannel, Cilium).	The internal phone lines between branches.

Did you know?
Kubernetes stopped using Docker Engine as its runtime ("dockershim" was removed in v1.24). Do not panic - images built with Docker still run perfectly, because they follow the OCI image standard, and containerd runs them directly.

1.5 Desired state and the control loop

This is the single most important idea in Kubernetes. You never say "start a container". You say "I want 3 copies running" - the **desired state**. Controllers then run a never-ending loop: observe what is actually running, compare it with what you asked for, and act to remove the difference. This is called **reconciliation**.

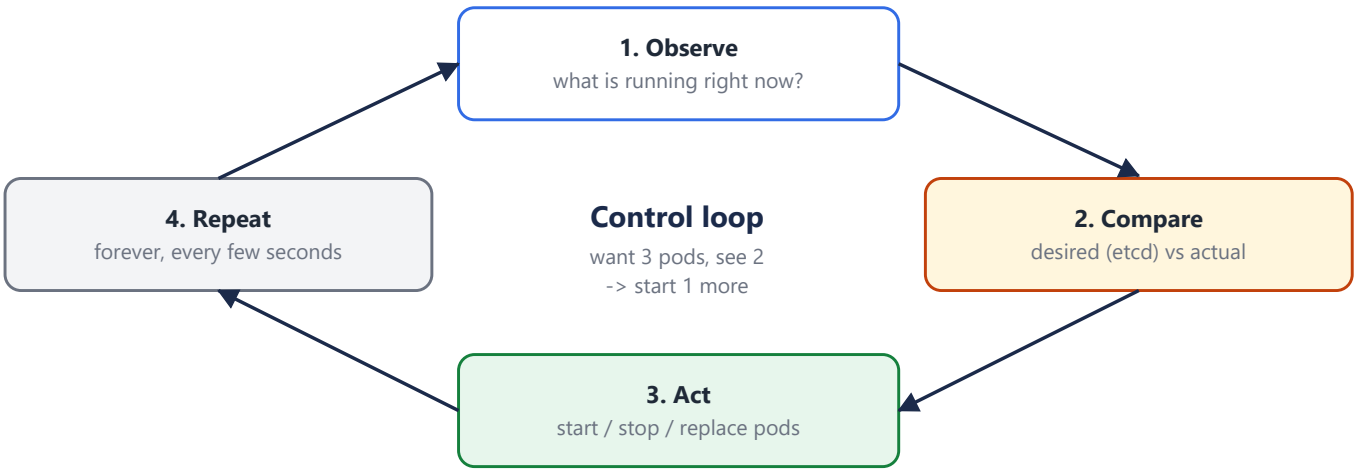


Figure 1.2 - The reconciliation loop that powers self-healing

Everyday example
An air conditioner works the same way. You set 24 degrees (desired state). The thermostat keeps measuring the room (actual state). If it is 27, it cools; if it is 22, it stops. You never tell it "run the compressor for 10 minutes" - you only declare the target.

1.6 What happens when you run kubectl apply

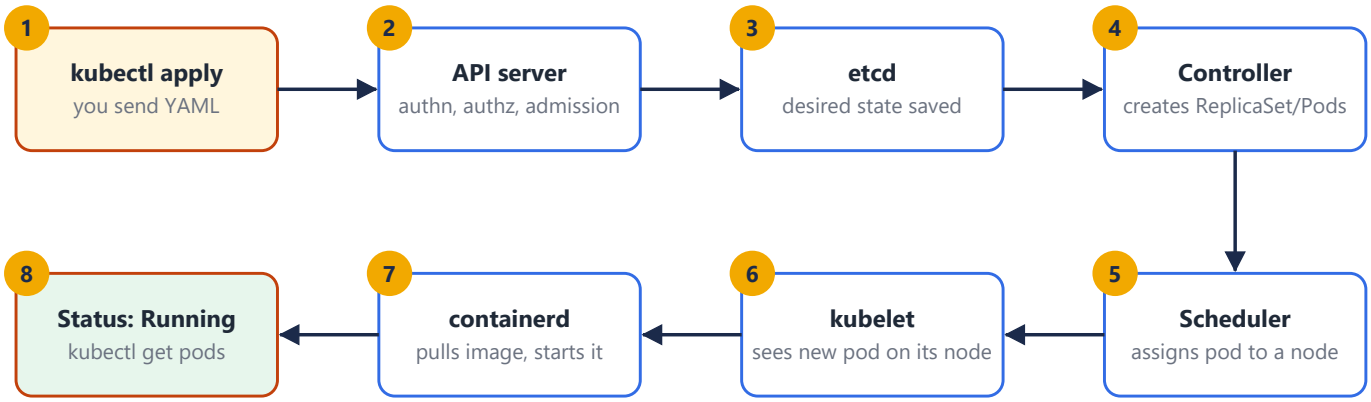


Figure 1.3 - The journey of a Deployment from YAML to running containers

- **1-2.** kubectl sends your YAML to the API server over HTTPS. The API server checks who you are (authentication), whether you are allowed (authorization/RBAC) and runs admission checks.
- **3.** The object is saved in etcd. Nothing is running yet - only the wish is recorded.
- **4.** The Deployment controller notices the new Deployment and creates a ReplicaSet, which creates Pod objects.
- **5.** The scheduler sees pods without a node and writes a node name into each one.
- **6-7.** The kubelet on that node sees a pod assigned to it and tells containerd to pull the image and start the containers.
- **8.** The kubelet reports status back; `kubectl get pods` shows Running.

Remember
Components never talk to each other directly - they all watch the API server. This loose coupling is why Kubernetes is so robust.

1.7 Live lab: your first cluster

Step 1 - Start and inspect the cluster

PowerShell

```
minikube start --driver=docker
kubectl cluster-info
kubectl get nodes -o wide
kubectl get pods -n kube-system
```

In `kube-system` you will see the control-plane components running as pods:

```
NAME                               READY   STATUS    RESTARTS   AGE
coredns-7c65d6cfc9-x2b8l           1/1     Running   0          2m
etcd-minikube                      1/1     Running   0          2m
kube-apiserver-minikube            1/1     Running   0          2m
kube-controller-manager-minikube   1/1     Running   0          2m
kube-proxy-9fk2s                   1/1     Running   0          2m
kube-scheduler-minikube            1/1     Running   0          2m
storage-provisioner                1/1     Running   0          2m
```

Step 2 - Run an app and watch self-healing

PowerShell

```
kubectl create deployment hello --image=nginx:1.27 --replicas=3
kubectl get pods -o wide

# open a second PowerShell window and keep this running:
kubectl get pods -w

# back in the first window, delete one pod:
kubectl delete pod <name-of-any-hello-pod>
```

In the watch window you will see the deleted pod go to **Terminating** and a brand-new pod with a different name appear within seconds. You asked for 3; Kubernetes keeps 3.

Step 3 - Scale and clean up

PowerShell

```
kubectl scale deployment hello --replicas=5
kubectl get pods
kubectl delete deployment hello
minikube dashboard        # opens a web UI in your browser (Ctrl+C to stop)
```

Try it yourself

- Run `kubectl api-resources` to see every object type Kubernetes knows about.
- Run `kubectl get events --sort-by=.metadata.creationTimestamp` after the self-healing demo and read the story the cluster tells.
- Run `minikube ssh` and then `sudo crictl ps` to see the real containers behind your pods.

Common mistake

- Saying "Kubernetes runs Docker containers". It runs **OCI containers** through a CRI runtime such as containerd.
- Thinking the control plane runs your apps. In production, app pods run on worker nodes.
- Forgetting that etcd holds everything - losing etcd without a backup means losing the cluster state.

Class 1 recap

- Kubernetes = an orchestrator that keeps the desired state you declare.
- Control plane: apiserver, etcd, scheduler, controller-manager, cloud-controller-manager.
- Every node: kubelet, kube-proxy, container runtime, CNI.
- Everything goes through the API server; controllers reconcile in a loop.

Class 1 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is Kubernetes?

An open-source container orchestration platform that automates deploying, scaling, healing, networking and updating containerized applications across a cluster of machines. You declare the desired state in YAML and Kubernetes keeps the actual state matching it.

Q2. Why do we need Kubernetes when we already have Docker?

Docker runs containers on a single host. Kubernetes manages containers across many hosts: it restarts failed containers, reschedules them when a node dies, scales them, load-balances traffic, and performs rolling updates. Docker builds and runs images; Kubernetes orchestrates them.

Q3. What is container orchestration?

Automatic management of the container lifecycle across many machines - placement (scheduling), scaling, health monitoring and restarts, networking and service discovery, configuration, and rolling updates.

Q4. Explain the Kubernetes architecture.

A cluster has a control plane and worker nodes. The control plane runs kube-apiserver, etcd, kube-scheduler, kube-controller-manager and optionally cloud-controller-manager. Each worker node runs kubelet, kube-proxy, a container runtime and a CNI plugin. All components communicate through the API server.

Q5. What does the kube-apiserver do?

It is the front end of the control plane and the only component that talks to etcd. It exposes the REST API, authenticates and authorizes requests, runs admission controllers, validates objects, and lets other components watch for changes.

Q6. What is etcd and why is it critical?

etcd is a consistent, distributed key-value store holding the entire cluster state - all objects, configs and secrets. If etcd is lost without a backup the cluster state is lost, so it is run in odd numbers (3 or 5) for quorum and backed up regularly with `etcdctl snapshot save`.

Q7. What does the kube-scheduler do?

It watches for pods with no assigned node and selects a node in two phases: filtering (nodes that have enough resources and satisfy rules like nodeSelector, taints and affinity) and scoring (ranking the remaining nodes). It then binds the pod to the best node. It does not start containers.

Q8. What is the kube-controller-manager?

A single process running many controllers such as the Deployment, ReplicaSet, Node, Job, Endpoints and ServiceAccount controllers. Each controller watches certain objects and reconciles actual state toward desired state.

Q9. What is the role of the kubelet?

The kubelet is the node agent. It watches the API server for pods assigned to its node, asks the container runtime to start them, mounts volumes, runs liveness/readiness/startup probes, and reports pod and node status back.

Q10. What does kube-proxy do?

It runs on each node and maintains network rules (iptables, IPVS or nftables) so that traffic sent to a Service's virtual IP is forwarded to one of the healthy backing pods.

Q11. What is a container runtime? Name some.

Software that actually pulls images and runs containers. The kubelet talks to it through the Container Runtime Interface (CRI). Common runtimes are containerd and CRI-O.

Q12. Did Kubernetes drop Docker? What changed?

Kubernetes removed dockershim in v1.24, so Docker Engine is no longer used directly as the runtime. Images built with Docker still work because they are OCI-compliant; runtimes like containerd run them.

Q13. What is desired state vs actual state?

Desired state is what you declare in manifests (for example 3 replicas of image v2). Actual state is what is really running. Controllers continuously compare the two and act to remove any difference.

Q14. What is a reconciliation loop?

The observe-compare-act cycle that every controller runs forever. It is the reason Kubernetes is self-healing: if a pod disappears, the loop sees 2 instead of 3 and creates a new one.

Q15. What is the difference between imperative and declarative management?

Imperative means giving step-by-step commands such as `kubectl run` or `kubectl scale`. Declarative means writing the desired state in YAML and running `kubectl apply -f`, letting Kubernetes work out the steps. Declarative is preferred in production because files can be versioned in Git.

Q16. What is a node? What are master and worker nodes?

A node is a physical or virtual machine in the cluster. Control-plane (formerly "master") nodes run the management components; worker nodes run application pods.

Q17. What happens if a worker node fails?

The node controller marks it NotReady after missing heartbeats. After a timeout (about 5 minutes by default) its pods are evicted, and controllers such as Deployments recreate them on healthy nodes.

Q18. What happens if the control plane goes down?

Running pods keep running because kubelets and containers work independently, but you cannot deploy, scale, or self-heal until the control plane is back. That is why production uses 3 control-plane nodes.

Q19. What are the advantages and disadvantages of Kubernetes?

Advantages: self-healing, scaling, rolling updates, service discovery, declarative config, portability and a huge ecosystem. Disadvantages: steep learning curve, operational complexity, overhead, and it is overkill for small applications.

Q20. What tools can you use to run Kubernetes locally?

minikube, kind (Kubernetes in Docker), k3d/k3s, and the Kubernetes option in Docker Desktop. For production you would use kubeadm or managed services such as EKS, AKS and GKE.

K8S · CLASS 02 · 60-90 minutes

Pods, YAML and kubectl

By the end you can write pod YAML from memory, run multi-container pods, and use labels, selectors and the most important kubectl commands.

What you will learn today

- What a pod is and why Kubernetes does not run bare containers
- Imperative commands vs declarative YAML
- The four required fields of every manifest
- Multi-container pods: sidecar pattern, shared volume, init containers
- Pod lifecycle phases and restart policies
- Labels, selectors and annotations
- Lab: kubectl get, describe, logs, exec, port-forward, explain, dry-run

Part	Topic	Time
1	Story and what a pod is	10 min
2	YAML anatomy and kubectl basics	20 min
3	Multi-container pods and lifecycle	20 min
4	Labels and selectors	10 min
5	Live lab, recap and interview questions	25 min

2.1 What is a pod?

Ravi's story

Ravi wanted to run the orders app together with a small log-shipper container that reads the app's log files and sends them to the monitoring system. In Docker he had to set up a shared volume and network by hand. His senior said: "In Kubernetes, put both in one **pod** - they automatically share network and storage."

A **pod** is the smallest deployable unit in Kubernetes. It wraps one or more containers that:

- share **one IP address** and port space - they talk to each other on `localhost`;
- can share **volumes** (folders);
- are always **scheduled together** on the same node and live and die together.

Everyday example

A pod is like a **flat** in an apartment building. The flat has one postal address (the IP). Flatmates (containers) share the kitchen and living room (network and volumes) and can shout to each other directly (localhost). If the flat is demolished, everyone moves out together.

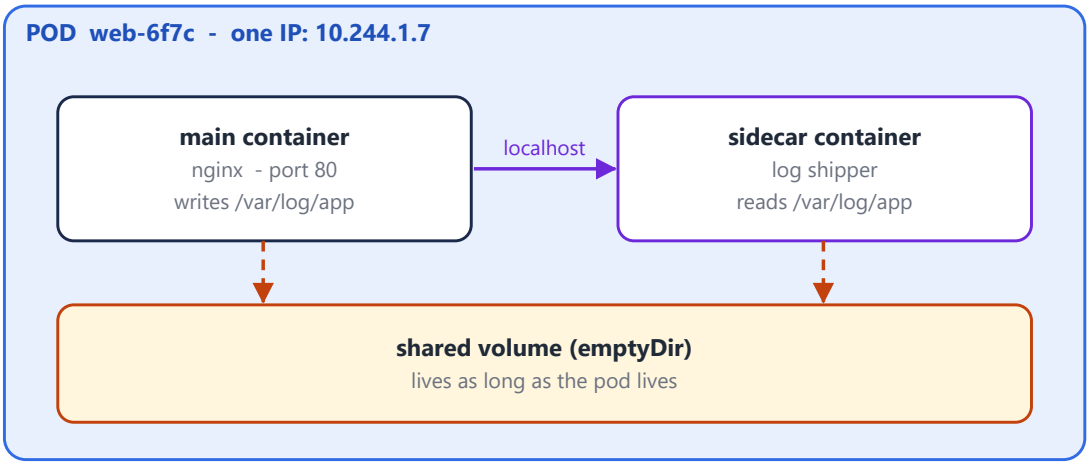


Figure 2.1 - A pod with a main container and a sidecar sharing IP and a volume

Remember
Most pods have exactly one container. Add a second container only when it must live and scale with the first one (log shipper, proxy, config reloader).

2.2 Imperative vs declarative

Style	Example	Use it for
Imperative command	`kubectl run web --image=nginx:1.27`	Quick tests, learning, exams
Imperative with file	`kubectl create -f pod.yaml`	One-time creation; fails if it exists
Declarative	`kubectl apply -f pod.yaml`	Real projects: files live in Git, apply is repeatable

2.3 Anatomy of a YAML manifest

Every Kubernetes object has the same four top-level fields:

Field	Meaning	Example
apiVersion	Which API group/version defines this object	`v1`, `apps/v1`, `networking.k8s.io/v1`
kind	The type of object	`Pod`, `Deployment`, `Service`
metadata	Name, namespace, labels, annotations	`name: web`
spec	The desired state - different for every kind	containers, replicas, ports...

pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: web
  labels:
    app: web
    tier: frontend
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

Kubernetes adds a fifth section, **status**, which you never write - it is the actual state reported by the cluster. See it with `kubectl get pod web -o yaml`.

Pro tip

- Do not memorise YAML - generate it: `kubectl run web --image=nginx:1.27 --dry-run=client -o yaml > pod.yaml`
- Forgot a field name? `kubectl explain pod.spec.containers` and `kubectl explain pod.spec --recursive` are built-in documentation.

Common mistake

- YAML uses **spaces only** - a single tab breaks the file.
- Indentation is meaning: `ports` must be inside the container item, not beside `containers`.
- Using `latest` image tags makes deployments unpredictable - always pin a version.

2.4 Multi-container patterns

Pattern	What the extra container does	Example
Sidecar	Adds a helper feature alongside the main app	Log shipper, service-mesh proxy (Envoy)
Ambassador	Proxies outbound connections for the app	Local proxy to a remote database
Adapter	Converts the app's output into a standard format	Exporter turning logs into metrics
Init container	Runs to completion **before** app containers start	Wait for DB, download config, run migrations

sidecar.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: web-with-sidecar
spec:
  volumes:
    - name: logs
      emptyDir: {}
  initContainers:
    - name: wait-a-bit
      image: busybox:1.36
      command: ["sh", "-c", "echo preparing...; sleep 5"]
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "while true; do date >> /logs/app.log; sleep 2; done"]
      volumeMounts:
        - name: logs
          mountPath: /logs
    - name: log-shipper
      image: busybox:1.36
      command: ["sh", "-c", "tail -F /logs/app.log"]
      volumeMounts:
        - name: logs
          mountPath: /logs
```

Did you know?
Since Kubernetes 1.33, **native sidecars** are stable: an init container with `restartPolicy: Always` starts before the app and keeps running beside it. This fixes old problems such as Jobs never finishing because the sidecar kept running.

2.5 Pod lifecycle

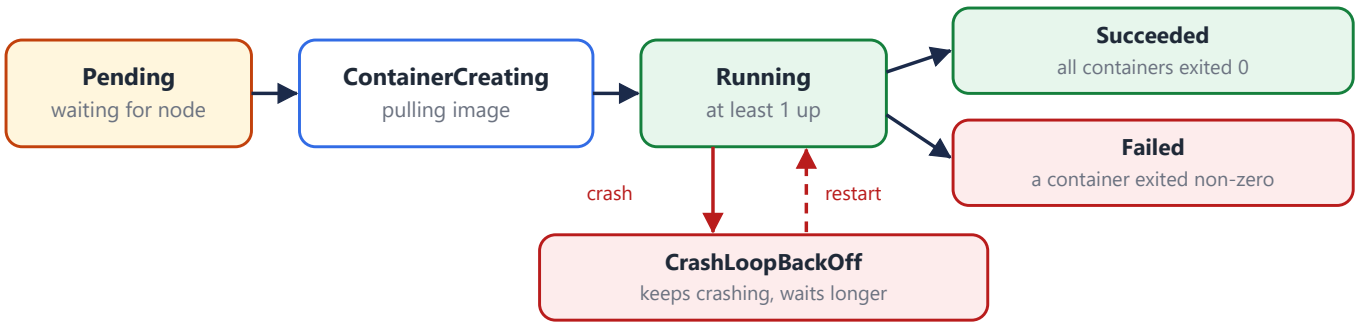


Figure 2.2 - Pod phases and the CrashLoopBackOff state

Phase / state	Meaning
Pending	Accepted, but not running yet - waiting for scheduling or image download.
Running	Bound to a node and at least one container is running.
Succeeded	All containers exited with code 0 and will not restart (typical for Jobs).
Failed	All containers stopped and at least one exited with an error.
Unknown	The node stopped reporting, so the state cannot be read.

Phase / state	Meaning
CrashLoopBackOff	Not a phase but a waiting reason: the container keeps crashing and Kubernetes waits longer (10s, 20s, 40s ... up to 5 min) before each restart.

The pod's `restartPolicy` decides what the kubelet does when a container exits: **Always** (default, for long-running apps), **OnFailure** (Jobs), or **Never**.

2.6 Labels, selectors and annotations

Labels are key-value tags on objects, such as `app=orders`, `env=prod`, `version=v2`. **Selectors** find objects by label. This is how a Deployment knows which pods it owns and how a Service knows where to send traffic - not by name, but by label.

> **Everyday example**
> In a supermarket, items carry stickers: "organic", "discount", "dairy". The store manager can say "move all items with discount AND dairy to aisle 3" without knowing each product name. Labels are the stickers; selectors are the manager's instruction.

PowerShell

```
kubectl get pods --show-labels
kubectl get pods -l app=web                    # equality
kubectl get pods -l 'env in (dev,qa)'          # set-based
kubectl label pod web env=dev                  # add a label
kubectl label pod web env-                     # remove it
```

Annotations are also key-value pairs but are *not* used for selection. They hold extra information for tools and humans: build number, Git commit, contact e-mail, or settings read by an ingress controller.

2.7 Live lab: the kubectl toolbox

PowerShell

```
kubectl apply -f pod.yaml
kubectl get pods -o wide                # IP and node
kubectl describe pod web                # full details + Events at the bottom
kubectl logs web                        # container output
kubectl exec -it web -- sh              # shell inside (type exit to leave)
kubectl port-forward pod/web 8080:80    # open http://localhost:8080 in a browser

kubectl apply -f sidecar.yaml
kubectl get pod web-with-sidecar -w     # watch Init:0/1 -> PodInitializing -> Running
kubectl logs web-with-sidecar -c log-shipper -f
kubectl exec -it web-with-sidecar -c app -- cat /logs/app.log

kubectl delete -f pod.yaml -f sidecar.yaml
```

Now break things on purpose and read the errors:

PowerShell

```
kubectl run broken --image=nginx:doesnotexist
kubectl get pods            # ErrImagePull, then ImagePullBackOff
kubectl describe pod broken # read the Events section

kubectl run crashy --image=busybox:1.36 -- sh -c "exit 1"
kubectl get pods -w         # Error -> CrashLoopBackOff
kubectl delete pod broken crashy
```

Command	What it is for
`kubectl get <kind>`	List objects; add `-o wide`, `-o yaml`, `-w`, `-A` (all namespaces)
`kubectl describe <kind> <name>`	Human-readable details and Events - first stop for debugging
`kubectl logs <pod> [-c ctr] [-f] [--previous]`	Container logs; `--previous` shows the crashed instance
`kubectl exec -it <pod> -- sh`	Run a command inside a container
`kubectl port-forward`	Tunnel a local port to a pod or service
`kubectl apply / delete -f`	Create/update or delete from files
`kubectl explain <path>`	Built-in field documentation

Common mistake

- Creating bare pods for real apps. A bare pod is **not** recreated if its node dies - use a Deployment (Class 3).
- Expecting to change most fields of a running pod. Pod specs are largely immutable - delete and recreate, or let a Deployment do it.

Class 2 recap

- Pod = smallest unit; one IP shared by its containers.
- YAML = apiVersion, kind, metadata, spec.
- Init containers run first, in order; sidecars run alongside.
- Labels + selectors connect objects together.
- Debug flow: get -> describe -> logs -> exec.

Class 2 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is a pod?

The smallest deployable unit in Kubernetes: a group of one or more containers that share a network namespace (one IP), can share volumes, and are always scheduled together on the same node.

Q2. Why does Kubernetes use pods instead of running containers directly?

Pods let tightly coupled containers share network and storage and be managed as one unit. They also give Kubernetes a runtime-neutral abstraction for scheduling, IP assignment and lifecycle.

Q3. Can a pod have more than one container? When would you do that?

Yes. Use it when containers must share resources and lifecycle - for example a sidecar log shipper, a service-mesh proxy, or an adapter. Unrelated services should be in separate pods so they scale independently.

Q4. How do containers in the same pod communicate?

Over `localhost` because they share the network namespace, and through shared volumes such as emptyDir. They must use different ports.

Q5. What is an init container?

A container that runs to completion before the app containers start. Multiple init containers run in order. Used to wait for dependencies, prepare files, or run database migrations.

Q6. What are sidecar, ambassador and adapter patterns?

Sidecar adds a helper feature (logging, proxy). Ambassador proxies the app's outbound connections. Adapter standardises the app's output (for example exposing metrics in Prometheus format).

Q7. What are the four required top-level fields in a manifest?

`apiVersion`, `kind`, `metadata` and `spec`. Kubernetes adds `status`, which reflects the actual state.

Q8. What is the difference between kubectl create and kubectl apply?

`create` is imperative and fails if the object exists. `apply` is declarative: it creates the object if missing or updates it to match the file, tracking the last applied configuration.

Q9. Explain the pod lifecycle phases.

Pending (accepted, not yet running), Running (on a node, at least one container running), Succeeded (all containers exited 0), Failed (at least one exited with error), and Unknown (node not reporting).

Q10. What is CrashLoopBackOff and how do you troubleshoot it?

The container keeps crashing and Kubernetes restarts it with growing delays. Check `kubectl logs <pod> --previous`, `kubectl describe pod` (exit code, OOMKilled, events), the command, environment variables, config files and probes.

Q11. What is ImagePullBackOff?

The kubelet cannot pull the image - usually a wrong name or tag, a private registry without imagePullSecrets, or no network access. `kubectl describe pod` shows the exact error.

Q12. What restart policies exist for pods?

Always (default; used by Deployments), OnFailure (restart only on non-zero exit; used by Jobs) and Never.

Q13. What are labels and selectors?

Labels are key-value tags on objects. Selectors query objects by labels - equality-based (`app=web`) or set-based (`env in (dev,qa)`). Deployments, Services and NetworkPolicies use selectors to find pods.

Q14. What is the difference between labels and annotations?

Labels identify and select objects and have size/format limits. Annotations store non-identifying metadata for tools and humans (build info, ingress settings) and cannot be used in selectors.

Q15. How do you get a shell inside a running container?

`kubectl exec -it <pod> -- sh` (or bash). For multi-container pods add `-c <container>`. If the image has no shell, use `kubectl debug` with an ephemeral container.

Q16. How do you view logs of a specific container or a previous crash?

`kubectl logs <pod> -c <container>`; add `-f` to follow and `--previous` to see the last terminated instance.

Q17. How can you generate YAML quickly without writing it by hand?

Use `--dry-run=client -o yaml`, e.g. `kubectl run web --image=nginx --dry-run=client -o yaml > pod.yaml`, and use `kubectl explain` to discover fields.

Q18. Are pods permanent? What happens if a node dies?

No, pods are ephemeral. A bare pod on a dead node is lost. Controllers like Deployments create replacement pods with new names and IPs, which is why we never run bare pods in production.

Q19. What is a static pod?

A pod managed directly by the kubelet from a manifest file on the node (usually /etc/kubernetes/manifests), not by the API server. Control-plane components in kubeadm clusters run as static pods.

Q20. What is kubectl port-forward used for?

It tunnels a local port to a pod or service, e.g. `kubectl port-forward pod/web 8080:80`, so you can test an app from your laptop without exposing it publicly.

K8S · CLASS 03 · 60-90 minutes

ReplicaSets and Deployments

By the end you can deploy, scale, update and roll back an application with zero downtime, and explain blue-green and canary releases.

What you will learn today

- ReplicaSet - keeping N copies alive
- Deployment - the object you use every day
- Scaling up and down
- Rolling updates: maxSurge and maxUnavailable; the Recreate strategy
- Rollout history, change-cause and rollback
- Release strategies: blue-green and canary
- Lab: upgrade nginx, break it with a bad image, roll back

Part	Topic	Time
1	Story, ReplicaSet and Deployment	15 min
2	Rolling update and rollback	20 min
3	Blue-green and canary	10 min
4	Live lab	25 min
5	Recap and interview questions	10 min

3.1 From pods to ReplicaSets

Ravi's story

Ravi's lunch-time rush needs 3 copies of the menu service. He created 3 pods by hand. At 1 PM a node rebooted and two pods vanished - nothing brought them back. Worse, releasing a new menu version meant deleting and recreating pods, and customers saw errors. He needed something that *keeps* 3 copies alive and swaps versions safely.

A **ReplicaSet** ensures that a specified number of identical pods are running at all times. It uses a label selector to count its pods and a pod template to create new ones.

A **Deployment** manages ReplicaSets for you and adds versioned updates and rollback. You almost never create a ReplicaSet directly - you create a Deployment.

Everyday example

A restaurant manager is told "always keep 3 cooks on the shift". If one goes home sick, the manager calls in a replacement (ReplicaSet). When the restaurant introduces a new uniform, the manager swaps cooks *one at a time* so the kitchen never closes, and keeps the old uniforms in case customers complain (Deployment with rollback).

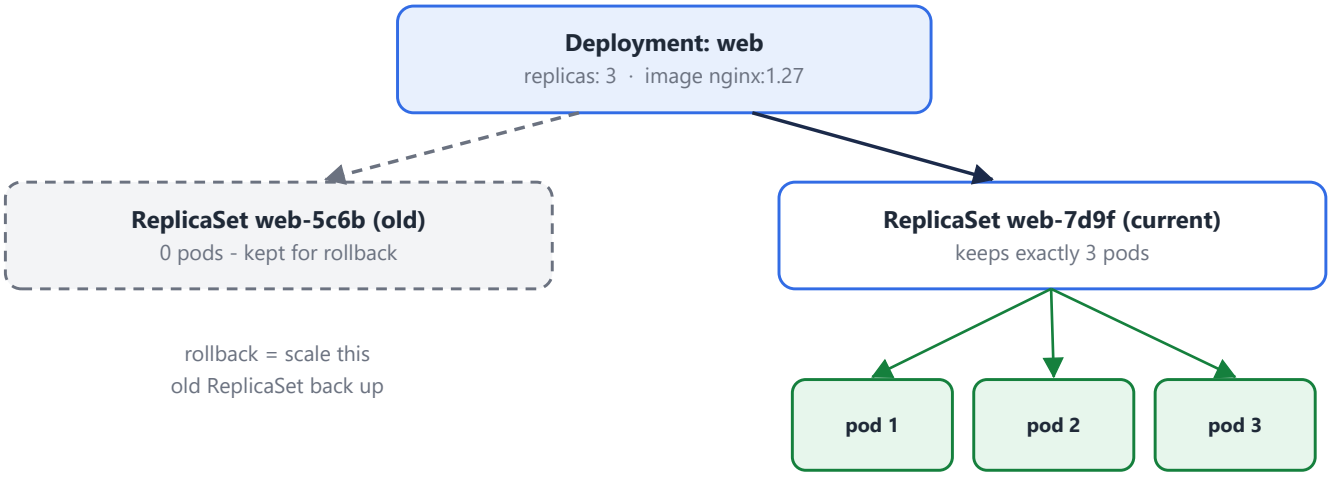


Figure 3.1 - A Deployment owns ReplicaSets, which own Pods

deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app: web
spec:
  replicas: 3
  revisionHistoryLimit: 10        # old ReplicaSets kept for rollback
  selector:
    matchLabels:
      app: web                    # must match the template labels
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1                 # at most 1 extra pod during update
      maxUnavailable: 0           # never go below 3 ready pods
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.26
          ports:
            - containerPort: 80
```

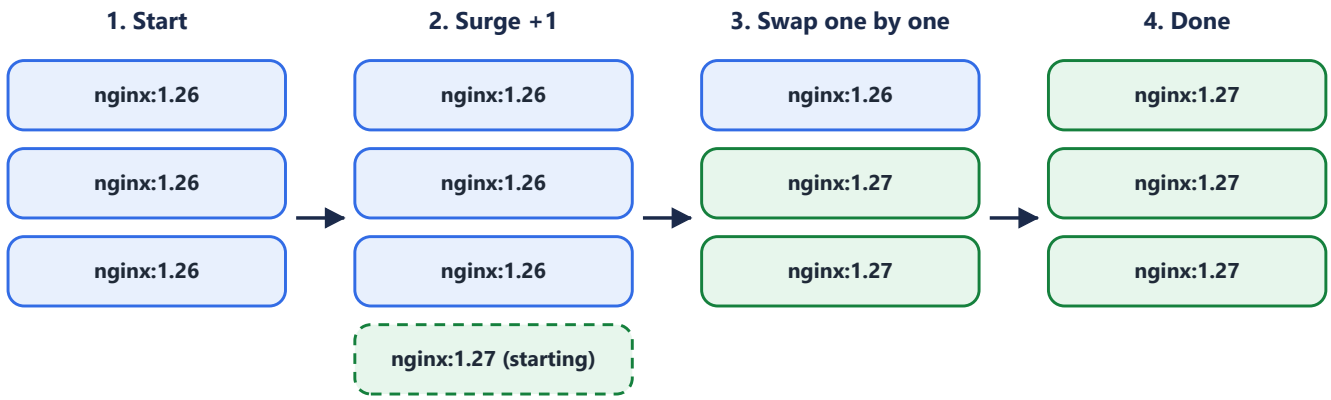
Common mistake
The `selector.matchLabels` must match `template.metadata.labels`. If they do not, the API server rejects the Deployment. The selector cannot be changed after creation.

3.2 Rolling updates

When you change the pod template (for example the image), the Deployment creates a **new ReplicaSet** and gradually moves pods from old to new. Two settings control the speed:

Setting	Meaning	Default
maxSurge	How many pods above `replicas` may exist during the update	25%

Setting	Meaning	Default
maxUnavailable	How many pods below `replicas` may be unavailable during the update	25%



maxSurge: 1 (one extra pod allowed) · maxUnavailable: 0 (never below 3 ready)

Figure 3.2 - Rolling update from nginx:1.26 to nginx:1.27 with maxSurge 1, maxUnavailable 0

The other strategy, **Recreate**, kills all old pods first and then starts new ones. It causes downtime but is needed when two versions cannot run side by side (for example a database schema that only one version understands).

> **Everyday example**
> Changing the tyres of a bus while passengers ride: you replace one tyre at a time (rolling update). Recreate is parking the bus at a garage, removing all tyres, then fitting new ones - simpler, but nobody travels in the meantime.

3.3 Rollout history and rollback

Each template change creates a **revision**. Old ReplicaSets are kept (scaled to 0), so rolling back is fast - Kubernetes just scales the old ReplicaSet up again.

Rollout commands

```
kubectl rollout status deployment/web
kubectl rollout history deployment/web
kubectl rollout history deployment/web --revision=2
kubectl rollout undo deployment/web                  # previous revision
kubectl rollout undo deployment/web --to-revision=1
kubectl rollout pause deployment/web                 # batch several changes
kubectl rollout resume deployment/web
kubectl rollout restart deployment/web               # restart pods, same spec
```

To record *why* a revision happened, set the `kubernetes.io/change-cause` annotation; it appears in `rollout history`.

3.4 Release strategies

Strategy	How it works	Pros / cons
Rolling update	Replace pods gradually (built into Deployment)	Zero downtime, no extra cost; both versions live briefly
Recreate	Stop all old, then start new	Simple; causes downtime
Blue-green	Run full v1 (blue) and v2 (green); switch the Service selector at once	Instant switch and rollback; needs double resources
Canary	Send a small share of traffic (e.g. 10%) to v2, watch metrics, then increase	Lowest risk; needs traffic splitting (replica ratio, Ingress, service mesh, Argo Rollouts)

Everyday example
A canary release is like a restaurant offering a new dish to a few tables first. If those diners love it, it goes on the main menu; if not, only a few people were affected. The name comes from coal miners who carried canaries to detect gas early.

3.5 Live lab: deploy, update, break, roll back

PowerShell - part 1

```
kubectl apply -f deploy.yaml
kubectl get deploy,rs,pods -l app=web

# scale
kubectl scale deployment web --replicas=5
kubectl get pods -l app=web

# rolling update - watch in a second window: kubectl get pods -l app=web -w
kubectl set image deployment/web nginx=nginx:1.27
kubectl annotate deployment/web kubernetes.io/change-cause="upgrade to nginx 1.27"
kubectl rollout status deployment/web
kubectl get rs -l app=web           # old RS has 0 pods, new RS has 5
```

PowerShell - part 2

```
# release a broken version
kubectl set image deployment/web nginx=nginx:9.99-broken
kubectl annotate deployment/web kubernetes.io/change-cause="bad release" --overwrite
kubectl get pods -l app=web         # new pods: ImagePullBackOff, old pods still serve!
kubectl rollout status deployment/web --timeout=30s

# roll back
kubectl rollout history deployment/web
kubectl rollout undo deployment/web
kubectl get pods -l app=web         # all healthy again on nginx:1.27
kubectl describe deployment web | Select-String Image
```

Notice that during the broken release, users were **not** affected: maxUnavailable kept the old pods serving until new ones became ready - which never happened.

Try it yourself
- Change `replicas` in deploy.yaml and run `kubectl apply -f deploy.yaml` again - this is the declarative way to scale.
- Try a simple blue-green: create `web-blue` and `web-green` Deployments with labels `version: blue/green`, then switch a Service selector between them after Class 4.

Common mistake

- Using `kubectl set image` in production and forgetting to update the YAML in Git - the next `apply` reverts your change. Keep Git as the source of truth.
- Thinking `kubectl scale` changes the history. Only pod template changes create revisions.

Class 3 recap

- ReplicaSet keeps N pods; Deployment manages ReplicaSets.
- Rolling update tuned by maxSurge / maxUnavailable.
- Rollback = scale an old ReplicaSet back up (`rollout undo`).
- Blue-green switches all traffic; canary shifts a little at a time.

Class 3 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is a ReplicaSet?

A controller that ensures a specified number of pod replicas matching a label selector are running. It creates pods from its template when there are too few and deletes them when there are too many.

Q2. What is a Deployment?

A higher-level object that manages ReplicaSets to provide declarative updates, rolling updates, rollback, scaling and pause/resume for stateless applications.

Q3. What is the difference between a ReplicaSet and a Deployment?

A ReplicaSet only maintains a pod count. A Deployment manages ReplicaSets and adds versioned rollouts, history and rollback. You normally create Deployments, not ReplicaSets.

Q4. What was a ReplicationController?

The older predecessor of ReplicaSet. It supported only equality-based selectors; ReplicaSet adds set-based selectors. It is considered legacy.

Q5. How does a Deployment perform a rolling update?

When the pod template changes, it creates a new ReplicaSet and scales it up while scaling the old one down, respecting maxSurge and maxUnavailable and waiting for new pods to become ready.

Q6. Explain maxSurge and maxUnavailable.

maxSurge is how many pods above the desired count may exist during an update; maxUnavailable is how many may be unavailable. Both default to 25%. maxSurge 1 / maxUnavailable 0 means one extra pod at a time and never fewer ready pods than desired.

Q7. What is the Recreate strategy and when would you use it?

It terminates all old pods before creating new ones, causing downtime. Use it when two versions cannot run simultaneously, such as incompatible schema changes or a single-writer volume.

Q8. How do you roll back a Deployment?

`kubectl rollout undo deployment/<name>` for the previous revision or `--to-revision=N` for a specific one. Kubernetes scales the old ReplicaSet back up.

Q9. How do you check rollout status and history?

`kubectl rollout status deployment/<name>` and `kubectl rollout history deployment/<name>`, with `--revision=N` for details of one revision.

Q10. What triggers a new revision?

Only changes to the pod template (`spec.template`) such as image, env, labels or resources. Scaling replicas does not create a revision.

Q11. What is revisionHistoryLimit?

The number of old ReplicaSets kept for rollback (default 10). Setting it to 0 disables rollback.

Q12. How do you scale a Deployment?

Imperatively with `kubectl scale deployment/<name> --replicas=N`, declaratively by changing `replicas` and running `kubectl apply`, or automatically with a HorizontalPodAutoscaler.

Q13. How do you restart all pods of a Deployment without changing the image?

`kubectl rollout restart deployment/<name>`. It adds a timestamp annotation to the template, triggering a rolling restart - useful after changing a ConfigMap used as env variables.

Q14. What happens if you deploy an image that does not exist?

New pods go into ErrImagePull/ImagePullBackOff and never become ready. With a sensible maxUnavailable the old pods keep serving traffic; the rollout stalls until you fix or undo it.

Q15. What is progressDeadlineSeconds?

How long a Deployment may go without progress before it is reported as failed (default 600 seconds). CI/CD pipelines use this through `kubectl rollout status` to detect bad releases.

Q16. Explain blue-green deployment in Kubernetes.

Run two full environments - blue (current) and green (new). Test green, then switch the Service selector from blue to green so all traffic moves at once. Rollback is switching back. It needs twice the resources.

Q17. Explain canary deployment.

Release the new version to a small share of users first, monitor errors and latency, and increase gradually. In plain Kubernetes this can be done with replica ratios behind one Service; finer control uses Ingress weights, a service mesh, or Argo Rollouts / Flagger.

Q18. Why must the Deployment selector match the template labels?

The selector tells the ReplicaSet which pods it owns. If template labels did not match, created pods would not be counted and the controller would create pods forever, so the API rejects it.

Q19. What does kubectl rollout pause do?

It stops the Deployment from rolling out template changes, letting you make several edits and then `rollout resume` to apply them as a single rollout.

Q20. Are Deployments suitable for databases?

Usually not. Deployment pods are interchangeable with random names and shared storage. Databases need stable identity and per-pod storage, which StatefulSets provide (Class 7).

K8S · CLASS 04 · 60-90 minutes

Services and cluster networking

By the end you can expose apps inside and outside the cluster, explain every Service type, and describe how DNS, kube-proxy and CNI make pod networking work.

What you will learn today

- Why pod IPs cannot be trusted
- Services, selectors and Endpoints/EndpointSlices
- Service types: ClusterIP, NodePort, LoadBalancer, ExternalName, headless
- Ports: port, targetPort, nodePort
- Cluster DNS with CoreDNS
- The Kubernetes networking model, kube-proxy and CNI plugins
- Lab: load balancing across pods, NodePort and LoadBalancer on minikube

Part	Topic	Time
1	Story and why Services exist	10 min
2	Service types and ports	20 min
3	DNS, kube-proxy, CNI	15 min
4	Live lab	25 min
5	Recap and interview questions	10 min

4.1 Why we need Services

Ravi's story

The orders service called the menu service at `10.244.1.5`. After a rolling update the menu pods got new IPs and every order failed with "connection refused". Ravi learned the hard way: **pod IPs change every time a pod is recreated**. He needed one address that never changes.

A **Service** gives a group of pods a stable virtual IP (ClusterIP) and a DNS name. It finds its pods using a **label selector** and load-balances traffic across the ones that are ready.

Everyday example

A restaurant prints one phone number on every menu card. Behind it, whichever staff member is free answers the call. Staff join and leave, but the number on the card never changes. The Service is that phone number; pods are the staff.

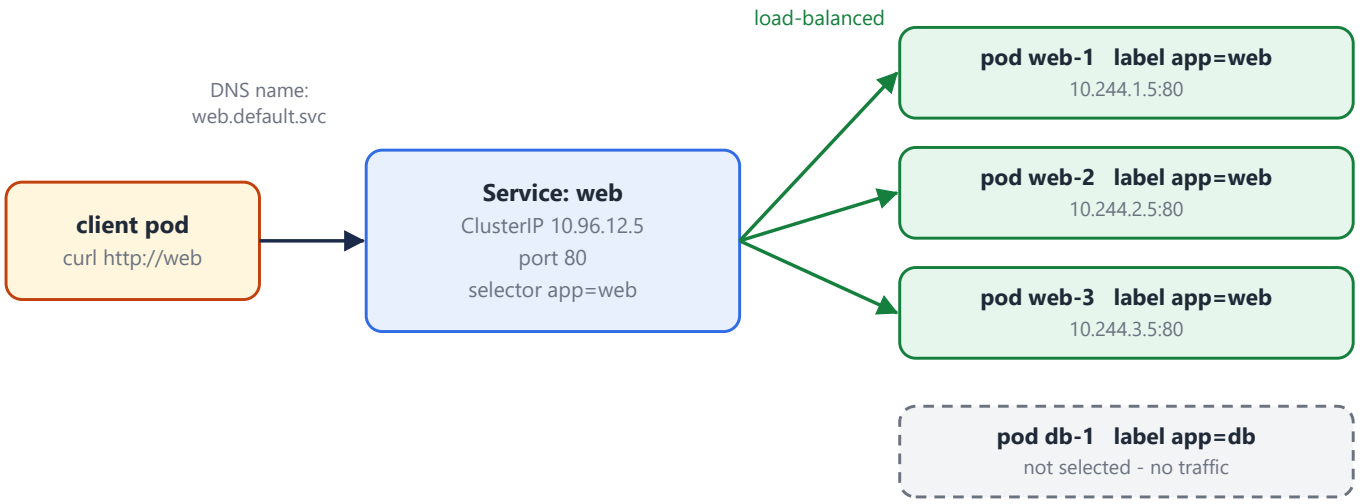


Figure 4.1 - A Service selects pods by label and spreads traffic across them

Behind the scenes Kubernetes keeps an **EndpointSlice** object listing the IP:port of every ready pod that matches the selector. If a Service has no endpoints, its selector or readiness is wrong - this is one of the most common bugs.

4.2 Ports explained

Field	Where	Example
port	The port the Service listens on (what clients call)	80
targetPort	The port on the container that receives traffic	8080
nodePort	Port opened on every node (NodePort/LoadBalancer only), range 30000-32767	30080
containerPort	Documentation in the pod spec - informational only	8080

4.3 Service types

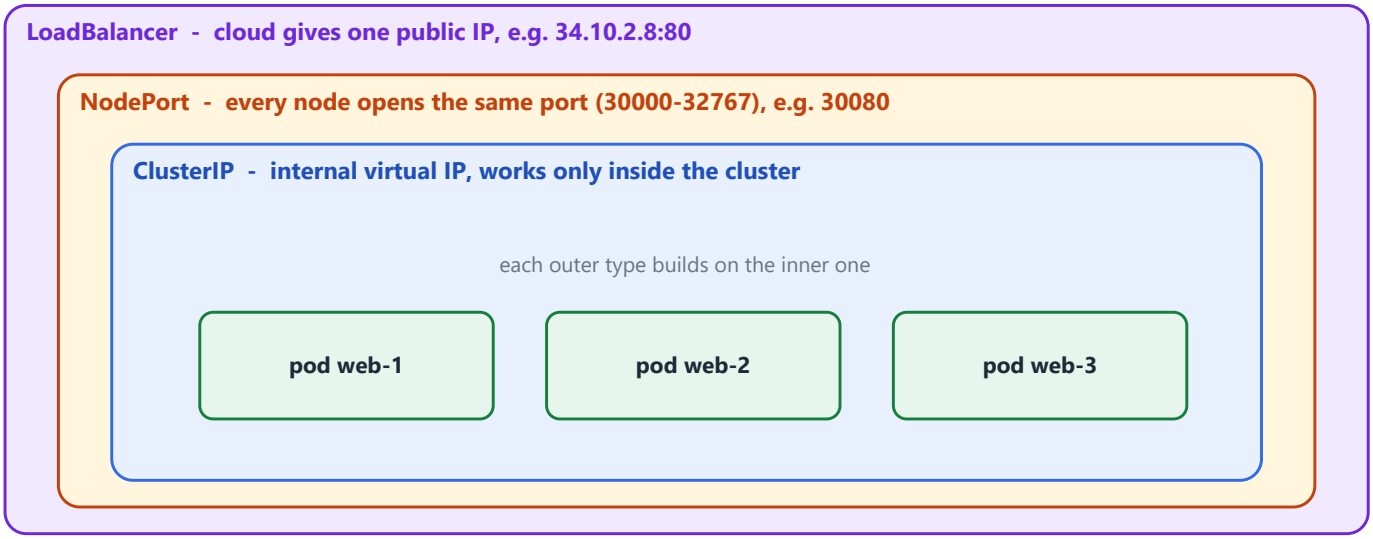


Figure 4.2 - Each Service type builds on the previous one

Type	Reachable from	Typical use
ClusterIP (default)	Inside the cluster only	Backend APIs, databases, internal microservices

Type	Reachable from	Typical use
NodePort	`<any-node-IP>:30000-32767`	Labs, on-prem setups, behind your own load balancer
LoadBalancer	Public/private IP from the cloud load balancer	Exposing an app directly on EKS/AKS/GKE
ExternalName	DNS alias (CNAME) to an outside hostname	Calling a managed DB like `db.example.com` by an internal name
Headless (`clusterIP: None`)	DNS returns pod IPs directly; no load balancing	StatefulSets, databases, clients that pick their own pod

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: ClusterIP          # change to NodePort or LoadBalancer
  selector:
    app: web
  ports:
    - port: 80             # service port
      targetPort: 80       # container port
      # nodePort: 30080    # only for NodePort/LoadBalancer
```

4.4 Cluster DNS

Every cluster runs **CoreDNS**. Each Service gets a DNS record:

```
<service>.<namespace>.svc.cluster.local

web                                  # from a pod in the same namespace
web.shop                             # from another namespace
web.shop.svc.cluster.local           # fully qualified
```

Pods find CoreDNS through `/etc/resolv.conf`, which the kubelet writes into every container.

4.5 How pod networking works

Kubernetes has a simple networking model with three rules:

- Every pod gets its own unique IP address.
- Pods can reach every other pod on any node without NAT.
- Agents on a node (like the kubelet) can reach all pods on that node.

Kubernetes itself does not implement this. A **CNI plugin** (Container Network Interface) does - Calico, Flannel, Cilium, Weave, or the cloud's own (AWS VPC CNI, Azure CNI). **kube-proxy** then implements Services by writing iptables/IPVS/nftables rules on every node that translate the Service IP into a real pod IP.

> **Everyday example**
> Think of an office campus. The CNI plugin builds the roads and gives every cabin a door number. kube-proxy is the receptionist with a rule book: "calls for Accounts go to cabin 12, 14 or 19 - pick any free one".

4.6 Live lab: Services in action

We use `traefik/whoami`, a tiny web app that replies with its own pod name - perfect for seeing load balancing.

PowerShell - ClusterIP

```
kubectl create deployment whoami --image=traefik/whoami --replicas=3
kubectl expose deployment whoami --port=80 --target-port=80
kubectl get svc whoami
kubectl get endpointslices -l kubernetes.io/service-name=whoami

# call it from inside the cluster several times
kubectl run client --rm -it --image=busybox:1.36 -- sh
  wget -qO- http://whoami | grep Hostname      # repeat 5 times - names change
  nslookup whoami
  exit
```

PowerShell - NodePort and LoadBalancer

```
kubectl expose deployment whoami --name=whoami-np --type=NodePort --port=80
kubectl get svc whoami-np               # note the 3xxxx port
minikube service whoami-np --url        # gives a localhost URL on Windows; keep it open
curl.exe <url-from-previous-command>

kubectl expose deployment whoami --name=whoami-lb --type=LoadBalancer --port=80
kubectl get svc whoami-lb               # EXTERNAL-IP is <pending>
minikube tunnel                         # in a separate admin window; leave running
kubectl get svc whoami-lb               # EXTERNAL-IP now 127.0.0.1
curl.exe http://127.0.0.1
```

Now break it: change the selector so it matches nothing and see the endpoints disappear.

PowerShell

```
kubectl set selector svc whoami app=wrong
kubectl get endpointslices -l kubernetes.io/service-name=whoami   # no endpoints
kubectl delete deployment whoami; kubectl delete svc whoami whoami-np whoami-lb
```

Try it yourself

- Scale whoami to 1 and back to 5 while looping requests - the Service adapts instantly.
- Run `kubectl get svc -A` to find the `kubernetes` Service, which points at the API server, and `kube-dns` in kube-system.

Common mistake

- Confusing `port` and `targetPort`; if the app listens on 8080, targetPort must be 8080.
- Labels typo in the selector: the Service is created fine but has zero endpoints.
- Expecting to ping a ClusterIP - it is virtual and only answers on its defined ports.

Class 4 recap

- Services give pods a stable IP + DNS name and load-balance.
- ClusterIP < NodePort < LoadBalancer; ExternalName = DNS alias; headless = no IP.
- DNS: service.namespace.svc.cluster.local (CoreDNS).
- CNI gives pods IPs; kube-proxy routes Service traffic.

Class 4 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is a Kubernetes Service and why is it needed?

An abstraction that gives a set of pods a stable virtual IP and DNS name and load-balances traffic to them. It is needed because pod IPs change whenever pods are recreated or rescheduled.

Q2. How does a Service know which pods to send traffic to?

Through its label selector. The EndpointSlice controller keeps a list of ready pods matching the selector, and kube-proxy routes traffic to those IPs.

Q3. What are the Service types?

ClusterIP (internal, default), NodePort (opens a port 30000-32767 on every node), LoadBalancer (provisions a cloud load balancer), ExternalName (DNS CNAME to an external name). A headless Service is a ClusterIP Service with `clusterIP: None`.

Q4. Difference between port, targetPort and nodePort?

port is what the Service listens on, targetPort is the container port traffic is forwarded to, and nodePort is the port opened on every node for NodePort/LoadBalancer Services.

Q5. What is a headless Service?

A Service with `clusterIP: None`. There is no virtual IP or load balancing; DNS returns the pod IPs directly. StatefulSets use it to give each pod a stable DNS name.

Q6. What is an ExternalName Service?

A Service that maps an internal name to an external DNS name via a CNAME record, with no selector or proxying - for example pointing `db` to a managed database hostname.

Q7. What are Endpoints and EndpointSlices?

Objects listing the IP:port pairs of the pods backing a Service. EndpointSlices are the newer, scalable version that splits large lists into chunks. `kubectl get endpointslices` helps debug Services.

Q8. A Service is not reachable. How do you troubleshoot?

Check the Service has endpoints, compare selector with pod labels, confirm pods are Ready, verify port/targetPort, test from a pod with wget/curl by IP and by DNS name, check CoreDNS, and check any NetworkPolicies.

Q9. How does DNS work inside Kubernetes?

CoreDNS runs in kube-system and is exposed as the kube-dns Service. The kubelet configures every pod's /etc/resolv.conf to use it. Services resolve as `<svc>.<ns>.svc.cluster.local`.

Q10. How can a pod reach a Service in another namespace?

By using the namespace in the name, e.g. `http://orders.shop` or the full `orders.shop.svc.cluster.local`.

Q11. What is kube-proxy and which modes does it support?

A node component that implements Services by programming iptables, IPVS or nftables rules that translate Service IPs to pod IPs. Some CNIs like Cilium can replace kube-proxy with eBPF.

Q12. What is CNI? Name some plugins.

Container Network Interface - a standard for plugins that give pods IPs and connect them across nodes. Examples: Calico, Flannel, Cilium, Weave, AWS VPC CNI, Azure CNI.

Q13. Explain the Kubernetes networking model.

Every pod gets a unique IP; all pods can communicate with all other pods across nodes without NAT; node agents can reach pods on their node. The CNI plugin implements it.

Q14. How do you expose an application outside the cluster?

With a NodePort or LoadBalancer Service, or an Ingress/Gateway in front of ClusterIP Services. `kubectl port-forward` works for temporary local testing.

Q15. What is the difference between NodePort and LoadBalancer?

NodePort opens a fixed port on each node; clients must know a node IP. LoadBalancer additionally asks the cloud for an external load balancer with one stable IP that forwards to the node ports.

Q16. How does a LoadBalancer Service work on bare metal or minikube?

There is no cloud controller, so EXTERNAL-IP stays pending unless you add something like MetalLB, or on minikube run `minikube tunnel`.

Q17. What is session affinity in a Service?

Setting `sessionAffinity: ClientIP` sends requests from the same client IP to the same pod. Default is None.

Q18. Can a Service exist without a selector?

Yes. You then create Endpoints/EndpointSlices manually to point to IPs outside the cluster, such as a legacy database.

Q19. Why can't you ping a ClusterIP?

A ClusterIP is a virtual IP implemented by kube-proxy rules for specific ports and protocols. No interface owns it, so ICMP ping usually does not work; test with curl on the service port.

Q20. What is the `kubernetes` Service in the default namespace?

A built-in ClusterIP Service pointing to the API server, so pods can reach the API at `https://kubernetes.default.svc`.

K8S · CLASS 05 · 60-90 minutes

Ingress, Gateway API and Namespaces

By the end you can route many apps through one entry point using host and path rules, add TLS, and organise a cluster with namespaces, quotas and contexts.

What you will learn today

- Why one LoadBalancer per app does not scale
- Ingress resources and Ingress controllers
- Host-based and path-based routing, TLS termination
- Gateway API - the modern successor
- Namespaces: isolation, DNS, which objects are namespaced
- ResourceQuota and LimitRange
- kubeconfig, clusters, users and contexts
- Lab: route /menu and /orders behind quickbite.local

Part	Topic	Time
1	Story and Ingress concepts	15 min
2	Gateway API	5 min
3	Namespaces, quotas, contexts	20 min
4	Live lab	25 min
5	Recap and interview questions	10 min

5.1 Why Ingress?

Ravi's story

QuickBite exposed menu, orders and payment with three LoadBalancer Services. The cloud bill showed three load balancers, three public IPs and three SSL certificates to renew. Marketing wanted everything under one address: quickbite.com/menu, quickbite.com/orders. Ravi needed one smart front door.

An **Ingress** is a set of HTTP/HTTPS routing rules: "requests for host X and path Y go to Service Z". An **Ingress controller** is the actual reverse proxy (NGINX, Traefik, HAProxy, cloud load balancers) that reads those rules and does the routing. Without a controller, an Ingress object does nothing.

Everyday example

A shopping mall has one main entrance and a directory board: "Food court - level 3, Cinema - level 4, Electronics - level 1". Visitors use one door and are directed to the right shop. The Ingress is the directory board; the Ingress controller is the security guard at the entrance who reads it and points you the right way.

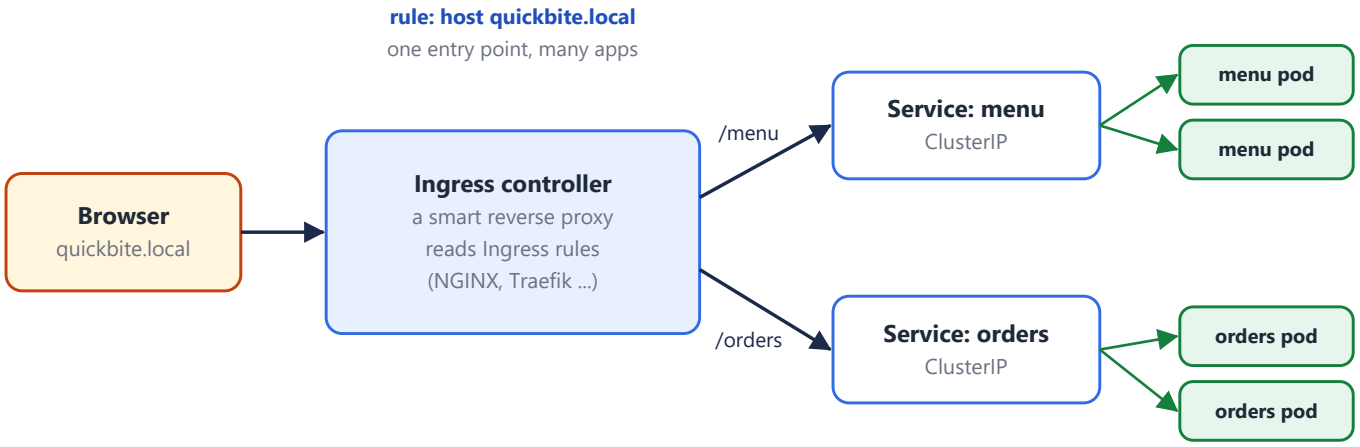


Figure 5.1 - One Ingress controller routes requests to several Services by path

ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: quickbite
spec:
  ingressClassName: nginx
  tls:                                # optional HTTPS
    - hosts: ["quickbite.local"]
      secretName: quickbite-tls
  rules:
    - host: quickbite.local
      http:
        paths:
          - path: /menu
            pathType: Prefix
            backend:
              service: { name: menu, port: { number: 80 } }
          - path: /orders
            pathType: Prefix
            backend:
              service: { name: orders, port: { number: 80 } }
```

Term	Meaning
Path-based routing	`/menu` to one Service, `/orders` to another
Host-based routing	`menu.quickbite.com` and `admin.quickbite.com` to different Services
pathType	`Prefix` (matches /menu/...), `Exact` (only /menu), `ImplementationSpecific`
TLS termination	The controller decrypts HTTPS using a certificate stored in a TLS Secret
IngressClass	Chooses which controller handles this Ingress when several are installed
cert-manager	A popular add-on that obtains and renews free Let's Encrypt certificates automatically

5.2 Gateway API - the next generation

Ingress only covers basic HTTP routing; advanced features ended up in controller-specific annotations. The **Gateway API** is the official successor with richer, portable objects:

- **GatewayClass** - which implementation (like IngressClass).
- **Gateway** - the actual listener (ports, TLS), usually owned by the platform team.
- **HTTPRoute / GRPCRoute / TCPRoute** - routing rules, owned by app teams; supports header matching and weighted traffic splitting for canaries.

Did you know?
The community **ingress-nginx** controller was retired in March 2026 and no longer receives fixes. Ingress objects still work with other controllers, but new projects should prefer the Gateway API or a maintained controller. minikube's ingress addon is still perfectly fine for learning the concepts.

5.3 Namespaces

A **namespace** is a virtual cluster inside a physical cluster. It groups objects, scopes names (two namespaces can each have a Service called `web`), and is the unit for quotas, RBAC permissions and network policies.

Everyday example
One office building, many floors. Finance on floor 2 and HR on floor 3 can both have a room called "Meeting Room A". Each floor has its own budget (quota) and access cards (RBAC), but they share the lifts, power and security (the cluster).

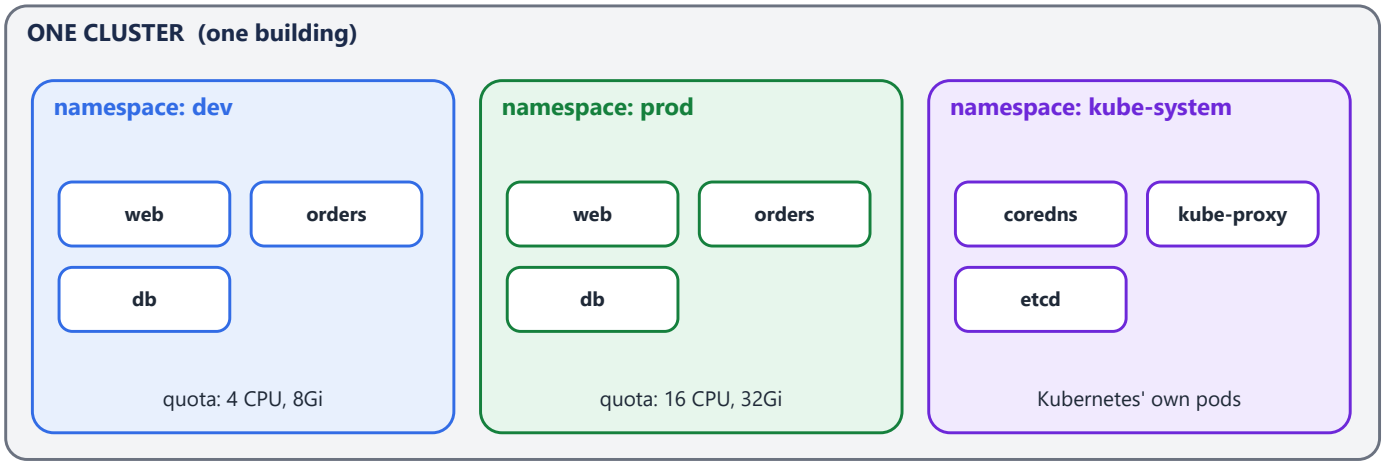


Figure 5.2 - Namespaces divide one cluster into isolated areas

Default namespace	Purpose
default	Where objects go if you do not specify a namespace
kube-system	Kubernetes' own components (CoreDNS, kube-proxy, control plane)
kube-public	Readable by everyone, rarely used
kube-node-lease	Node heartbeat Lease objects for fast failure detection

Not everything is namespaced. Nodes, PersistentVolumes, StorageClasses, ClusterRoles and Namespaces themselves are **cluster-scoped**. Check with `kubectl api-resources --namespaced=false`.

ResourceQuota and LimitRange

quota.yaml

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev
spec:
  hard:
    pods: "10"
    requests.cpu: "2"
    requests.memory: 4Gi
    limits.cpu: "4"
    limits.memory: 8Gi
---
apiVersion: v1
kind: LimitRange
metadata:
  name: dev-defaults
  namespace: dev
spec:
  limits:
    - type: Container
      default:          { cpu: 500m, memory: 256Mi }   # limit if none given
      defaultRequest:   { cpu: 100m, memory: 128Mi }   # request if none given
```

ResourceQuota caps the total a namespace may use. **LimitRange** sets default and min/max values for each container, so pods without resources still get sensible values.

kubeconfig and contexts

kubectl reads `~/.kube/config` (on Windows `C:\Users\<you>\.kube\config`). It lists **clusters** (API server addresses), **users** (credentials) and **contexts** (cluster + user + default namespace). Switching context switches which cluster you control.

```
kubectl config get-contexts
kubectl config current-context
kubectl config use-context minikube
kubectl config set-context --current --namespace=dev   # change default namespace
```

Common mistake
Running `kubectl delete` while your context points at production. Always check `kubectl config current-context` - many engineers show it in their shell prompt.

5.4 Live lab: one door, two apps

PowerShell - part 1

```
minikube addons enable ingress
kubectl get pods -n ingress-nginx          # wait until controller is Running

kubectl create namespace shop
kubectl config set-context --current --namespace=shop

kubectl create deployment menu   --image=hashicorp/http-echo -- /http-echo -text="Menu: biryani"
kubectl create deployment orders --image=hashicorp/http-echo -- /http-echo -text="Orders service OK"
kubectl expose deployment menu   --port=80 --target-port=5678
kubectl expose deployment orders --port=80 --target-port=5678
```

Save the Ingress from section 5.1 as `ingress.yaml`, remove the `tls` block for now, and apply it:

PowerShell - part 2

```
kubectl apply -f ingress.yaml
kubectl get ingress
minikube tunnel                          # separate admin window; keep it open

curl.exe -H "Host: quickbite.local" http://127.0.0.1/menu
curl.exe -H "Host: quickbite.local" http://127.0.0.1/orders
curl.exe -H "Host: quickbite.local" http://127.0.0.1/unknown     # 404 from controller
```

PowerShell - quotas

```
kubectl apply -f quota.yaml            # change namespace: dev -> shop first
kubectl describe quota -n shop
kubectl create deployment big --image=nginx:1.27 --replicas=15
kubectl get deploy big                 # only 10 pods allowed - see why:
kubectl describe rs -l app=big | Select-String quota

kubectl config set-context --current --namespace=default
kubectl delete namespace shop          # deletes everything inside it
```

Try it yourself

- Add `127.0.0.1 quickbite.local` to `C:\Windows\System32\drivers\etc\hosts` (as admin) and open http://quickbite.local/menu in your browser.
- Create a self-signed certificate with OpenSSL, store it with `kubectl create secret tls quickbite-tls --cert=tls.crt --key=tls.key`, add the tls block back and test https.

Class 5 recap

- Ingress = HTTP routing rules; Ingress controller = the proxy that applies them.
- Gateway API = modern, role-oriented successor of Ingress.
- Namespaces scope names, quotas, RBAC and policies.
- ResourceQuota caps totals; LimitRange sets per-container defaults.
- Context = cluster + user + namespace; always check it.

Class 5 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is an Ingress?

An API object that defines HTTP/HTTPS routing rules from outside the cluster to Services, based on host names and URL paths, with optional TLS termination.

Q2. What is an Ingress controller and why is it required?

The component that implements Ingress rules - a reverse proxy such as NGINX, Traefik, HAProxy or a cloud load balancer controller. An Ingress object has no effect unless a controller is running.

Q3. Ingress vs LoadBalancer Service?

A LoadBalancer Service exposes one Service at layer 4 and usually costs one cloud load balancer each. Ingress works at layer 7 and routes many hosts/paths to many Services through one entry point.

Q4. What is path-based vs host-based routing?

Path-based routes by URL path (/menu, /orders) on the same host; host-based routes by hostname (menu.example.com, admin.example.com).

Q5. Explain pathType values.

Prefix matches the path and anything below it split by `/`; Exact matches only the exact path; ImplementationSpecific leaves matching to the controller.

Q6. How do you enable HTTPS with Ingress?

Store the certificate and key in a Secret of type kubernetes.io/tls and reference it under `spec.tls` with the host names. cert-manager can issue and renew certificates automatically.

Q7. What is an IngressClass?

A resource that identifies which controller should handle an Ingress, referenced with `spec.ingressClassName`. It allows multiple controllers in one cluster.

Q8. What is the Gateway API?

The newer Kubernetes networking API that succeeds Ingress. It splits responsibilities into GatewayClass, Gateway and Routes (HTTPRoute, GRPCRoute, etc.), and natively supports header matching, traffic splitting and more protocols.

Q9. What is a namespace?

A logical partition of a cluster that scopes object names and is the boundary for RBAC, quotas, LimitRanges and NetworkPolicies.

Q10. Which namespaces exist by default?

default, kube-system, kube-public and kube-node-lease. Add-ons may create more, like ingress-nginx.

Q11. Are all objects namespaced?

No. Nodes, PersistentVolumes, StorageClasses, Namespaces, ClusterRoles, ClusterRoleBindings and CRDs are cluster-scoped. `kubectl api-resources --namespaced=false` lists them.

Q12. Do namespaces provide network isolation?

No, by default pods in different namespaces can talk to each other. Isolation requires NetworkPolicies and a CNI that enforces them.

Q13. What is a ResourceQuota?

A namespace-level limit on total resource consumption - CPU and memory requests/limits, number of pods, Services, PVCs and so on. New objects that would exceed it are rejected.

Q14. What is a LimitRange?

A namespace policy that sets default requests/limits and minimum/maximum values per container or pod, so workloads without resource settings still get sensible values.

Q15. What happens when you delete a namespace?

All objects inside it are deleted. It stays in Terminating until finalizers complete.

Q16. What is kubeconfig?

A YAML file (default ~/.kube/config) holding clusters, users and contexts that kubectl uses to connect and authenticate. The KUBECONFIG variable can point to other files.

Q17. What is a context in kubectl?

A named combination of cluster, user and default namespace. `kubectl config use-context` switches between them; `set-context --current --namespace=x` changes the default namespace.

Q18. How do you run a command against a namespace without switching?

Add `-n <namespace>` to the command, or `-A`/`--all-namespaces` to list across all.

Q19. Why would you use multiple namespaces instead of multiple clusters?

Namespaces are cheaper and simpler for separating teams or environments with quotas and RBAC. Separate clusters give stronger isolation and are preferred for production vs non-production or strict compliance.

Q20. The Ingress returns 404 or 503. How do you debug?

404: check host header and path rules, pathType and ingressClassName. 503: the backend Service has no ready endpoints - check selectors, ports and pod readiness. Also read the controller pod logs.

K8S · CLASS 06 · 60-90 minutes

ConfigMaps and Secrets

By the end you can keep configuration and passwords out of your images, inject them as environment variables or files, and explain how to keep Secrets truly secret.

What you will learn today

- Why configuration must not be baked into images
- ConfigMaps: create from literals, files and YAML
- Four ways to consume config: env, envFrom, volume, command arguments
- Secrets: types, base64, stringData, imagePullSecrets
- Updating config: what reloads automatically and what does not
- Immutable ConfigMaps/Secrets and the Downward API
- Securing Secrets: encryption at rest, RBAC, external secret managers

Part	Topic	Time
1	Story and ConfigMaps	15 min
2	Secrets and security	20 min
3	Updates, immutability, Downward API	10 min
4	Live lab	25 min
5	Recap and interview questions	10 min

6.1 Separate config from code

Ravi's story
The QuickBite orders image had the database password and the payment-gateway URL written inside it. To use a different database in testing, Ravi had to rebuild the image. Worse, anyone who pulled the image could read the production password. His team lead quoted the Twelve-Factor App rule: "Store config in the environment."

Build **one image** and run it everywhere - dev, test, prod - with different settings supplied at runtime. Kubernetes gives you two objects for that:

Object	Holds	Example values
ConfigMap	Non-sensitive settings, as key-value pairs or whole files	APP_MODE=prod, LOG_LEVEL=info, nginx.conf, menu.json
Secret	Sensitive data: passwords, tokens, keys, certificates	DB_PASSWORD, API keys, TLS cert, registry login

Everyday example
A ConfigMap is the **notice board** in the office kitchen - anyone can read today's menu and timings. A Secret is the **locker** with a key - only people who are given access can open it. Both can change without rebuilding the office.

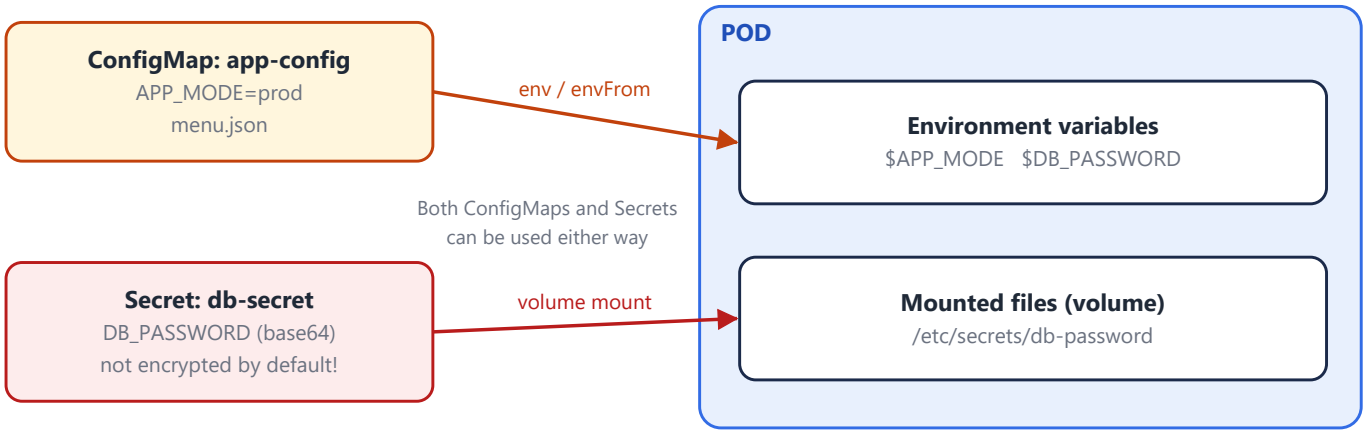


Figure 6.1 - ConfigMaps and Secrets reach a pod as environment variables or files

6.2 Creating ConfigMaps

Imperative

```
kubectl create configmap app-config --from-literal=APP_MODE=prod --from-literal=LOG_LEVEL=info
kubectl create configmap menu-file --from-file=menu.json
kubectl create configmap app-config --from-env-file=app.env --dry-run=client -o yaml
```

configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: prod
  LOG_LEVEL: info
  menu.json: |
    {"specials": ["biryani", "masala dosa"]}
```

6.3 Consuming config in a pod

config-demo.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: config-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "echo mode=$APP_MODE pass=$DB_PASSWORD; sleep 3600"]
      env:
        - name: APP_MODE                    # 1. a single key
          valueFrom:
            configMapKeyRef: { name: app-config, key: APP_MODE }
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef: { name: db-secret, key: password }
      envFrom:                              # 2. every key as a variable
        - configMapRef: { name: app-config }
      volumeMounts:                         # 3. keys as files
        - name: config-vol
          mountPath: /etc/config
        - name: secret-vol
          mountPath: /etc/secrets
          readOnly: true
  volumes:
    - name: config-vol
      configMap: { name: app-config }
    - name: secret-vol
      secret: { secretName: db-secret }
```

Method	Good for	Updates live?
`env` **with keyRef**	A few specific values	No - restart the pod
`envFrom`	Load all keys at once	No - restart the pod
Volume mount	Config files (nginx.conf, application.properties)	Yes, within about a minute (not with subPath)
Command args `$(VAR)`	Passing values as flags	No - restart the pod

6.4 Secrets

PowerShell

```
kubectl create secret generic db-secret `
  --from-literal=password='S3cr3t!' --from-literal=user=quickbite
kubectl get secret db-secret -o yaml           # values look scrambled... but are they?
kubectl get secret db-secret -o jsonpath='{.data.password}'
# decode in PowerShell:
[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String("UzNjcjN0IQ=="))
```

Common mistake

base64 is encoding, not encryption. Anyone who can read the Secret object can decode it in one line. Never commit Secret YAML with real values to Git.

Secret type	Used for
Opaque (generic)	Any user-defined data - the default
kubernetes.io/tls	TLS certificate and key, used by Ingress
kubernetes.io/dockerconfigjson	Private registry login, referenced as imagePullSecrets
kubernetes.io/service-account-token	Long-lived ServiceAccount tokens (legacy)
kubernetes.io/basic-auth, ssh-auth	Username/password, SSH keys

In YAML you can use `stringData` to write plain text and let Kubernetes encode it:

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
stringData:
  user: quickbite
  password: S3cr3t!
```

How to really protect Secrets

- **Encryption at rest** - configure the API server (EncryptionConfiguration or a cloud KMS) so Secrets are encrypted inside etcd. Managed clusters offer this as a setting.
- **RBAC** - give `get/list` on Secrets only to the ServiceAccounts that need them.
- **Mount as files** rather than env vars where possible; env vars leak into crash dumps and logs.
- **External secret managers** - HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, via the External Secrets Operator or the Secrets Store CSI driver.
- **Safe Git** - Sealed Secrets or SOPS encrypt Secret files so they can live in Git.

6.5 Immutable config and the Downward API

Setting `immutable: true` on a ConfigMap or Secret prevents accidental edits and reduces load on the API server (kubelets stop watching it). To change it, create a new one, e.g. `app-config-v2`, and update the Deployment - which also gives you a clean rollout and rollback.

The **Downward API** lets a pod read facts about itself - its name, namespace, IP, node name, labels and resource limits - as env vars or files, without calling the API server:

```
env:
  - name: POD_NAME
    valueFrom: { fieldRef: { fieldPath: metadata.name } }
  - name: NODE_NAME
    valueFrom: { fieldRef: { fieldPath: spec.nodeName } }
  - name: MEM_LIMIT
    valueFrom: { resourceFieldRef: { resource: limits.memory } }
```

6.6 Live lab

PowerShell

```
kubectl apply -f configmap.yaml
kubectl create secret generic db-secret `
  --from-literal=password='S3cr3t!' --from-literal=user=quickbite
kubectl apply -f config-demo.yaml
kubectl logs config-demo                      # mode=prod pass=S3cr3t!
kubectl exec config-demo -- env | Select-String APP_
kubectl exec config-demo -- ls /etc/config /etc/secrets
kubectl exec config-demo -- cat /etc/config/menu.json

# live update test
kubectl edit configmap app-config             # change LOG_LEVEL to debug, save
kubectl exec config-demo -- cat /etc/config/LOG_LEVEL   # wait ~60 s: becomes debug
kubectl exec config-demo -- printenv LOG_LEVEL          # still info - env never changes

kubectl delete pod config-demo; kubectl delete cm app-config; kubectl delete secret db-secret
```

Try it yourself

- Reference a ConfigMap that does not exist and look at the pod status: **CreateContainerConfigError**. Fix it by creating the ConfigMap.
- For a Deployment, run `kubectl rollout restart deployment/<name>` after changing a ConfigMap used as env vars.

Class 6 recap

- ConfigMap = plain settings; Secret = sensitive data (base64, not encrypted).
- Consume via env, envFrom, volume files or args.
- Volume files update live; env vars need a restart.
- Protect Secrets with encryption at rest, RBAC and external managers.
- Downward API exposes pod facts to the container.

Class 6 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is a ConfigMap?

An API object that stores non-confidential configuration as key-value pairs or files, so the same image can run with different configuration in different environments.

Q2. What is a Secret?

An object for small amounts of sensitive data such as passwords, tokens and keys. It is stored base64-encoded and can be protected with RBAC and encryption at rest.

Q3. What is the difference between a ConfigMap and a Secret?

Both inject configuration. Secrets are meant for sensitive data: they are base64-encoded, can be encrypted at rest, are mounted on tmpfs in memory on nodes, and are usually restricted by RBAC.

Q4. Are Kubernetes Secrets secure by default?

Not fully. They are only base64-encoded and stored unencrypted in etcd unless encryption at rest is enabled. Anyone with read access to Secrets or etcd can see them.

Q5. How can you make Secrets more secure?

Enable encryption at rest with a KMS provider, limit access with RBAC, avoid committing them to Git (use Sealed Secrets or SOPS), prefer file mounts over env vars, and use external managers like Vault or cloud secret stores through External Secrets Operator or the CSI driver.

Q6. What are the ways to use a ConfigMap in a pod?

As individual env vars (configMapKeyRef), all keys as env vars (envFrom), as files in a volume, or as command-line arguments via env var substitution.

Q7. If you update a ConfigMap, does the pod see the change?

Volume-mounted files update automatically after a short delay (except subPath mounts). Environment variables do not change until the pod restarts, e.g. via `kubectl rollout restart`.

Q8. What is the difference between data and stringData in a Secret?

`data` expects base64-encoded values; `stringData` accepts plain text and the API server encodes it. stringData is write-only convenience and is merged into data.

Q9. What Secret types exist?

Opaque, kubernetes.io/tls, kubernetes.io/dockerconfigjson, kubernetes.io/basic-auth, kubernetes.io/ssh-auth, kubernetes.io/service-account-token and bootstrap tokens.

Q10. How do you pull an image from a private registry?

Create a docker-registry Secret (`kubectl create secret docker-registry ...`) and reference it in the pod spec under `imagePullSecrets`, or attach it to the ServiceAccount.

Q11. What is an immutable ConfigMap or Secret?

One with `immutable: true`. Its data cannot be changed, which prevents accidental updates and reduces API server watch load. To change it you create a new object and update references.

Q12. What is the size limit of a ConfigMap or Secret?

1 MiB each, because they are stored in etcd. Larger data should go in volumes or external storage.

Q13. What happens if a pod references a missing ConfigMap?

The container cannot start and shows CreateContainerConfigError, unless the reference is marked `optional: true`.

Q14. What is the Downward API?

A way to expose pod and container metadata - name, namespace, IP, node name, labels, annotations, resource requests and limits - to containers as env vars or files.

Q15. How do you create a ConfigMap from a file?

`kubectl create configmap <name> --from-file=<path>`; the file name becomes the key and its content the value. `--from-env-file` turns each line into a key.

Q16. Why should you prefer mounting Secrets as files over env vars?

Env vars can leak through logs, crash reports, `kubectl describe` output or child processes, and they cannot be rotated without a restart. Files on tmpfs can be updated and have tighter permissions.

Q17. How do you decode a Secret value?

`kubectl get secret <name> -o jsonpath='{.data.<key>}'` then base64-decode it. This shows why base64 is not encryption.

Q18. What is the External Secrets Operator?

A controller that reads secrets from external systems such as AWS Secrets Manager, Azure Key Vault, GCP Secret Manager or Vault and creates/updates Kubernetes Secrets automatically.

Q19. What are Sealed Secrets?

A Bitnami tool where you encrypt a Secret with the cluster's public key into a SealedSecret that is safe to store in Git; only the controller in the cluster can decrypt it.

Q20. How do you trigger a rollout when a ConfigMap changes?

Run `kubectl rollout restart`, or put a hash of the ConfigMap in a pod-template annotation (Helm and Kustomize do this) so any change modifies the template and triggers a rolling update.

K8S · CLASS 07 · 60-90 minutes

Storage, PersistentVolumes and StatefulSets

By the end you can give pods durable storage, explain PV/PVC/StorageClass, and run a stateful app whose pods keep their names and their data.

What you will learn today

- Why container storage disappears
- Volume types: emptyDir, hostPath, configMap/secret, CSI
- PersistentVolume, PersistentVolumeClaim and binding
- StorageClass and dynamic provisioning
- Access modes, reclaim policies, volume expansion and snapshots
- StatefulSets: stable names, ordered startup, per-pod storage
- Lab: a StatefulSet that keeps its data after pod deletion

Part	Topic	Time
1	Story and volumes	10 min
2	PV, PVC, StorageClass	20 min
3	StatefulSets	15 min
4	Live lab	25 min
5	Recap and interview questions	10 min

7.1 The disappearing data problem

Ravi's story

Ravi ran MySQL as a Deployment. A node was patched and rebooted on Sunday, the MySQL pod was recreated on another node... with an empty database. All test orders were gone. Container file systems are temporary; the data had lived and died with the container.

A container's writable layer is deleted when the container is. Kubernetes **volumes** attach storage to a pod, and **persistent** volumes keep data beyond the life of any pod.

Volume type	Lifetime	Use
emptyDir	As long as the pod (survives container restarts)	Scratch space, cache, sharing files between containers
hostPath	As long as the node	Node agents reading /var/log; avoid for apps - ties pod to a node and is a security risk
configMap / secret / downwardAPI	Managed by Kubernetes	Config files (Class 6)
persistentVolumeClaim	Independent of pods	Databases, uploads, anything that must survive
CSI drivers	Defined by the storage system	AWS EBS/EFS, Azure Disk/Files, GCE PD, Ceph, NFS

7.2 PersistentVolume and PersistentVolumeClaim

- **PersistentVolume (PV)** - a piece of real storage in the cluster, created by an admin or automatically. Cluster-scoped.
- **PersistentVolumeClaim (PVC)** - a request for storage by a user: "I need 1Gi, read-write by one node". Namespaced. Kubernetes **binds** the claim to a matching PV.
- **Pod** - refers only to the PVC by name. It never needs to know where the disk really is.

Everyday example
A gym has lockers (PVs) of different sizes. A member fills a request slip: "one medium locker" (PVC). The front desk assigns a locker that matches and gives the member the key. The member only remembers "my locker slip", not the locker's serial number. If the gym builds lockers on demand when a slip arrives, that is a StorageClass.

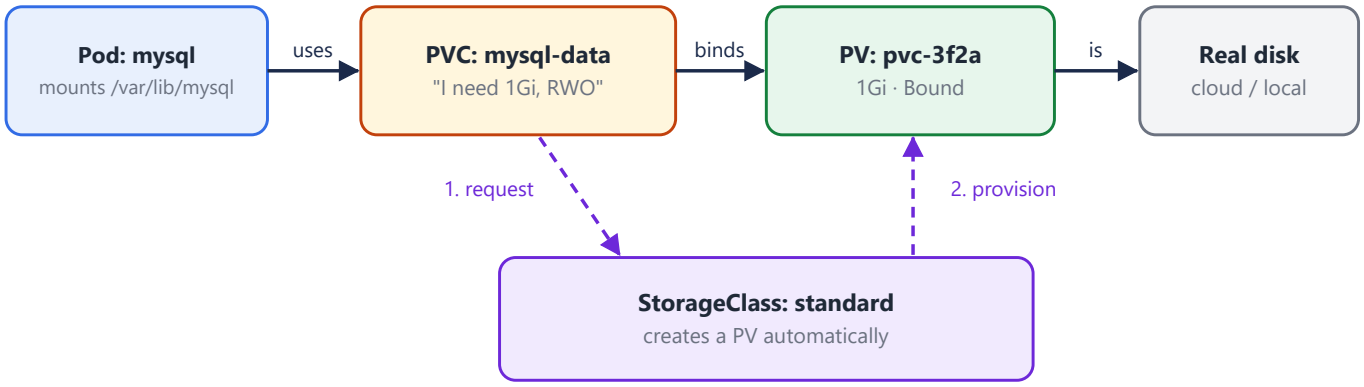


Figure 7.1 - A pod uses a PVC, which binds to a PV that a StorageClass can create on demand

pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-data
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: standard        # minikube's default class
  resources:
    requests:
      storage: 1Gi
---
# in the pod spec
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: mysql-data
  containers:
    - name: mysql
      volumeMounts:
        - name: data
          mountPath: /var/lib/mysql
```

Access mode	Short	Meaning
ReadWriteOnce	RWO	Mounted read-write by one node (most block disks)
ReadOnlyMany	ROX	Mounted read-only by many nodes

Access mode	Short	Meaning
ReadWriteMany	RWX	Read-write by many nodes (NFS, EFS, Azure Files, CephFS)
ReadWriteOncePod	RWOP	Read-write by exactly one pod

Reclaim policy	What happens to the PV when the PVC is deleted
Delete	The PV and the real disk are deleted (default for dynamic provisioning)
Retain	The PV and data are kept for manual recovery - use for important data
Recycle	Deprecated - basic scrub; do not use

7.3 StorageClass and dynamic provisioning

Creating PVs by hand does not scale. A **StorageClass** describes a type of storage (fast SSD, cheap HDD, replicated) and the **provisioner** that creates it. When a PVC names a StorageClass, the disk and PV are created automatically. One StorageClass can be marked as the cluster default.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com           # on AWS; minikube uses k8s.io/minikube-hostpath
parameters:
  type: gp3
reclaimPolicy: Retain
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer   # create the disk in the zone where the pod lands
```

Other features you should know: **volume expansion** (edit the PVC to a larger size if the class allows it) and **VolumeSnapshots** (point-in-time backups through CSI). Tools like **Velero** back up whole namespaces including volumes.

7.4 StatefulSets

A Deployment's pods are cattle: random names (`web-7d9f-x2b8l`), interchangeable, sharing one PVC if any. Databases, Kafka, Zookeeper and Elasticsearch need pets: each member has an identity and its own data. A **StatefulSet** provides:

- **Stable, ordered names** - `web-0`, `web-1`, `web-2` - kept after rescheduling.
- **Stable DNS** through a headless Service - `web-0.web-hl.default.svc.cluster.local`.
- **Per-pod storage** via `volumeClaimTemplates` - `data-web-0`, `data-web-1`... which follow the pod.
- **Ordered operations** - created 0, 1, 2 and deleted 2, 1, 0 (configurable with `podManagementPolicy`).
- **Rolling updates** from the highest ordinal down, with `partition` for staged rollouts.

> **Everyday example**
> Train seats are numbered. If passenger in seat 12 goes to the washroom and comes back, they return to seat 12 and their bag is still under it. That is a StatefulSet. A bus with free seating - sit anywhere, no fixed bag space - is a Deployment.

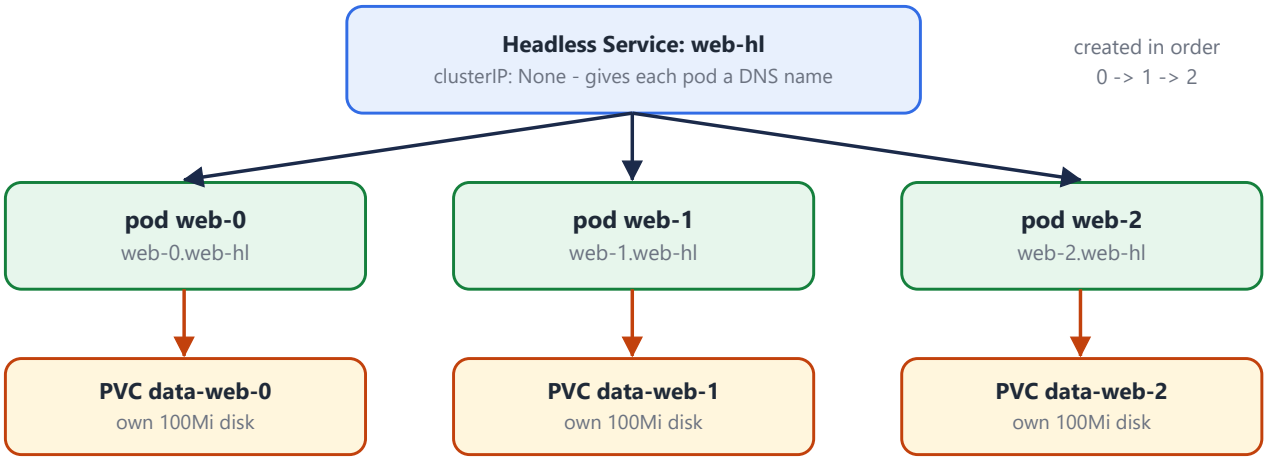


Figure 7.2 - StatefulSet pods keep their name, DNS entry and own PVC

statefulset.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: web-hl
spec:
  clusterIP: None              # headless
  selector: { app: web-sts }
  ports: [{ port: 80 }]
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: web-hl
  replicas: 3
  selector:
    matchLabels: { app: web-sts }
  template:
    metadata:
      labels: { app: web-sts }
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          volumeMounts:
            - name: data
              mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 100Mi
```

7.5 Live lab

PowerShell - part 1

```
kubectl get storageclass                      # 'standard (default)'
kubectl apply -f statefulset.yaml
kubectl get pods -l app=web-sts -w            # web-0, then web-1, then web-2
kubectl get pvc                               # data-web-0, data-web-1, data-web-2 - Bound
kubectl get pv

# give each pod its own page
0..2 | ForEach-Object {
  kubectl exec web-$_ -- sh -c "echo I am web-$_ > /usr/share/nginx/html/index.html"
}

kubectl run client --rm -it --image=busybox:1.36 -- sh
  wget -qO- web-1.web-hl                      # I am web-1
  nslookup web-hl                             # returns 3 pod IPs
  exit
```

PowerShell - part 2

```
kubectl delete pod web-1
kubectl get pods -l app=web-sts -w            # web-1 comes back with the SAME name
kubectl exec web-1 -- cat /usr/share/nginx/html/index.html   # still "I am web-1"

kubectl scale statefulset web --replicas=1    # web-2 then web-1 are removed
kubectl get pvc                               # PVCs are kept on purpose!
kubectl delete statefulset web; kubectl delete svc web-hl
kubectl delete pvc data-web-0 data-web-1 data-web-2
```

Common mistake

- Deleting a StatefulSet does **not** delete its PVCs - this protects data, but costs money in the cloud if forgotten.
- Running a database with RWO storage as a Deployment with 2+ replicas - the second pod cannot mount the disk or corrupts data.
- Thinking a StatefulSet makes a database highly available by itself - replication is the database's job; use an Operator for production databases.

Class 7 recap

- emptyDir lives with the pod; PVC data outlives pods.
- PV = the storage, PVC = the request, StorageClass = the factory.
- Access modes RWO/ROX/RWX/RWOP; reclaim Delete vs Retain.
- StatefulSet = stable names + stable DNS + per-pod PVC + ordering.

Class 7 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. Why do we need volumes in Kubernetes?

A container's file system is ephemeral - data is lost when the container restarts or the pod is replaced. Volumes provide storage that survives restarts and can be shared between containers.

Q2. What is an emptyDir volume?

A volume created empty when a pod starts and deleted when the pod is removed. It survives container restarts and is used for scratch space or sharing files between containers. It can be RAM-backed with `medium: Memory`.

Q3. What is hostPath and why is it risky?

It mounts a file or directory from the node into the pod. It ties the pod to that node's data and can expose the host file system, so it is used only for system agents, not applications.

Q4. What is a PersistentVolume?

A cluster-scoped storage resource provisioned by an admin or dynamically by a StorageClass, with its own lifecycle independent of pods.

Q5. What is a PersistentVolumeClaim?

A namespaced request for storage specifying size, access mode and optionally a StorageClass. Kubernetes binds it to a suitable PV; pods use storage by referencing the PVC.

Q6. Explain the PV and PVC binding process.

When a PVC is created, the controller looks for a PV with enough capacity, matching access mode and class. If found, they bind one-to-one; if a StorageClass is set and no PV exists, one is dynamically provisioned. Otherwise the PVC stays Pending.

Q7. What is a StorageClass?

An object describing a class of storage - provisioner, parameters, reclaim policy, expansion and binding mode - used for dynamic provisioning of PVs.

Q8. What is dynamic provisioning?

Automatic creation of a PV and the underlying disk when a PVC requests a StorageClass, so admins do not need to pre-create volumes.

Q9. What access modes are available?

ReadWriteOnce (one node read-write), ReadOnlyMany (many nodes read-only), ReadWriteMany (many nodes read-write) and ReadWriteOncePod (exactly one pod read-write).

Q10. What are reclaim policies?

They decide what happens to a PV after its PVC is deleted: Delete removes the PV and backing storage, Retain keeps them for manual recovery, and Recycle is deprecated.

Q11. What is volumeBindingMode WaitForFirstConsumer?

It delays provisioning and binding until a pod using the PVC is scheduled, so the disk is created in the same zone as the pod. Immediate binding can place the disk in the wrong zone.

Q12. Can you resize a PVC?

Yes, if the StorageClass has `allowVolumeExpansion: true`. Edit the PVC to a larger size; shrinking is not supported.

Q13. What is CSI?

The Container Storage Interface - a standard that lets storage vendors write drivers for Kubernetes without changing Kubernetes code. EBS, Azure Disk and Ceph drivers are CSI drivers.

Q14. What is a StatefulSet?

A workload controller for stateful applications that gives each pod a stable ordinal name, stable network identity through a headless Service, its own persistent storage, and ordered deployment and scaling.

Q15. StatefulSet vs Deployment?

Deployment pods are interchangeable with random names and typically shared or no storage. StatefulSet pods have fixed names (app-0, app-1), stable DNS, per-pod PVCs and ordered start, stop and updates.

Q16. Why does a StatefulSet need a headless Service?

The headless Service creates DNS records for each pod, such as `web-0.web-hl.ns.svc.cluster.local`, so peers and clients can reach a specific member.

Q17. What are volumeClaimTemplates?

Templates in a StatefulSet from which a separate PVC is created for each pod (data-web-0, data-web-1). The PVC is reattached when that pod is rescheduled.

Q18. What happens to PVCs when you scale down or delete a StatefulSet?

By default they are kept to protect data. Newer versions allow `persistentVolumeClaimRetentionPolicy` to delete them on scale-down or deletion.

Q19. How would you run a production database on Kubernetes?

Use a StatefulSet managed by an Operator (for example CloudNativePG, Percona or Strimzi for Kafka) with replicated storage, backups, monitoring and anti-affinity - or use a managed cloud database.

Q20. What are VolumeSnapshots?

CSI-based point-in-time copies of a PVC, defined with VolumeSnapshot and VolumeSnapshotClass objects. A new PVC can be restored from a snapshot.

K8S · CLASS 08 · 60-90 minutes

Workload types and scheduling

By the end you can choose the right workload controller and control exactly where pods run and how much CPU and memory they get.

What you will learn today

- DaemonSet, Job and CronJob - and when to use each controller
- How the scheduler picks a node
- nodeSelector, node affinity, pod affinity and anti-affinity
- Taints and tolerations; cordon and drain
- Resource requests and limits, QoS classes, OOMKilled
- PriorityClass, preemption and topology spread
- Lab: 3-node cluster - place, block and evict pods

Part	Topic	Time
1	Story and workload types	15 min
2	Scheduling rules	20 min
3	Resources and QoS	15 min
4	Live lab	20 min
5	Recap and interview questions	10 min

8.1 Choosing a workload controller

Ravi's story

Three new requests reached Ravi in one week: run a log collector on **every** server, run a database cleanup **once**, and generate a sales report **every night at 2 AM**. A Deployment fits none of them. Kubernetes has a controller for each.

Controller	Runs	Typical use	Everyday example
Deployment	N interchangeable pods, forever	Web apps, APIs	Cooks on every shift
StatefulSet	N pods with identity and storage	Databases, Kafka	Numbered train seats
DaemonSet	One pod on each (matching) node	Log/monitoring agents, CNI, kube-proxy	A security guard on every floor
Job	Pods until a task completes	Migrations, batch processing	A one-time deep cleaning
CronJob	A Job on a schedule	Backups, reports, cleanups	A daily alarm clock

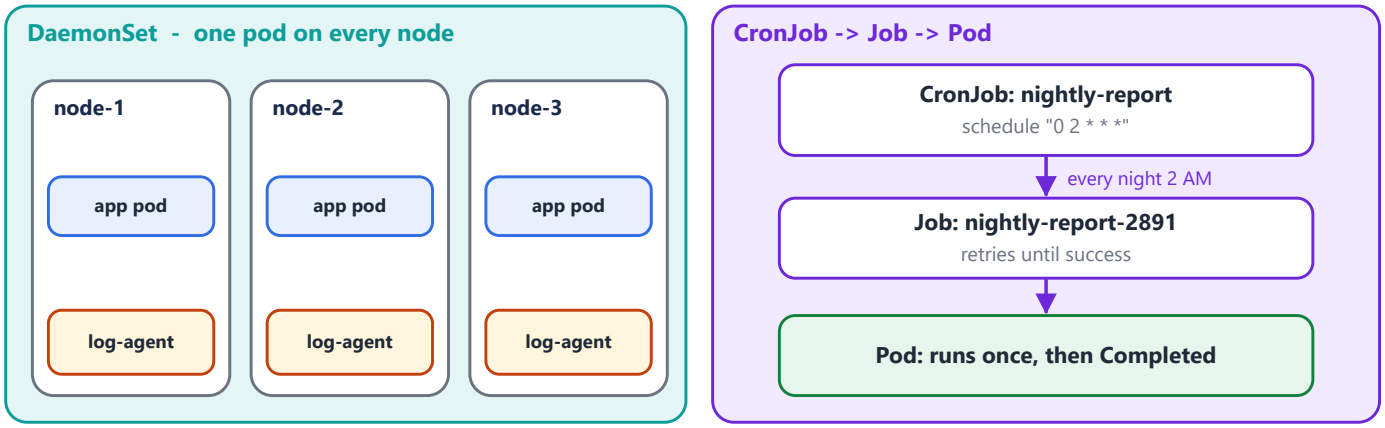


Figure 8.1 - A DaemonSet runs one pod per node; a CronJob creates Jobs on a schedule

cronjob.yaml

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-report
spec:
  schedule: "0 2 * * *"            # min hour day month weekday
  concurrencyPolicy: Forbid        # Allow | Forbid | Replace
  successfulJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 4              # retries before marking failed
      completions: 1
      parallelism: 1
      activeDeadlineSeconds: 600
      ttlSecondsAfterFinished: 3600
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: report
              image: busybox:1.36
              command: ["sh", "-c", "echo Sales report generated at $(date)"]
```

8.2 How scheduling works

For every new pod, the scheduler **filters** out nodes that cannot run it (not enough free requests, wrong labels, taints, port conflicts, volume zone) and then **scores** the rest (spread, balanced usage, affinity preferences). The highest-scoring node wins and the pod is bound to it. If no node passes the filter, the pod stays **Pending**.

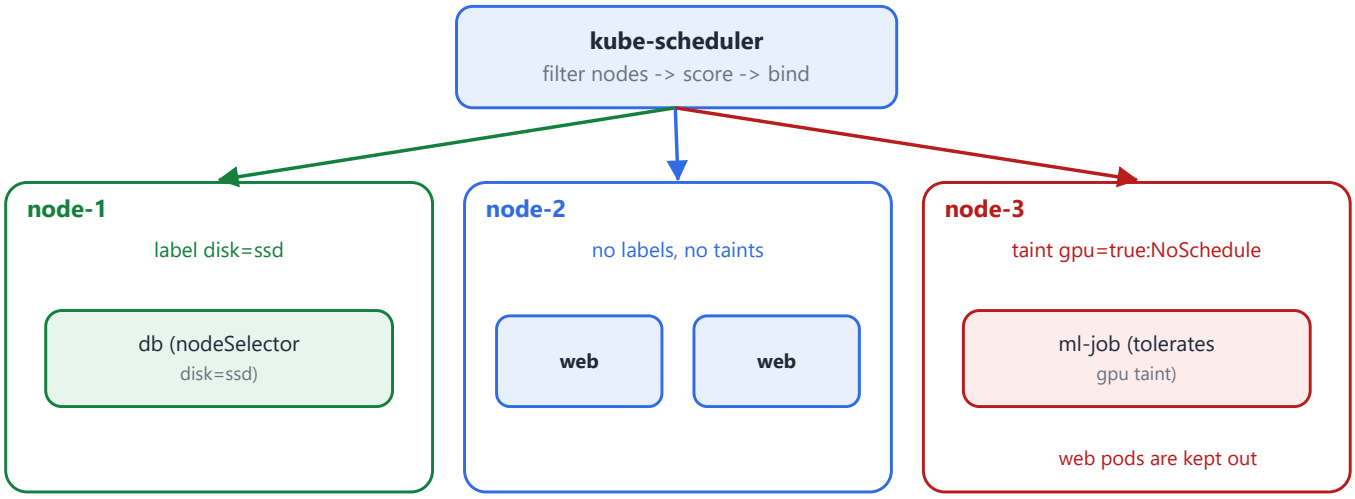


Figure 8.2 - nodeSelector attracts pods; taints repel pods without a toleration

Tool	Direction	Example
nodeSelector	Pod must land on nodes with a label	`disk: ssd`
Node affinity	Like nodeSelector, with operators and required/preferred rules	requiredDuringScheduling... `zone In [a, b]`
Pod affinity	Place near other pods	Put cache next to the web app
Pod anti-affinity	Keep away from other pods	Spread 3 replicas over 3 nodes for HA
Taint (on node)	Node repels pods	`gpu=true:NoSchedule`
Toleration (on pod)	Pod is allowed on a tainted node	`key: gpu, effect: NoSchedule`
topologySpreadConstraints	Spread evenly across zones/nodes	`maxSkew: 1` per zone

Everyday example
Taints and tolerations work like a **VIP lounge** at an airport. The lounge (node) has a sign "members only" (taint). Only passengers with a lounge card (toleration) may enter. But having a card does not force you into the lounge - to force placement you also need affinity or nodeSelector.

Taint effect	Behaviour
NoSchedule	New pods without a toleration are not scheduled here
PreferNoSchedule	The scheduler tries to avoid the node, but may use it
NoExecute	New pods are blocked and existing pods without a toleration are evicted

Cordon marks a node unschedulable (existing pods stay). **Drain** cordons it and safely evicts its pods - used before maintenance or upgrades. **Uncordon** brings it back.

8.3 Requests, limits and QoS

```
resources:
  requests:            # reserved; used by the scheduler
    cpu: 250m          # 250 millicores = 1/4 CPU
    memory: 128Mi
  limits:              # hard ceiling enforced on the node
    cpu: "1"
    memory: 256Mi
```

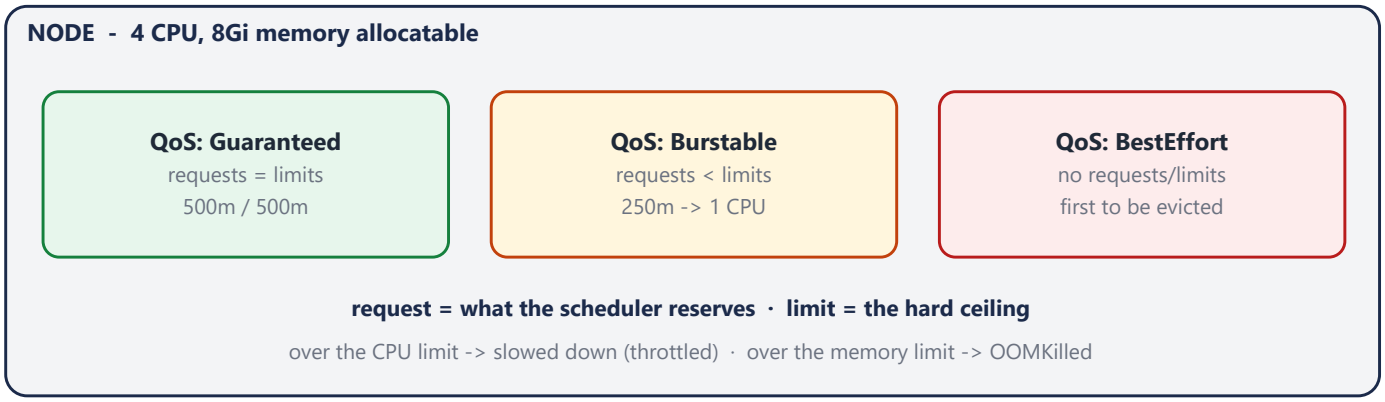


Figure 8.3 - Requests, limits and the three QoS classes

- Over the **CPU limit**: the container is **throttled** (slowed down), never killed.
- Over the **memory limit**: the container is **OOMKilled** (exit code 137) and restarted.
- When a node runs short of memory, the kubelet **evicts** pods: BestEffort first, then Burstable, Guaranteed last.

Everyday example
Booking a restaurant table for 4 is the **request** - the restaurant reserves those seats even if only 2 come. The fire-safety limit of 6 people per table is the **limit**. If a seventh person squeezes in, the manager asks someone to leave (OOMKilled).

PriorityClass gives pods a priority number. When the cluster is full, the scheduler may **preempt** (evict) lower-priority pods to make room for critical ones, such as payment over a nightly report.

8.4 Live lab: a 3-node cluster

ds.yaml

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-agent
spec:
  selector:
    matchLabels: { app: node-agent }
  template:
    metadata:
      labels: { app: node-agent }
    spec:
      containers:
        - name: agent
          image: busybox:1.36
          env:
            - name: NODE_NAME
              valueFrom: { fieldRef: { fieldPath: spec.nodeName } }
          command: ["sh", "-c", "while true; do echo collecting logs on $NODE_NAME; sleep 10; done"]
```

db-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: db
spec:
  nodeSelector:
    disk: ssd
  containers:
    - name: db
      image: nginx:1.27
```

PowerShell - workloads

```
minikube start -p lab --nodes 3 --driver=docker
kubectl get nodes                               # lab, lab-m02, lab-m03

# DaemonSet: one pod per node
kubectl apply -f ds.yaml
kubectl get pods -o wide -l app=node-agent      # one per node
kubectl logs -l app=node-agent --prefix

# Job and CronJob
kubectl create job hello --image=busybox:1.36 -- echo "cleanup done"
kubectl get jobs; kubectl logs job/hello
kubectl apply -f cronjob.yaml
kubectl create job manual-run --from=cronjob/nightly-report
```

PowerShell - scheduling

```
# nodeSelector
kubectl apply -f db-pod.yaml
kubectl get pod db                              # Pending - no node has disk=ssd yet
kubectl label node lab-m02 disk=ssd
kubectl get pod db -o wide                      # now Running on lab-m02

# taints
kubectl taint node lab-m03 gpu=true:NoSchedule
kubectl create deployment web --image=nginx:1.27 --replicas=6
kubectl get pods -o wide -l app=web             # none on lab-m03

# drain for maintenance
kubectl drain lab-m02 --ignore-daemonsets --delete-emptydir-data --force
kubectl get pods -o wide                        # pods moved away
kubectl uncordon lab-m02
kubectl taint node lab-m03 gpu=true:NoSchedule-   # remove taint
```

oom.yaml - try: kubectl apply, get pod -w, describe

```
apiVersion: v1
kind: Pod
metadata:
  name: memory-hog
spec:
  containers:
    - name: stress
      image: polinux/stress
      resources:
        limits: { memory: 100Mi }
        requests: { memory: 50Mi }
      command: ["stress"]
      args: ["--vm", "1", "--vm-bytes", "250M", "--vm-hang", "1"]
```

You will see **OOMKilled**, then CrashLoopBackOff. `kubectl describe pod memory-hog` shows `Last State: Terminated, Reason: OOMKilled, Exit Code: 137`. Clean up with `minikube delete -p lab`.

Common mistake

- Setting no requests: the scheduler thinks the pod needs nothing and overpacks nodes.
- Setting CPU limits too low for Java apps: start-up becomes painfully slow due to throttling.
- Expecting a toleration to *attract* a pod - it only *permits*; add affinity too.

Class 8 recap

- DaemonSet = one per node; Job = run to completion; CronJob = scheduled Job.
- Scheduler: filter then score. Pending = no node passed the filter.
- nodeSelector/affinity attract; taints repel; tolerations permit.
- Requests reserve, limits cap. CPU throttles, memory OOMKills.
- QoS: Guaranteed > Burstable > BestEffort.

Class 8 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is a DaemonSet? Give examples.

A controller that runs one copy of a pod on every node, or every node matching a selector. New nodes automatically get the pod. Examples: log collectors (Fluent Bit), monitoring agents (node-exporter), kube-proxy and CNI agents.

Q2. What is a Job?

A controller that runs pods until a specified number complete successfully. It retries failed pods up to backoffLimit (default 6). Used for batch tasks and migrations.

Q3. What is a CronJob?

A controller that creates Jobs on a cron schedule, such as "0 2 * * *" for 2 AM daily, with settings for concurrency, history limits and deadlines.

Q4. Explain completions and parallelism in a Job.

completions is how many successful pods are needed; parallelism is how many may run at once. completions 10 and parallelism 3 processes ten items three at a time.

Q5. What does concurrencyPolicy do in a CronJob?

It controls overlapping runs: Allow (default) lets them overlap, Forbid skips a new run if the previous one is still running, Replace stops the old run and starts the new one.

Q6. How does the kube-scheduler decide where to place a pod?

It filters nodes that satisfy requirements (resources, nodeSelector, affinity, taints, ports, volume topology), scores the remaining nodes by priorities such as spreading and balanced usage, and binds the pod to the highest-scoring node.

Q7. What is nodeSelector?

The simplest placement constraint: a map of labels that a node must have for the pod to be scheduled there.

Q8. Node affinity vs nodeSelector?

Node affinity is more expressive: operators like In, NotIn, Exists, Gt, and both hard (requiredDuringSchedulingIgnoredDuringExecution) and soft (preferred...) rules with weights.

Q9. What are pod affinity and anti-affinity?

Rules based on labels of other pods. Affinity co-locates pods (cache with app); anti-affinity separates them, commonly to spread replicas across nodes or zones using topologyKey.

Q10. What are taints and tolerations?

A taint on a node repels pods; a toleration on a pod allows it onto a node with a matching taint. They are used to dedicate nodes, for example GPU or control-plane nodes.

Q11. What are the taint effects?

NoSchedule blocks new pods, PreferNoSchedule avoids the node if possible, and NoExecute also evicts existing pods that do not tolerate the taint.

Q12. Taints vs node affinity?

Taints are node-centric and keep unwanted pods away; affinity is pod-centric and attracts pods to nodes. To dedicate nodes to specific pods you combine both.

Q13. What is the difference between cordon and drain?

Cordon marks a node unschedulable but leaves existing pods. Drain cordons and evicts pods (respecting PodDisruptionBudgets) so maintenance can be done. Uncordon makes it schedulable again.

Q14. What are resource requests and limits?

Requests are the CPU/memory guaranteed and used by the scheduler for placement. Limits are the maximum a container may use, enforced by the kubelet and Linux cgroups.

Q15. What happens when a container exceeds its CPU or memory limit?

Exceeding CPU leads to throttling. Exceeding memory makes the kernel kill the container (OOMKilled, exit code 137) and the kubelet restarts it according to the restart policy.

Q16. What are QoS classes?

Guaranteed (every container has requests equal to limits for CPU and memory), Burstable (some requests/limits set but not equal) and BestEffort (none set). Under memory pressure BestEffort is evicted first and Guaranteed last.

Q17. What does 250m CPU mean?

250 millicores - a quarter of one CPU core. Memory uses binary units such as Mi and Gi.

Q18. What is a PriorityClass and preemption?

A PriorityClass assigns an integer priority to pods. If a high-priority pod cannot be scheduled, the scheduler can preempt (evict) lower-priority pods to make room.

Q19. Why would a pod stay in Pending?

Insufficient CPU/memory on all nodes, unmatched nodeSelector or affinity, untolerated taints, an unbound PVC, quota limits or no ready nodes. `kubectl describe pod` shows the scheduler's reason in Events.

Q20. What are topologySpreadConstraints?

Rules that spread pods evenly across topology domains such as zones or nodes, using maxSkew, topologyKey and whenUnsatisfiable - improving high availability.

K8S · CLASS 09 · 60-90 minutes

Health checks, autoscaling, monitoring and troubleshooting

By the end you can make apps self-heal with probes, scale automatically on load, observe the cluster, and debug the most common failures quickly.

What you will learn today

- Liveness, readiness and startup probes - HTTP, TCP, exec, gRPC
- Graceful shutdown: preStop hooks and termination grace period
- HorizontalPodAutoscaler with metrics-server; VPA and Cluster Autoscaler
- PodDisruptionBudget for safe maintenance
- Monitoring and logging: kubectl top, Prometheus, Grafana, Loki
- A troubleshooting playbook for the common errors
- Lab: probe failures and a live autoscaling demo

Part	Topic	Time
1	Story and probes	15 min
2	Autoscaling and PDB	15 min
3	Observability and troubleshooting	15 min
4	Live lab	25 min
5	Recap and interview questions	10 min

9.1 Health probes

Ravi's story

During a sale the payment service froze - the process was alive but answered nothing. Kubernetes saw a running container and kept sending it traffic. Another day, new pods received orders before they had connected to the database, and the first 200 customers got errors. Ravi needed Kubernetes to know not just "is it running?" but "is it healthy?" and "is it ready?"

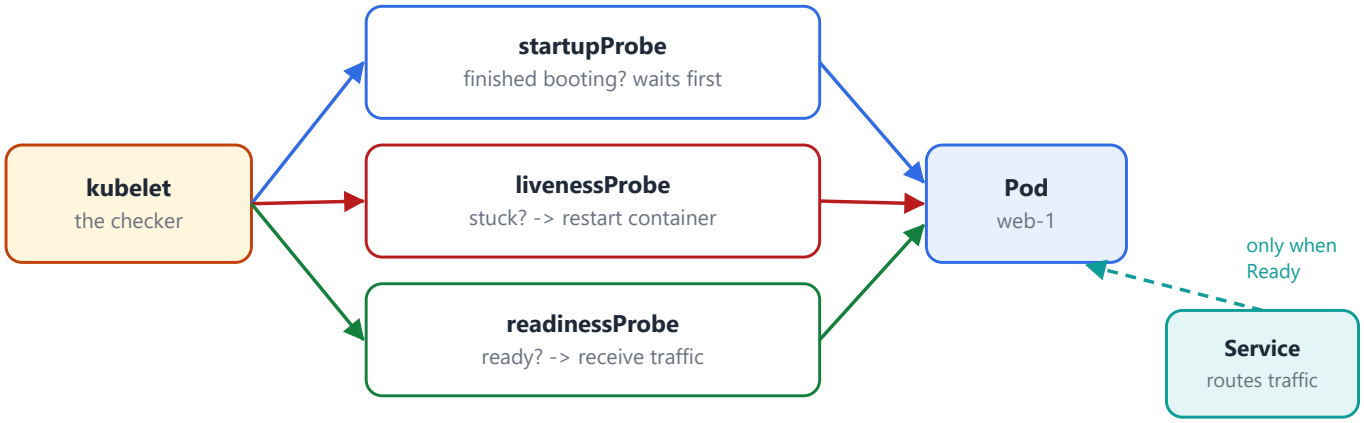


Figure 9.1 - The kubelet runs three kinds of probes; only Ready pods get Service traffic

Probe	Question	On failure
startupProbe	Has the app finished starting?	Keep waiting; other probes are paused. Kill after failureThreshold
livenessProbe	Is the app still alive or stuck?	Container is restarted
readinessProbe	Can it serve traffic now?	Pod removed from Service endpoints (not restarted)

Everyday example

Probes are like a hospital. The **startup** check: is the patient awake after surgery? Do not judge anything else until then. The **liveness** check: is the heart beating? If not, resuscitate (restart). The **readiness** check: is the doctor ready to see patients? If on a break, the receptionist simply stops sending patients for a while.

Probe configuration

```
containers:
  - name: api
    image: my-api:2.1
    ports: [{ containerPort: 8080 }]
    startupProbe:
      httpGet: { path: /healthz, port: 8080 }
      failureThreshold: 30          # 30 x 5s = up to 150s to start
      periodSeconds: 5
    livenessProbe:
      httpGet: { path: /healthz, port: 8080 }
      periodSeconds: 10
      failureThreshold: 3
    readinessProbe:
      httpGet: { path: /ready, port: 8080 }   # checks DB connection, etc.
      periodSeconds: 5
    lifecycle:
      preStop:
        exec: { command: ["sh", "-c", "sleep 10"] }   # let load balancers drain
terminationGracePeriodSeconds: 30
```

Probe mechanisms: `httpGet` (2xx/3xx = healthy), `tcpSocket` (port opens), `exec` (command exits 0) and `grpc`. Useful tuning fields: `initialDelaySeconds`, `periodSeconds`, `timeoutSeconds`, `failureThreshold`, `successThreshold`.

When a pod is deleted, Kubernetes removes it from endpoints, runs the **preStop** hook, sends **SIGTERM**, waits up to `terminationGracePeriodSeconds` (default 30) and finally sends **SIGKILL**. Apps should finish in-flight requests on SIGTERM.

Common mistake

- Making the liveness probe check the database. If the DB blips, every pod restarts at once. Liveness should only check the process itself; put dependencies in readiness.
- No startupProbe for slow apps (Spring Boot, large Java apps) - liveness kills them before they finish booting.

9.2 Autoscaling

Autoscaler	Scales	Based on
HorizontalPodAutoscaler (HPA)	Number of pod replicas	CPU, memory, or custom/external metrics
VerticalPodAutoscaler (VPA)	Requests/limits of each pod	Historical usage (add-on)
Cluster Autoscaler / Karpenter	Number of nodes	Pending pods that do not fit / idle nodes
KEDA	Replicas, even to zero	Events: queue length, Kafka lag, cron

Everyday example
A supermarket opens extra billing counters when queues grow and closes them when it is quiet - that is the HPA. Giving one cashier a faster scanner is the VPA. Renting the shop next door when there is no space for more counters is the Cluster Autoscaler.

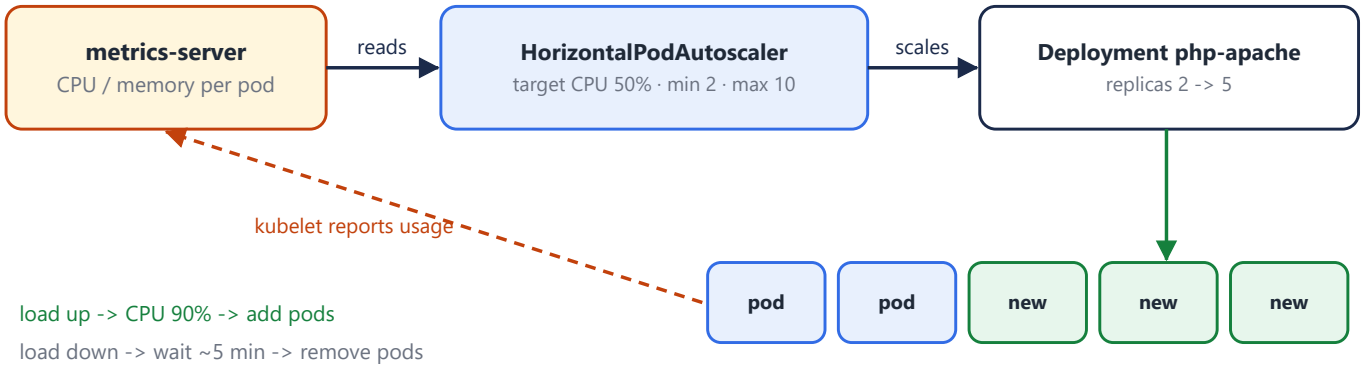


Figure 9.2 - HPA reads metrics and changes the Deployment's replica count

The HPA formula: `desired = ceil(current replicas x current metric / target)`. With 2 pods at 90% CPU and a 50% target: ceil(2 x 90 / 50) = 4 pods. Scale-down waits for a stabilisation window (5 minutes by default) to avoid flapping. HPA needs **requests** to be set, because CPU percentage is measured against the request.

hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 50 }
```

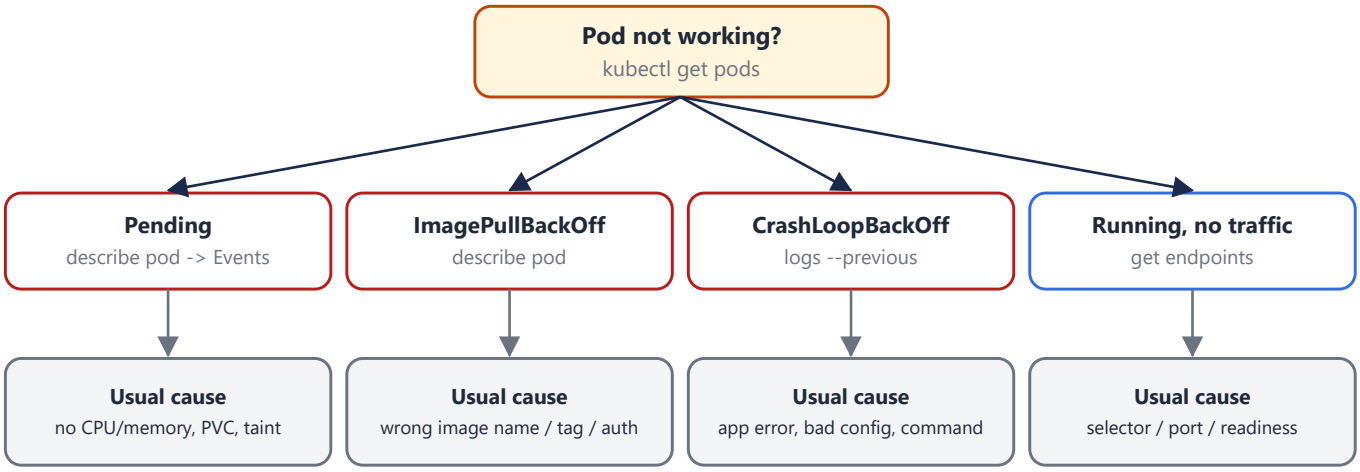
PodDisruptionBudget

A **PDB** limits how many pods of an app may be down at once during *voluntary* disruptions such as `kubectl drain` or cluster upgrades: `minAvailable: 2` or `maxUnavailable: 1`. It does not protect against node crashes.

9.3 Observability

Need	Tool
Quick CPU/memory numbers	`kubectl top nodes`, `kubectl top pods` (needs metrics-server)
Metrics, alerts, dashboards	Prometheus + Alertmanager + Grafana (kube-prometheus-stack Helm chart)
Central logs	Fluent Bit / Fluentd shipping to Loki, Elasticsearch/OpenSearch or a cloud service
Distributed tracing	OpenTelemetry with Jaeger or Tempo
Cluster events	`kubectl get events -A --sort-by=.lastTimestamp`

9.4 Troubleshooting playbook



Golden order: get -> describe -> logs -> events -> exec

Figure 9.3 - Start from the pod status and follow the matching path

Symptom	First command	Typical fix
Pending	`kubectl describe pod`	Lower requests, add nodes, fix nodeSelector/taint, bind the PVC
ErrImagePull / ImagePullBackOff	`kubectl describe pod`	Correct image/tag, add imagePullSecrets
CrashLoopBackOff	`kubectl logs --previous`	Fix the app error, command, env or config
OOMKilled (exit 137)	`kubectl describe pod`	Raise memory limit or fix the memory leak
CreateContainerConfigError	`kubectl describe pod`	Create the missing ConfigMap/Secret or key
Running but 0/1 Ready	`kubectl describe pod`	Fix the readiness probe path/port or the dependency
Service not reachable	`kubectl get endpointslices`	Fix selector labels, targetPort, NetworkPolicy
Node NotReady	`kubectl describe node`	Check kubelet, disk/memory pressure, network

For images without a shell, attach a debugging container: `kubectl debug -it <pod> --image=busybox:1.36 --target=<container>`. `kubectl debug node/<node> -it --image=busybox:1.36` opens a shell on a node.

9.5 Live lab

liveness.yaml - after ~45 s RESTARTS goes up

```
apiVersion: v1
kind: Pod
metadata:
  name: liveness-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      args: ["sh", "-c", "touch /tmp/healthy; sleep 30; rm -f /tmp/healthy; sleep 600"]
      livenessProbe:
        exec: { command: ["cat", "/tmp/healthy"] }
        initialDelaySeconds: 5
        periodSeconds: 5
```

PowerShell

```
kubectl apply -f liveness.yaml
kubectl get pod liveness-demo -w
kubectl describe pod liveness-demo        # "Liveness probe failed: cat: can't open..."

minikube addons enable metrics-server
kubectl create deployment php-apache --image=registry.k8s.io/hpa-example
kubectl set resources deployment php-apache --requests=cpu=200m --limits=cpu=500m
kubectl expose deployment php-apache --port=80
kubectl apply -f hpa.yaml
kubectl get hpa -w                         # keep this window open

# second window - generate load:
kubectl run load --rm -it --image=busybox:1.36 -- `
  sh -c "while true; do wget -q -O- http://php-apache; done"
# watch TARGETS climb above 50% and REPLICAS grow; press Ctrl+C to stop load
kubectl top pods
```

After you stop the load, replicas return to 1 after about five minutes. Clean up: `kubectl delete hpa php-apache; kubectl delete deploy,svc php-apache; kubectl delete pod liveness-demo`.

Class 9 recap

- Startup protects slow boots; liveness restarts; readiness removes from traffic.
- Graceful shutdown: preStop -> SIGTERM -> grace period -> SIGKILL.
- HPA scales pods (needs requests + metrics-server); VPA resizes; Cluster Autoscaler adds nodes.
- PDB keeps a minimum running during drains and upgrades.
- Debug order: get -> describe -> logs -> events -> exec/debug.

Class 9 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What are liveness, readiness and startup probes?

Liveness checks whether a container should be restarted; readiness checks whether it should receive traffic; startup checks whether the application has finished starting and holds off the other probes until it has.

Q2. What happens when a liveness probe fails versus a readiness probe?

A failing liveness probe makes the kubelet restart the container. A failing readiness probe removes the pod from Service endpoints without restarting it, and it is added back when it passes again.

Q3. When do you need a startup probe?

For applications with long or unpredictable start-up times, such as large Java apps. It prevents the liveness probe from killing the container before it is up.

Q4. What probe mechanisms are available?

httpGet (success on status 200-399), tcpSocket (port accepts a connection), exec (command exits 0) and grpc.

Q5. Name important probe parameters.

initialDelaySeconds, periodSeconds, timeoutSeconds, failureThreshold and successThreshold.

Q6. Should a liveness probe check the database?

No. If the database has a problem every pod would restart, making things worse. Liveness should check only the process; dependency checks belong in readiness.

Q7. Explain graceful termination of a pod.

The pod is marked Terminating and removed from endpoints, the preStop hook runs, the container gets SIGTERM, Kubernetes waits up to terminationGracePeriodSeconds (default 30) and then sends SIGKILL.

Q8. What is the HorizontalPodAutoscaler?

A controller that adjusts the replica count of a Deployment, StatefulSet or ReplicaSet based on observed metrics such as CPU, memory, or custom and external metrics, within min and max bounds.

Q9. What are the prerequisites for CPU-based HPA?

A metrics source such as metrics-server and CPU requests defined on the containers, because utilisation is calculated as a percentage of the request.

Q10. How does the HPA calculate the desired replicas?

desiredReplicas = ceil(currentReplicas x currentMetricValue / targetValue). Scale-down uses a stabilisation window (300 seconds by default) to avoid flapping.

Q11. What is the VerticalPodAutoscaler?

An add-on that recommends or automatically sets CPU and memory requests based on historical usage. It should not be combined with HPA on the same CPU or memory metric.

Q12. What is the Cluster Autoscaler?

A component that adds nodes when pods are Pending because of insufficient resources and removes under-used nodes. Karpenter is a faster, more flexible alternative on AWS and other clouds.

Q13. What is a PodDisruptionBudget?

A policy limiting how many pods of an application can be unavailable during voluntary disruptions like drains and upgrades, using minAvailable or maxUnavailable.

Q14. How do you check resource usage of pods and nodes?

`kubectl top pods` and `kubectl top nodes`, which need metrics-server. For history and alerts use Prometheus and Grafana.

Q15. How is logging typically done in Kubernetes?

Apps write to stdout/stderr; the runtime stores logs on the node; a DaemonSet agent such as Fluent Bit ships them to a central store such as Loki, Elasticsearch or a cloud logging service.

Q16. How would you troubleshoot a pod in CrashLoopBackOff?

Check `kubectl logs --previous`, then `kubectl describe pod` for exit code, reason (OOMKilled) and events. Verify command, args, env vars, ConfigMaps/Secrets, probes and resource limits; reproduce with `kubectl debug` if needed.

Q17. What does exit code 137 mean?

The container was killed with SIGKILL (128 + 9), usually OOMKilled for exceeding its memory limit, or killed after the grace period.

Q18. How do you debug a container that has no shell?

Use `kubectl debug -it <pod> --image=busybox --target=<container>` to attach an ephemeral container that shares the process namespace, or copy the pod with `--copy-to`.

Q19. How do you view cluster events?

`kubectl get events -n <ns> --sort-by=.lastTimestamp` or the Events section of `kubectl describe`. Events are kept for about one hour by default.

Q20. What is the difference between monitoring and observability?

Monitoring watches known metrics and alerts on thresholds. Observability combines metrics, logs and traces so you can ask new questions and understand unknown failures.

K8S · CLASS 10 · 60-90 minutes

Security, GitOps and production Kubernetes

By the end you can secure a cluster with RBAC, pod security and NetworkPolicies, explain GitOps, and describe how Kubernetes runs in real companies.

What you will learn today

- Authentication, authorization (RBAC) and admission
- ServiceAccounts, Roles, ClusterRoles and bindings
- securityContext and Pod Security Admission
- NetworkPolicies
- CI/CD and GitOps with Argo CD
- Managed Kubernetes, high availability, service mesh and cost
- Final project: deploy WordPress + MySQL end to end

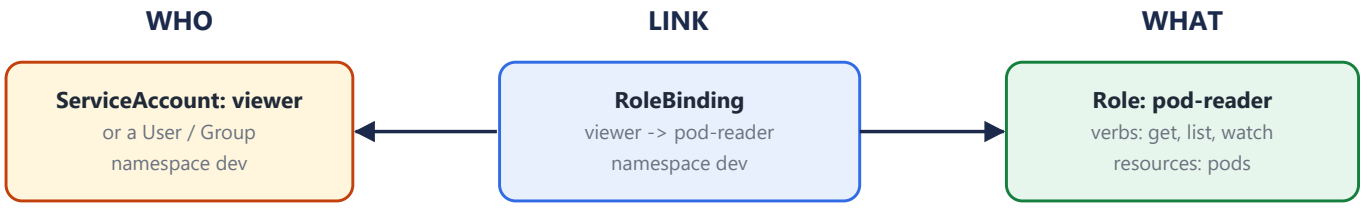
Part	Topic	Time
1	Story, RBAC and pod security	25 min
2	NetworkPolicies	15 min
3	CI/CD, GitOps and production topics	15 min
4	Live lab and final project brief	20 min
5	Recap and interview questions	10 min

10.1 Who can do what? (RBAC)

Ravi's story

An intern ran `kubectl delete namespace prod` thinking he was on his test cluster. He had full admin rights because everyone shared one kubeconfig. QuickBite was down for an hour. Ravi was asked to make sure "people and apps get only the access they need".

Every API request passes three gates: **authentication** (who are you? - certificates, tokens, cloud IAM, OIDC), **authorization** (are you allowed? - RBAC) and **admission control** (is the request acceptable? - quotas, Pod Security, policy engines).



ClusterRole + ClusterRoleBinding = the same idea for the whole cluster

Default is DENY - you only get what a binding gives you

Figure 10.1 - RBAC connects a subject to a set of permissions through a binding

Object	Scope	Purpose
Role	Namespace	Permissions (verbs on resources) inside one namespace
ClusterRole	Cluster	Permissions cluster-wide or for cluster-scoped resources; reusable in namespaces
RoleBinding	Namespace	Grants a Role or ClusterRole to subjects within one namespace
ClusterRoleBinding	Cluster	Grants a ClusterRole across the whole cluster
ServiceAccount	Namespace	Identity for pods/apps talking to the API

Everyday example
Office ID cards. Your card (ServiceAccount or user) says who you are. The access policy (Role) says "can open the 3rd-floor store room". The HR register entry (RoleBinding) links your card to that policy. Without an entry, the door stays shut - default deny.

rbac.yaml

```
apiVersion: v1
kind: ServiceAccount
metadata: { name: viewer, namespace: dev }
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: pod-reader, namespace: dev }
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: viewer-reads-pods, namespace: dev }
subjects:
  - kind: ServiceAccount
    name: viewer
    namespace: dev
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

10.2 Securing pods

Hardened securityContext

```
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    fsGroup: 2000
    seccompProfile: { type: RuntimeDefault }
  containers:
    - name: app
      image: my-api:2.1
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: { drop: ["ALL"] }
```

Pod Security Admission enforces the Pod Security Standards per namespace with a label: `privileged` (anything), `baseline` (blocks known escalations) or `restricted` (best practice). Example: `kubectl label ns prod pod-security.kubernetes.io/enforce=restricted`. It replaced the old PodSecurityPolicy, removed in v1.25. For custom rules teams add **Kyverno** or **OPA Gatekeeper**.

Security layer	Good practice
Images	Small base images, pinned tags or digests, scanned with Trivy/Grype, signed with cosign
API access	RBAC least privilege, no shared admin kubeconfig, SSO/OIDC, audit logs
Pods	Non-root, read-only file system, drop capabilities, Pod Security `restricted`
Network	Default-deny NetworkPolicies, TLS everywhere, service mesh mTLS (Istio, Linkerd)
Secrets	Encryption at rest, external secret managers, no secrets in Git
Nodes & cluster	Patch regularly, private API endpoint, CIS benchmark (kube-bench)

10.3 NetworkPolicies

By default every pod can talk to every other pod. A **NetworkPolicy** selects pods by label and lists which traffic is allowed in (ingress) and out (egress). Once a pod is selected by any policy, everything not allowed is denied. Policies need a CNI that enforces them - Calico, Cilium or the cloud equivalents; Flannel alone does not.

Everyday example
A housing society gate: by default anyone walks in. After the committee adds rules - "only residents and registered delivery partners may enter Tower B" - the guard stops everyone else. The guard is the CNI; the rule book is the NetworkPolicy.

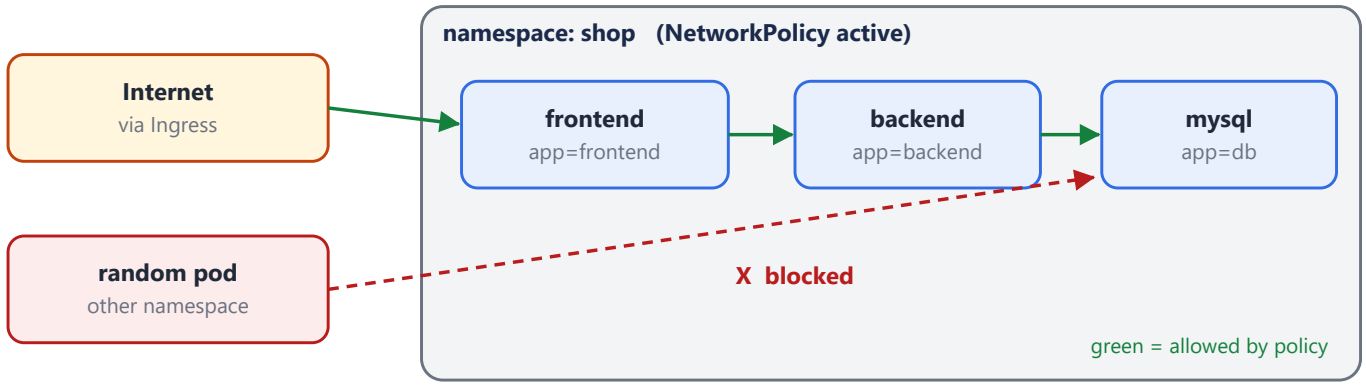


Figure 10.2 - Only the frontend may call the backend, and only the backend may call MySQL

netpol.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: backend-allow-frontend, namespace: shop }
spec:
  podSelector:
    matchLabels: { app: backend }
  policyTypes: ["Ingress"]
  ingress:
    - from:
        - podSelector:
            matchLabels: { app: frontend }
      ports:
        - port: 80
```

10.4 CI/CD and GitOps

In **push-based CI/CD** a pipeline (Jenkins, GitHub Actions, GitLab CI, Azure DevOps) builds the image and runs `kubectl apply` or `helm upgrade` against the cluster. In **GitOps**, Git holds the desired state and an agent inside the cluster - **Argo CD** or **Flux** - pulls and applies it continuously, reverting manual drift. The manifests in Git are usually packaged as a Helm chart or Kustomize overlays - both are covered in depth in Class 11.

Everyday example
GitOps is like a building's approved floor plan kept at the municipal office. If someone secretly adds a wall, the inspector compares the building with the plan and removes it. To change the building, you first get the plan changed and approved (a pull request).

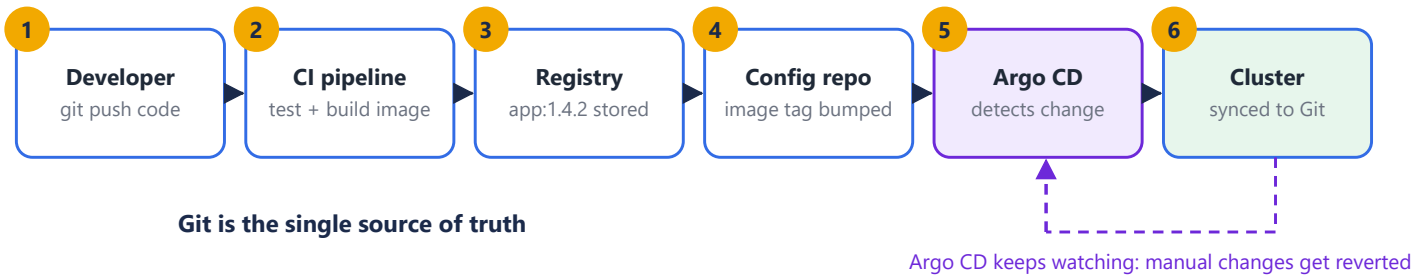


Figure 10.3 - GitOps flow: Git is the single source of truth and Argo CD keeps the cluster in sync

argocd-app.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: guestbook
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/argoproj/argocd-example-apps.git
    targetRevision: HEAD
    path: guestbook
  destination:
    server: https://kubernetes.default.svc
    namespace: guestbook
  syncPolicy:
    automated: { prune: true, selfHeal: true }   # delete removed objects, undo manual edits
    syncOptions: ["CreateNamespace=true"]
```

10.5 Kubernetes in production

Topic	What you should know
Managed Kubernetes	EKS (AWS), AKS (Azure), GKE (Google): the cloud runs the control plane; you manage nodes and workloads
Self-managed	kubeadm (official), k3s (lightweight), Rancher, OpenShift (enterprise distribution)
High availability	3 control-plane nodes, etcd quorum, nodes across availability zones, anti-affinity, PDBs
Upgrades	One minor version at a time: control plane first, then nodes (drain, upgrade, uncordon)
Backup	`etcdctl snapshot save` for etcd; Velero for namespaces and volumes
CRDs	CustomResourceDefinitions add new object types, e.g. `Certificate`, `Application`, `PostgresCluster`
Operators	A CRD plus a controller that encodes human operational knowledge - install, backup, failover
Service mesh	Istio, Linkerd: mTLS, retries, traffic splitting and telemetry without code changes
Cost	Right-size requests, autoscale nodes, use spot instances for stateless work, delete unused volumes/LBs

Did you know?
Many cluster add-ons you met in this course are themselves Operators built on CRDs - cert-manager (`Certificate`), Argo CD (`Application`), Prometheus Operator (`ServiceMonitor`). Kubernetes is a platform for building platforms.

10.6 Live lab: security and GitOps

PowerShell - RBAC

```
kubectl create namespace dev
kubectl apply -f rbac.yaml
kubectl auth can-i list pods -n dev --as=system:serviceaccount:dev:viewer      # yes
kubectl auth can-i delete pods -n dev --as=system:serviceaccount:dev:viewer    # no
kubectl auth can-i list secrets -n dev --as=system:serviceaccount:dev:viewer   # no
kubectl auth can-i --list -n dev --as=system:serviceaccount:dev:viewer
```

PowerShell - NetworkPolicy

```
minikube start -p secure --cni=calico --driver=docker
kubectl create namespace shop
kubectl -n shop run backend  --image=nginx:1.27 --labels=app=backend --port=80
kubectl -n shop expose pod backend --port=80
kubectl -n shop run frontend --image=busybox:1.36 --labels=app=frontend -- sleep 3600
kubectl -n shop run random   --image=busybox:1.36 --labels=app=random   -- sleep 3600

kubectl -n shop exec frontend -- wget -qO- -T 3 backend     # works
kubectl apply -f netpol.yaml
kubectl -n shop exec frontend -- wget -qO- -T 3 backend     # still works
kubectl -n shop exec random   -- wget -qO- -T 3 backend     # times out - blocked
minikube delete -p secure
```

PowerShell - Argo CD

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side `
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd get pods -w                 # wait until all are Running

# admin password, then open https://localhost:8080 (user: admin)
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}"
kubectl -n argocd port-forward svc/argocd-server 8080:443

kubectl apply -f argocd-app.yaml              # in a second window
kubectl -n guestbook get pods                 # deployed from Git by Argo CD
kubectl -n guestbook scale deploy guestbook-ui --replicas=5
kubectl -n guestbook get deploy guestbook-ui -w   # selfHeal puts it back to 1
```

The password is base64-encoded - decode it as shown in Class 6. Remove Argo CD afterwards with `kubectl delete namespace argocd guestbook`.

10.7 Final project: WordPress + MySQL

Put the whole course together. Build it in a namespace called `blog`, store every file in a Git repository, and tick each item as you go:

Step	Objects	Concept from class
1	Namespace `blog` with a ResourceQuota and LimitRange	5
2	Secret `mysql-pass` with root and user passwords	6
3	MySQL StatefulSet (`mysql:8.0`) + headless Service + 1Gi volumeClaimTemplate	4, 7
4	ConfigMap with WordPress settings (DB host `mysql-0.mysql`, DB name)	6
5	WordPress Deployment (2 replicas) with readiness/liveness probes and requests/limits	3, 8, 9
6	ClusterIP Service + Ingress for `blog.local`	4, 5
7	HPA for WordPress (CPU 60%, 2-5 replicas) and a PDB (minAvailable 1)	9
8	NetworkPolicy: only WordPress pods can reach MySQL on 3306	10
9	Delete the MySQL pod and prove posts survive; roll out a new WordPress tag and roll back	3, 7
10	Bonus: convert everything into a Helm chart and deploy it with Argo CD	10, 11

Class 10 recap

- Every request: authentication -> authorization (RBAC) -> admission.
- Role/RoleBinding = namespace; ClusterRole/ClusterRoleBinding = cluster.
- Harden pods: non-root, read-only FS, drop capabilities, Pod Security `restricted`.
- NetworkPolicy = firewall for pods (needs Calico/Cilium).
- GitOps = Git is the truth; Argo CD pulls, syncs and self-heals drift.

Class 10 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is RBAC in Kubernetes?

Role-Based Access Control: an authorization mechanism where Roles and ClusterRoles define allowed verbs on resources, and RoleBindings and ClusterRoleBindings grant them to users, groups or ServiceAccounts. Everything not granted is denied.

Q2. Role vs ClusterRole?

A Role grants permissions within one namespace. A ClusterRole is cluster-wide and can cover cluster-scoped resources such as nodes, or be reused in many namespaces through RoleBindings.

Q3. RoleBinding vs ClusterRoleBinding?

A RoleBinding grants a Role or ClusterRole within a single namespace. A ClusterRoleBinding grants a ClusterRole across all namespaces.

Q4. What is a ServiceAccount?

A namespaced identity for processes running in pods. Pods get a short-lived token automatically mounted, used to call the API server with the permissions bound to that ServiceAccount.

Q5. How do you check if a user or ServiceAccount can perform an action?

`kubectl auth can-i <verb> <resource> -n <ns> --as=<user>` - for a ServiceAccount use `--as=system:serviceaccount:<ns>:<name>`; `--list` shows all permissions.

Q6. What are the stages an API request goes through?

Authentication (who), authorization (allowed, usually RBAC), then mutating and validating admission controllers (defaults, quotas, Pod Security, webhooks), and finally persistence in etcd.

Q7. What is a securityContext?

Pod or container settings that control privileges: runAsUser, runAsNonRoot, fsGroup, readOnlyRootFilesystem, allowPrivilegeEscalation, capabilities, seccomp and SELinux options.

Q8. What replaced PodSecurityPolicy?

Pod Security Admission, which enforces the Pod Security Standards (privileged, baseline, restricted) per namespace through labels. Kyverno or OPA Gatekeeper add custom policies.

Q9. What is a NetworkPolicy?

A namespaced object that controls pod-level ingress and egress traffic using pod selectors, namespace selectors, IP blocks and ports. Pods selected by a policy deny all traffic not explicitly allowed.

Q10. Does every cluster enforce NetworkPolicies?

No. Enforcement depends on the CNI plugin. Calico, Cilium and most cloud CNIs support it; plain Flannel does not, so policies would have no effect.

Q11. How do you create a default-deny policy for a namespace?

A NetworkPolicy with an empty podSelector (`{}`) and policyTypes Ingress (and Egress) but no rules. It selects all pods and allows nothing, and specific allow policies are added on top.

Q12. Why should containers not run as root?

If an attacker breaks out of a root container they may gain root on the node. Running as a non-root user with dropped capabilities, no privilege escalation and a read-only file system limits the damage.

Q13. What are admission webhooks?

HTTP callbacks the API server calls during admission. Mutating webhooks can change objects (inject sidecars, add defaults); validating webhooks can reject them. Kyverno, Gatekeeper and Istio use them.

Q14. How do you make sure only trusted images run in the cluster?

Scan images in CI (Trivy, Grype), sign them (cosign/Sigstore), and enforce policies with Kyverno or Gatekeeper that allow only approved registries, pinned tags or digests, and verified signatures.

Q15. What is Kubernetes audit logging?

The API server can record who did what and when for every request, according to an audit policy. Audit logs are essential for security investigations and compliance.

Q16. What is GitOps?

An operating model where the desired state of the system is stored in Git and an in-cluster agent (Argo CD, Flux) continuously pulls and reconciles it, giving audit history, easy rollback via git revert, and drift correction.

Q17. What is Argo CD?

A declarative GitOps continuous-delivery tool for Kubernetes. It watches Git repositories, compares them with live cluster state, shows differences in a UI, and syncs automatically or on demand.

Q18. What are CRDs and Operators?

A CustomResourceDefinition extends the Kubernetes API with new object types. An Operator is a controller that watches those custom resources and automates operational tasks such as installing, backing up, scaling and failing over complex applications.

Q19. How do you upgrade a Kubernetes cluster?

Read release notes and deprecations, back up etcd, upgrade one minor version at a time - control plane first, then worker nodes one by one with drain, upgrade kubelet, uncordon - using PDBs to keep apps available. Managed services automate much of this.

Q20. What are managed Kubernetes services and why use them?

EKS, AKS and GKE run and patch the control plane, integrate with cloud IAM, load balancers and storage, and offer node autoscaling. Teams focus on workloads instead of operating etcd and the API server.

K8S · CLASS 11 · 60-90 minutes

Helm charts in depth, Kustomize and cluster essentials

By the end you can install, write, template, test, package and roll back your own Helm chart, manage environments with Kustomize, and handle key cluster-admin tasks.

What you will learn today

- Why Helm: the copy-paste YAML problem
- Helm concepts: chart, values, release, revision, repository
- Using public charts: search, install, upgrade, rollback, uninstall
- Anatomy of a chart: Chart.yaml, values.yaml, templates, _helpers.tpl, NOTES.txt
- The template language: objects, functions, pipelines, if/range/with
- Values for many environments and the override order
- Dependencies (subcharts), hooks, tests, packaging and OCI registries
- Helm in CI/CD and GitOps; Kustomize as the alternative
- Other essentials: imagePullPolicy, garbage collection and finalizers, CRDs and Operators, kubeadm, cluster upgrades, etcd backup and restore
- Lab: build the quickbite chart and deploy dev and prod releases

Part	Topic	Time
1	Story, Helm concepts and public charts	15 min
2	Writing a chart: templates and values	25 min
3	Dependencies, hooks, packaging, CI/CD	10 min
4	Kustomize and cluster essentials	15 min
5	Live lab	15 min
6	Recap and interview questions	5 min

11.1 Why Helm?

Ravi's story

QuickBite now runs menu, orders and payment in dev, QA and prod. Each app needs a Deployment, Service, Ingress, ConfigMap, Secret, HPA and PDB - 7 files x 3 apps x 3 environments = 63 YAML files that are 90% identical. Last week someone changed the memory limit in dev but forgot prod, and prod crashed. Ravi's lead said: "Write it once as a **Helm chart**, and keep only the differences per environment."

Helm is the package manager for Kubernetes - what apt is to Ubuntu, npm to Node.js or Maven to Java. It turns a folder of templated YAML into a versioned package that you can install, upgrade and roll back with one command.

Everyday example

Think of a wedding card printer. The design (template) is the same for every family; only the names, date and venue change (values). The printer fills in the blanks and prints the final cards (rendered YAML). If the date changes, you reprint from the same design with a new value - you never redraw the card.

Term	Meaning	Example
Chart	A package: templates + default values + metadata	`quickbite`, `podinfo`, `kube-prometheus-stack`
Values	The settings that fill the templates	`replicaCount: 3`, `image.tag: 1.4.2`
Release	One installed copy of a chart, with a name, in a namespace	`shop-dev`, `shop-prod`
Revision	A numbered version of a release, created on every install/upgrade/rollback	1, 2, 3
Repository	A server or OCI registry where charts are published	Artifact Hub, Docker Hub, ECR, ACR

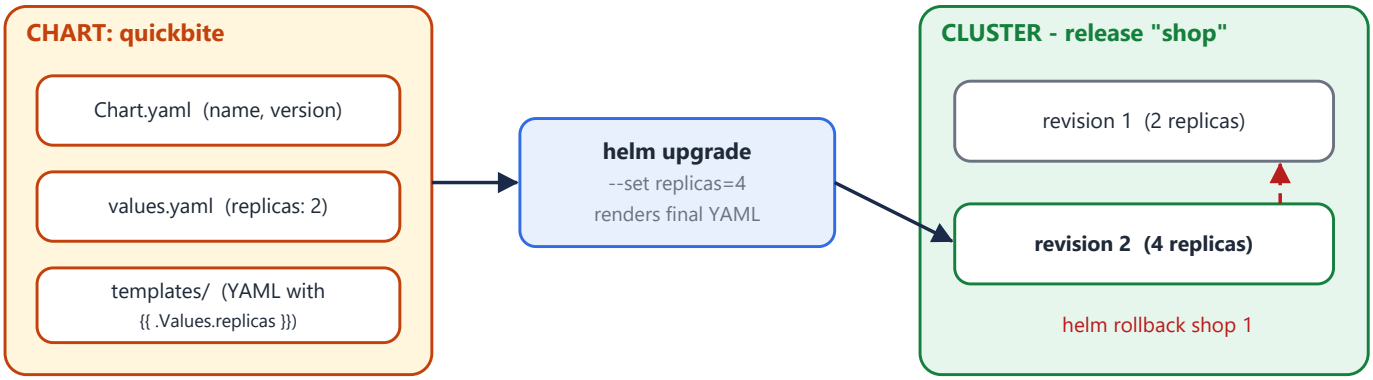


Figure 11.1 - A chart plus values becomes a release with numbered revisions

How Helm works inside

Since Helm 3, Helm is just a client program - there is nothing to install in the cluster (Helm 2's server component, Tiller, was removed for security). Helm renders the templates on your laptop or CI runner, sends the resulting YAML to the API server using your kubeconfig permissions, and stores a record of each revision as a Secret in the release's namespace.

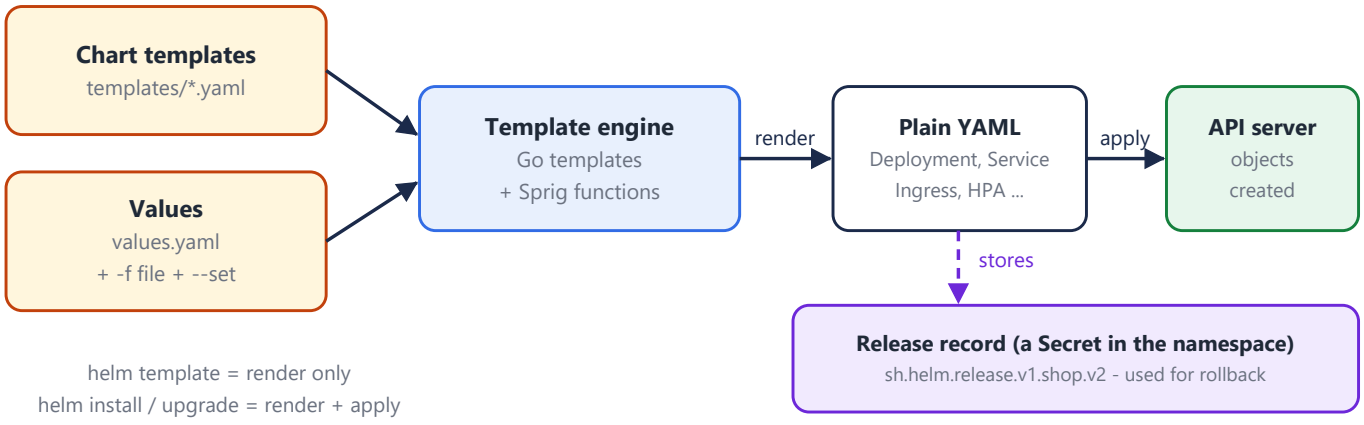


Figure 11.2 - Templates plus values are rendered into plain YAML, applied, and recorded

11.2 Using public charts

Thousands of ready-made charts are listed on **Artifact Hub** (artifacthub.io) - monitoring, databases, ingress controllers, cert-manager, Argo CD and more. We practise with **podinfo**, a small, well-maintained demo app.

PowerShell - public chart

```
helm repo add podinfo https://stefanprodan.github.io/podinfo
helm repo update
helm search repo podinfo
helm show values podinfo/podinfo > podinfo-values.yaml   # every setting you can change

helm install my-app podinfo/podinfo --namespace demo --create-namespace --set replicaCount=2
helm list -n demo
helm status my-app -n demo
kubectl -n demo port-forward svc/my-app-podinfo 9898:9898   # open http://localhost:9898

helm upgrade my-app podinfo/podinfo -n demo --set replicaCount=3 --set ui.message="Hello QuickBite"
helm history my-app -n demo
helm get values my-app -n demo          # the values you supplied
helm get manifest my-app -n demo        # the YAML Helm applied
helm rollback my-app 1 -n demo
helm uninstall my-app -n demo
```

Did you know?
Charts can also live in OCI registries, the same place your images live. Install directly with `helm install my-app oci://ghcr.io/stefanprodan/charts/podinfo` - no `helm repo add` needed.

11.3 Anatomy of a chart

`helm create quickbite` generates a complete, working chart. This is its structure:

Chart folder

```
quickbite/
  Chart.yaml              # name, chart version, appVersion, dependencies
  values.yaml             # default values
  values.schema.json      # optional: validates values (types, required fields)
  charts/                 # downloaded dependency charts (subcharts)
  .helmignore             # files to leave out when packaging
  templates/
    _helpers.tpl          # reusable named templates (names, labels)
    deployment.yaml
    service.yaml
    ingress.yaml
    hpa.yaml
    serviceaccount.yaml
    NOTES.txt             # message printed after install
    tests/
      test-connection.yaml   # run with: helm test <release>
```

Chart.yaml

```
apiVersion: v2                # Helm 3 chart format
name: quickbite
description: QuickBite orders service
type: application             # or 'library' (only helpers, nothing installed)
version: 0.1.0                # CHART version - bump on every chart change (SemVer)
appVersion: "1.4.2"           # version of the APP inside - informational
```

values.yaml

```
replicaCount: 2

image:
  repository: nginx
  tag: "1.27"
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80

ingress:
  enabled: false
  host: quickbite.local

resources:
  requests: { cpu: 100m, memory: 128Mi }
  limits:   { memory: 256Mi }

env:
  APP_MODE: dev
  LOG_LEVEL: info
```

templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "quickbite.fullname" . }}
  labels:
    {{- include "quickbite.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      {{- include "quickbite.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "quickbite.selectorLabels" . | nindent 8 }}
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion}}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: 80
          env:
            {{- range $key, $value := .Values.env }}
            - name: {{ $key }}
              value: {{ $value | quote }}
            {{- end }}
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
```

The `checksum/config` annotation is a well-known trick: whenever the ConfigMap changes, the hash changes, the pod template changes, and the Deployment rolls out new pods automatically. Add that line only once your chart has a `templates/configmap.yaml` - the chart from `helm create` does not include one.

templates/_helpers.tpl

```
{{- define "quickbite.fullname" -}}
{{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" }}
{{- end }}

{{- define "quickbite.selectorLabels" -}}
app.kubernetes.io/name: {{ .Chart.Name }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}

{{- define "quickbite.labels" -}}
{{ include "quickbite.selectorLabels" . }}
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}
```

templates/ingress.yaml - created only when ingress.enabled is true

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "quickbite.fullname" . }}
spec:
  rules:
    - host: {{ .Values.ingress.host }}
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: {{ include "quickbite.fullname" . }}
                port: { number: {{ .Values.service.port }} }
{{- end }}
```

11.4 The template language

Helm uses Go templates plus about 70 extra functions from the Sprig library. Everything between `{{ }}` is evaluated; everything else is copied as-is.

Feature	Example	Meaning
Built-in objects	`.Values`, `.Release.Name`, `.Release.Namespace`, `.Chart.Version`, `.Capabilities`, `.Files`	Data available to every template
Pipelines	`{{ .Values.env.mode | upper | quote }}`	Pass a value through functions, left to right
Default	`{{ .Values.image.tag | default "latest" }}`	Fallback when a value is empty
Required	`{{ required "image.tag is required" .Values.image.tag }}`	Fail the install with a clear message
Condition	`{{- if .Values.ingress.enabled }} ... {{- end }}`	Include a block only when true
Loop	`{{- range .Values.hosts }} - {{ . }} {{- end }}`	Repeat for each list item
Scope	`{{- with .Values.nodeSelector }} ... {{- end }}`	Shortcut into a nested value

Feature	Example	Meaning
YAML blocks	`{{- toYaml .Values.resources | nindent 12 }}`	Insert a whole map with correct indentation
Named template	`{{ include "quickbite.labels" . }}`	Reuse a block defined in _helpers.tpl
Whitespace	`{{-` and `-}}`	Trim spaces/newlines left or right of the tag

Common mistake

- Wrong indentation in rendered YAML is the number-one Helm bug. Use `nindent` with the exact column, and always check with `helm template`.
- Forgetting `quote` on strings such as `"true"` or `"1.10"` - YAML turns them into a boolean or the number 1.1.

11.5 Values for many environments

Keep one chart and one small values file per environment. Only the differences go in it:

```
# values-dev.yaml
replicaCount: 1
image:
  tag: "dev-42"
env:
  APP_MODE: dev
  LOG_LEVEL: debug
ingress:
  enabled: false
```

```
# values-prod.yaml
 replicaCount: 3
 image:
   tag: "1.4.2"
 env:
   APP_MODE: prod
   LOG_LEVEL: warn
 ingress:
   enabled: true
   host: orders.quickbite.com
```

```
helm upgrade --install shop-dev  ./quickbite -n dev  --create-namespace -f values-dev.yaml
helm upgrade --install shop-prod ./quickbite -n prod --create-namespace -f values-prod.yaml
```

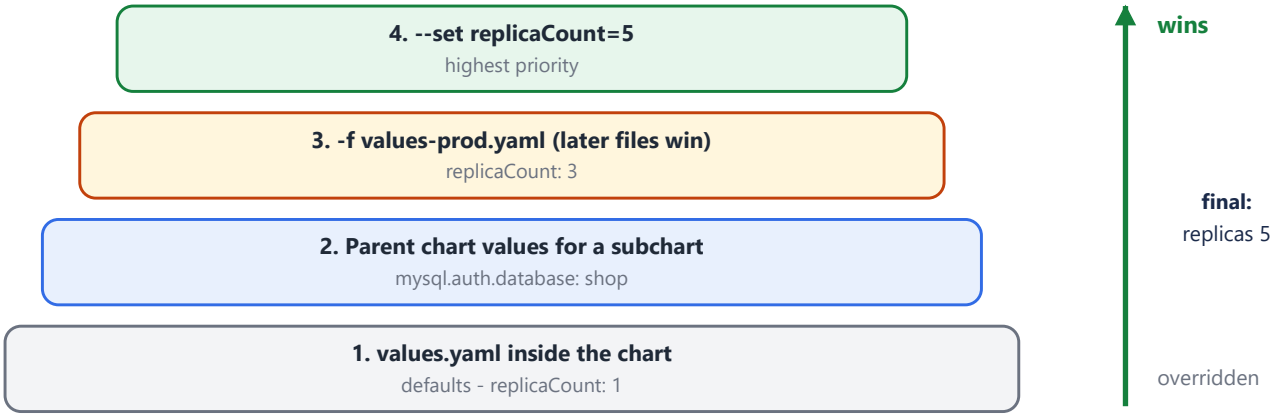


Figure 11.3 - When the same value is set in several places, the higher layer wins

Remember

- Order of priority (lowest to highest): chart `values.yaml` -> parent chart values -> `-f` files (the last file wins) -> `--set` / `--set-string` / `--set-file`.
- Use `-f` files kept in Git for anything permanent; use `--set` only for quick changes or the image tag from CI.
- Never put real passwords in values files in Git - use External Secrets, Sealed Secrets or the helm-secrets plugin with SOPS.

11.6 Dependencies, hooks and tests

Dependencies (subcharts)

A chart can depend on other charts - for example orders needs a Redis cache. Declare it in `Chart.yaml`, then run `helm dependency update` to download it into `charts/`. The parent configures the subchart under a key with the subchart's name; `global:` values are shared with all subcharts.

Chart.yaml + values.yaml

```
dependencies:
  - name: redis
    version: "~20.0.0"                # check the chart's current version on Artifact Hub
    repository: oci://registry-1.docker.io/bitnamicharts
    condition: redis.enabled          # install only when redis.enabled is true

# values.yaml of the parent chart
redis:
  enabled: true
  architecture: standalone
```

Hooks

Hooks run a resource - usually a Job - at a specific moment of the release lifecycle: `pre-install`, `post-install`, `pre-upgrade`, `post-upgrade`, `pre-rollback`, `pre-delete`, `post-delete` and `test`. The classic use is running database migrations before new pods start.

templates/db-migrate-job.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: {{ include "quickbite.fullname" . }}-db-migrate
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "0"                      # lower weights run first
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  backoffLimit: 1
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: busybox:1.36
          command: ["sh", "-c", "echo running DB migrations for {{ .Chart.AppVersion }}"]
```

Tests

A pod with the annotation `"helm.sh/hook": test` is a chart test. `helm test <release>` runs it after installation; the test passes if the pod exits with code 0. The generated chart already includes `tests/test-connection.yaml`, which runs `wget` against the Service.

11.7 Debug, package and share

Command	Use
`helm lint ./quickbite`	Check the chart for errors and best-practice warnings
`helm template shop ./quickbite -f values-prod.yaml`	Render YAML locally - nothing touches the cluster
`helm install shop ./quickbite --dry-run --debug`	Render and validate against the cluster without installing
`helm diff upgrade shop ./quickbite`	Show what would change (helm-diff plugin) - great in pull requests
`helm package ./quickbite`	Create `quickbite-0.1.0.tgz`
`helm registry login <registry>`	Log in to an OCI registry (Docker Hub, ECR, ACR, GHCR, Harbor)
`helm push quickbite-0.1.0.tgz oci://<registry>/<path>`	Publish the chart
`helm pull oci://<registry>/<path>/quickbite --version 0.1.0`	Download a published chart

Chart version vs appVersion: `version` is the version of the packaging and must change whenever any chart file changes. `appVersion` is the version of the software inside. A chart 0.3.0 might deploy app 1.4.2.

11.8 Helm in CI/CD and GitOps

Typical pipeline step (bash)

```
helm upgrade --install orders ./charts/quickbite \
  --namespace prod --create-namespace \
  -f values-prod.yaml \
  --set image.tag=$GIT_COMMIT_SHA \
  --atomic --wait --timeout 5m          # roll back automatically if pods are not healthy in 5 min
```

- `upgrade --install` works whether or not the release already exists - one command for every run.
- `--wait` waits until pods are Ready; `--atomic` rolls back automatically on failure.
- In GitOps, Argo CD or Flux render the chart themselves from Git with your values files - you never run `helm install` by hand in production. Note that `helm list` will not show Argo CD apps, because Argo CD uses Helm only to render.
- Helmfile manages many releases across environments from one declarative file.

Did you know?
Helm 4 was released in November 2025. Charts written for Helm 3 keep working. The main changes are server-side apply, smarter waiting for resources, WebAssembly plugins, and some renamed flags - for example `--atomic` is now also called `--rollback-on-failure`. Run `helm version` to see which one you have; every command in this class works on both.

11.9 Kustomize - the template-free alternative

Kustomize is built into kubectl. Instead of templates, you keep plain, valid YAML in a **base** folder and small **overlays** that patch it for each environment.

Everyday example
Kustomize is like buying a plain kurta (base) and having the tailor alter only the sleeves for one person and the length for another (overlays). Helm is like a pattern with blanks for measurements - every kurta is cut fresh from the pattern.

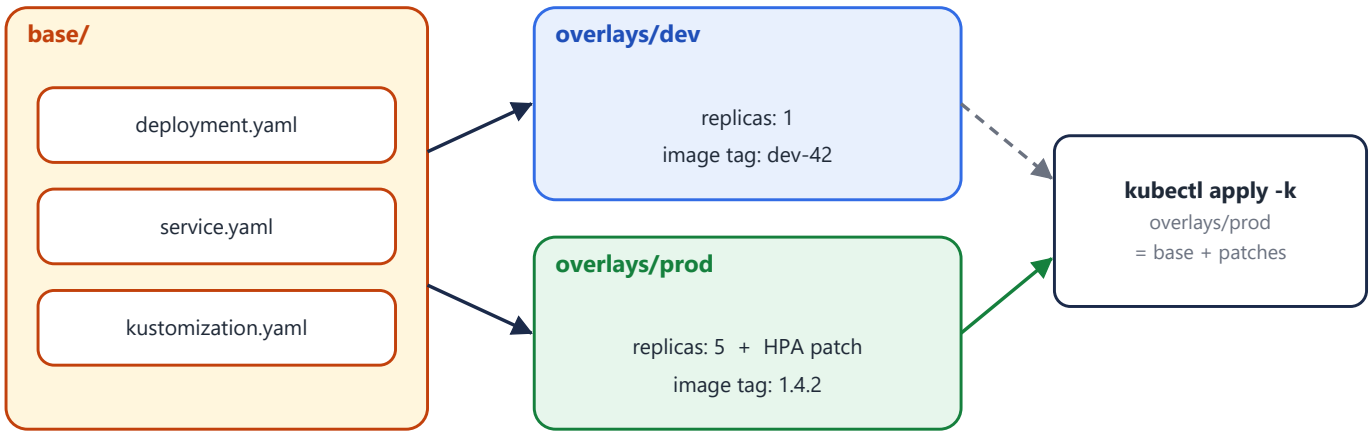


Figure 11.4 - Kustomize overlays patch a shared base for each environment

kustomization files

```
# base/kustomization.yaml
resources:
  - deployment.yaml
  - service.yaml

# overlays/prod/kustomization.yaml
resources:
  - ../../base
namespace: prod
namePrefix: prod-
replicas:
  - name: web
    count: 5
images:
  - name: nginx
    newTag: "1.27"
configMapGenerator:                # adds a hash suffix -> pods restart on change
  - name: app-config
    literals: ["APP_MODE=prod"]
patches:
  - path: memory-patch.yaml
```

```
kubectl kustomize overlays/prod          # preview the final YAML
kubectl apply -k overlays/prod
kubectl delete -k overlays/prod
```

	Helm	Kustomize
Approach	Templates + values	Plain YAML + patches
Installed with	helm CLI	Built into kubectl (`-k`)
Packaging and versions	Yes - charts, repositories, OCI	No - just folders in Git
Release history and rollback	Yes - `helm rollback`	No - use Git revert / GitOps
Hooks and tests	Yes	No

	Helm	Kustomize
Best for	Distributing apps, third-party software, many settings	Your own apps with small per-environment changes

Many teams use both: install third-party software with Helm, and manage their own apps with Kustomize - or even post-process a Helm chart with Kustomize.

11.10 Other important concepts

imagePullPolicy

Value	Behaviour
IfNotPresent	Pull only if the image is not on the node. Default for a fixed tag like `nginx:1.27`.
Always	Check the registry on every container start. Default when the tag is `latest` or missing.
Never	Use only a local image; fail if absent. Handy with `minikube image load`.

Owner references, garbage collection and finalizers

Pods created by a ReplicaSet carry an **ownerReference** pointing to it, and the ReplicaSet points to its Deployment. When an owner is deleted, the **garbage collector** deletes its dependents (cascading deletion). `kubectl delete deploy web --cascade=orphan` keeps the pods instead.

A **finalizer** is a key in `metadata.finalizers` that blocks deletion until a controller finishes cleanup - for example `kubernetes.io/pvc-protection` stops a PVC in use from being deleted. If the controller is gone, the object stays **Terminating** forever; that is why namespaces sometimes get stuck. Removing finalizers by hand is a last resort.

CRDs and Operators

A **CustomResourceDefinition** teaches the API server a new kind of object. An **Operator** is a controller that watches those objects and does what a human expert would: create the StatefulSet, take backups, fail over, upgrade. You install Operators - often with Helm - and then simply write short custom resources.

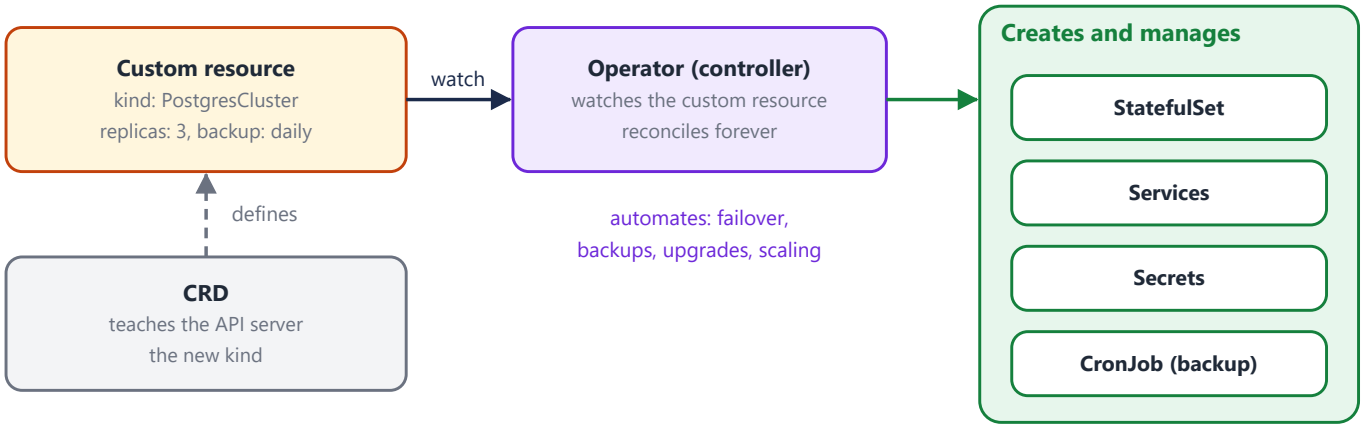


Figure 11.5 - An Operator turns a short custom resource into many managed objects

crd.yaml - try: kubectl apply, kubectl get backups

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: backups.quickbite.io
spec:
  group: quickbite.io
  scope: Namespaced
  names: { kind: Backup, plural: backups, singular: backup }
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                database: { type: string }
                schedule: { type: string }
---
apiVersion: quickbite.io/v1
kind: Backup
metadata: { name: orders-nightly }
spec: { database: orders, schedule: "0 1 * * *" }
```

Without an Operator the Backup object is only stored data; an Operator gives it behaviour. Real examples: cert-manager, Prometheus Operator, Strimzi (Kafka), CloudNativePG (PostgreSQL).

Building a cluster with kubeadm

minikube is for learning; on real Linux VMs the official tool is **kubeadm**. It is also the basis of the CKA exam.

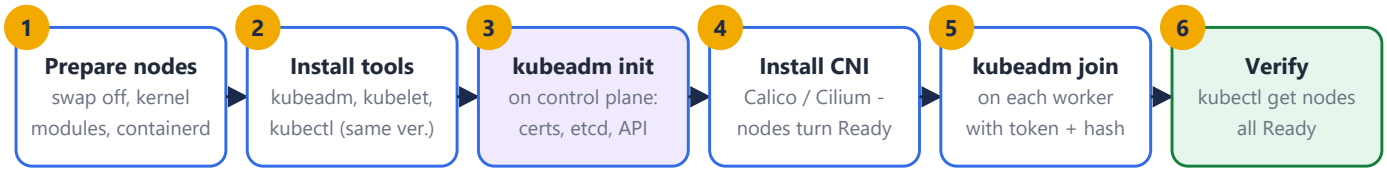


Figure 11.6 - Building a cluster with kubeadm

bash on Linux VMs

```
# on every node (Ubuntu): swap off, containerd with SystemdCgroup = true,
# kubelet + kubeadm + kubectl from pkgs.k8s.io, then: sudo apt-mark hold kubelet kubeadm kubectl
sudo swapoff -a

# control plane
sudo kubeadm init --pod-network-cidr=192.168.0.0/16
mkdir -p $HOME/.kube && sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
kubectl apply -f <calico-manifest-url>         # install the CNI; nodes become Ready
kubeadm token create --print-join-command       # prints the join command

# each worker
sudo kubeadm join 10.0.0.10:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

Upgrading a cluster

- Upgrade **one minor version at a time** (1.33 to 1.34, never 1.32 to 1.34). Read the release notes for removed APIs first.
- Control plane first: upgrade the kubeadm package, run `kubeadm upgrade plan`, then `sudo kubeadm upgrade apply v1.xx.y`; drain the node, upgrade kubelet and kubectl, restart kubelet, uncordon.
- Then each worker: drain, upgrade kubeadm, `sudo kubeadm upgrade node`, upgrade kubelet, restart, uncordon.
- Version skew rule: the kubelet may be up to three minor versions older than the API server, but never newer.
- On EKS, AKS and GKE the cloud upgrades the control plane; you upgrade node pools.

Backing up and restoring etcd

bash on the control-plane node

```
sudo ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-$(date +%F).db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

sudo etcdutl snapshot status /backup/etcd-2026-10-10.db --write-out=table

# restore into a NEW data directory, then point the etcd static pod to it
sudo etcdutl snapshot restore /backup/etcd-2026-10-10.db --data-dir /var/lib/etcd-restore
# edit the hostPath /var/lib/etcd -> /var/lib/etcd-restore
sudo vi /etc/kubernetes/manifests/etcd.yaml
```

etcd snapshots protect cluster objects only. Application data in volumes needs its own backup - **Velero** backs up namespaces and volume snapshots together, and can restore them into another cluster.

11.11 Live lab: your own Helm chart

PowerShell - part 1

```
helm create quickbite
cd quickbite
#  edit values.yaml: image.repository: nginx, image.tag: "1.26", replicaCount: 1
cd ..
helm lint ./quickbite
helm template shop ./quickbite | more        # read the rendered YAML

helm install shop-dev ./quickbite -n dev --create-namespace
kubectl -n dev get deploy,svc,pods
helm test shop-dev -n dev                    # runs templates/tests/test-connection.yaml
```

PowerShell - part 2

```
# create values-prod.yaml with: replicaCount: 3 and image.tag: "1.27"
helm install shop-prod ./quickbite -n prod --create-namespace -f values-prod.yaml
kubectl -n prod get pods                     # 3 pods on nginx:1.27

# a bad upgrade, then rollback
helm upgrade shop-prod ./quickbite -n prod -f values-prod.yaml --set image.tag=9.99-broken
kubectl -n prod get pods                     # ImagePullBackOff
helm history shop-prod -n prod
helm rollback shop-prod 1 -n prod
kubectl -n prod get secrets -l owner=helm    # the release records Helm keeps

# safer upgrades
helm upgrade shop-prod ./quickbite -n prod -f values-prod.yaml `
  --set image.tag=9.99-broken --atomic --wait --timeout 60s   # fails, rolls back by itself

helm package ./quickbite                     # quickbite-0.1.0.tgz
helm uninstall shop-dev -n dev; helm uninstall shop-prod -n prod
```

Try it yourself

- Add a `configmap.yaml` template driven by `.Values.env`, plus the checksum annotation, and watch pods roll when you change a value.
- Add the db-migrate hook from section 11.6 and watch the Job run before each upgrade: `kubectl -n prod get jobs -w`.
- Convert the same app to Kustomize with `base` and `overlays/dev|prod` folders and compare.

Common mistake

- Changing Helm-managed objects with `kubectl edit` - the next `helm upgrade` overwrites your change. Change the values and upgrade instead.
- Forgetting to bump `version` in Chart.yaml before packaging - registries refuse a duplicate version, or worse, users get stale charts.
- Using `--set` for many values in production - nobody can see them later. Put them in a values file in Git.

Class 11 recap

- Helm = package manager: chart (templates + values) -> release -> revisions.
- Key commands: repo add, search, show values, install, upgrade --install, list, history, rollback, uninstall, template, lint, test, package, push.
- Values priority: chart defaults < parent < -f files < --set.
- Hooks run Jobs at lifecycle moments; `helm test` validates a release.
- Kustomize = base + overlays, built into kubectl, no templates.
- Admin essentials: owner references and finalizers, CRDs and Operators, kubeadm, one-minor-at-a-time upgrades, etcd snapshots.

Class 11 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is Helm and why is it used?

Helm is the package manager for Kubernetes. It packages related manifests as a chart with templates and default values, so one chart can be installed in many environments with different settings, and it tracks releases so you can upgrade and roll back with one command.

Q2. Explain chart, values, release, revision and repository.

A chart is the package; values are the configuration that fills its templates; a release is one installed copy of a chart in a namespace; a revision is a numbered version of that release created by each install, upgrade or rollback; a repository or OCI registry stores published charts.

Q3. What changed between Helm 2 and Helm 3?

Helm 3 removed Tiller, the in-cluster server component, so Helm now uses the user's own kubeconfig and RBAC permissions. Release data is stored as Secrets in the release namespace, chart apiVersion became v2, and OCI registry support was added.

Q4. Describe the structure of a Helm chart.

Chart.yaml (metadata, versions, dependencies), values.yaml (defaults), an optional values.schema.json, a charts/ folder for subcharts, a templates/ folder with manifests, _helpers.tpl, NOTES.txt and tests/, and a .helmignore file.

Q5. What is the difference between version and appVersion in Chart.yaml?

version is the SemVer version of the chart itself and must change whenever the chart changes. appVersion is the version of the application the chart deploys and is informational - often used as the default image tag.

Q6. How do you override values when installing a chart?

With `-f my-values.yaml` (several files allowed; the last wins) or `--set key=value`, `--set-string` and `--set-file`. Priority from lowest to highest: chart values.yaml, parent chart values, -f files, --set.

Q7. What is the difference between helm install, helm upgrade and helm upgrade --install?

install creates a new release and fails if it exists; upgrade changes an existing release and fails if it does not exist; upgrade --install does whichever is needed, which makes it ideal for CI/CD pipelines.

Q8. How do you roll back a Helm release?

Use `helm history <release>` to find the revision, then `helm rollback <release> <revision>`. The rollback itself becomes a new revision. `--atomic` on upgrade rolls back automatically on failure.

Q9. Where does Helm store release information?

In Secrets of type helm.sh/release.v1 in the release's namespace, named like sh.helm.release.v1.<release>.v<revision>. They contain the rendered manifests and values used.

Q10. What is _helpers.tpl?

A file of named templates defined with `define` and used with `include` - typically the full name, common labels and selector labels - so they are written once and reused in every template. Files starting with an underscore do not produce manifests.

Q11. What is the difference between include and template?

Both render a named template, but include returns a string that can be piped into functions such as `nindent` or `quote`, so include is preferred.

Q12. How do you debug a Helm chart before installing it?

Run `helm lint`, render locally with `helm template`, validate against the cluster with `helm install --dry-run --debug`, and preview upgrades with the helm-diff plugin.

Q13. What are Helm hooks? Give an example.

Resources annotated with `helm.sh/hook` that run at lifecycle points such as pre-install, pre-upgrade, post-install, pre-delete or test. A common example is a pre-upgrade Job that runs database migrations before new pods start. Weights order hooks and delete policies clean them up.

Q14. How do you test a Helm release?

Put pods with the annotation `helm.sh/hook: test` in templates/tests and run `helm test <release>`. The test passes if the pods complete successfully.

Q15. How do chart dependencies work?

Dependencies are listed in Chart.yaml with name, version, repository and optional condition. `helm dependency update` downloads them into charts/. The parent sets subchart values under the subchart's name, and `global` values are shared with all subcharts.

Q16. How do you share a chart with other teams?

Package it with `helm package` and push it to an OCI registry with `helm push` after `helm registry login`, or publish to a classic chart repository such as GitHub Pages, ChartMuseum or Harbor.

Q17. How do you make pods restart when a ConfigMap rendered by Helm changes?

Add an annotation to the pod template containing a checksum of the rendered ConfigMap, e.g. `checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}`. Any change alters the template and triggers a rolling update.

Q18. Helm vs Kustomize - when would you use each?

Helm suits distributing and versioning apps, third-party software and charts with many options, and offers hooks, tests and rollback. Kustomize suits your own apps with small per-environment differences using plain YAML overlays, built into kubectl. Many teams use both.

Q19. What are finalizers and owner references?

Owner references link dependents to owners (Pod to ReplicaSet to Deployment) so the garbage collector deletes dependents when the owner is deleted. Finalizers block deletion of an object until a controller completes cleanup; if the controller is missing the object stays Terminating.

Q20. How do you back up and restore etcd?

Take a snapshot with `etcdctl snapshot save` using the etcd endpoint and its CA, cert and key; check it with `etcdutl snapshot status`; restore with `etcdutl snapshot restore --data-dir <new dir>` and point the etcd static pod manifest to the new directory. Use Velero for application volumes.

kubectl cheat sheet

Task	Command
Cluster info	`kubectl cluster-info`, `kubectl get nodes -o wide`
List everything in a namespace	`kubectl get all -n <ns>`
All namespaces	`kubectl get pods -A`
Details and events	`kubectl describe pod <name>`
YAML of a live object	`kubectl get deploy web -o yaml`
Generate YAML	`kubectl create deploy web --image=nginx --dry-run=client -o yaml`
Apply / delete files	`kubectl apply -f file.yaml`, `kubectl delete -f file.yaml`
Preview changes	`kubectl diff -f file.yaml`
Logs	`kubectl logs <pod> -c <ctr> -f --previous`
Shell	`kubectl exec -it <pod> -- sh`
Debug container	`kubectl debug -it <pod> --image=busybox:1.36 --target=<ctr>`
Port forward	`kubectl port-forward svc/web 8080:80`
Scale	`kubectl scale deploy web --replicas=5`
Update image	`kubectl set image deploy/web nginx=nginx:1.27`
Rollout	`kubectl rollout status|history|undo|restart deploy/web`
Expose	`kubectl expose deploy web --port=80 --type=NodePort`
Labels	`kubectl get pods -l app=web --show-labels`
Resource usage	`kubectl top pods`, `kubectl top nodes`
Events	`kubectl get events --sort-by=.lastTimestamp`
Contexts	`kubectl config get-contexts`, `use-context`, `set-context --current --namespace=x`
Permissions	`kubectl auth can-i <verb> <resource> --as=<user>`
Node maintenance	`kubectl cordon|drain|uncordon <node>`
Docs	`kubectl explain deploy.spec.strategy`, `kubectl api-resources`
Kustomize	`kubectl kustomize overlays/prod`, `kubectl apply -k overlays/prod`

Helm cheat sheet

Task	Command
Add and refresh a repository	`helm repo add <name> <url>`, `helm repo update`
Find charts	`helm search repo <word>`, `helm search hub <word>`
See settings of a chart	`helm show values <repo/chart>`
Install / upgrade in one command	`helm upgrade --install <rel> <chart> -n <ns> --create-namespace -f values.yaml`
Safe upgrade	`helm upgrade <rel> <chart> --atomic --wait --timeout 5m`
List and inspect releases	`helm list -A`, `helm status <rel>`, `helm get values|manifest <rel>`
History and rollback	`helm history <rel>`, `helm rollback <rel> <revision>`
Remove	`helm uninstall <rel> -n <ns>`
Create and check a chart	`helm create <name>`, `helm lint`, `helm template`, `--dry-run --debug`
Dependencies	`helm dependency update`
Test a release	`helm test <rel>`
Package and publish	`helm package <dir>`, `helm push <tgz> oci://<registry>/<path>`

Speed tips

- Add `Set-Alias k kubectl` to your PowerShell profile (`notepad $PROFILE`) and type `k get pods`.
- Enable completion: `kubectl completion powershell | Out-String | Invoke-Expression`.

Concept coverage map

Use this map to revise a topic quickly. Every major Kubernetes concept in the CKAD/CKA syllabus is covered somewhere in the eleven classes.

Concept	Class
Architecture, control plane, nodes, etcd, kubelet, kube-proxy, CRI	1
Desired state, controllers, reconciliation, self-healing	1, 3
Pods, multi-container, init and sidecar containers, static pods	2
Labels, selectors, annotations, YAML, kubectl	2
ReplicaSet, Deployment, rolling update, rollback, blue-green, canary	3
Services (ClusterIP, NodePort, LoadBalancer, ExternalName, headless), EndpointSlices	4
CoreDNS, networking model, CNI, kube-proxy	4
Ingress, IngressClass, TLS, Gateway API	5
Namespaces, ResourceQuota, LimitRange, kubeconfig and contexts	5
ConfigMaps, Secrets, Downward API, imagePullSecrets	6
Volumes, PV, PVC, StorageClass, CSI, access modes, snapshots	7
StatefulSet and headless Services	7
DaemonSet, Job, CronJob	8
Scheduling: nodeSelector, affinity, anti-affinity, taints, tolerations, topology spread	8
Requests, limits, QoS, OOMKilled, PriorityClass, cordon/drain	8
Probes, lifecycle hooks, graceful shutdown	9
HPA, VPA, Cluster Autoscaler, KEDA, PodDisruptionBudget	9
Monitoring, logging, troubleshooting, kubectl debug	9
Authentication, RBAC, ServiceAccounts, admission, Pod Security, securityContext	10
NetworkPolicy	10
CI/CD, GitOps (Argo CD), managed Kubernetes, high availability, service mesh	10
Helm: charts, templates, values, releases, hooks, tests, dependencies, OCI packaging	11
Kustomize: base, overlays, patches, generators	11
imagePullPolicy, owner references, garbage collection, finalizers	11
CRDs and Operators	10, 11
kubeadm cluster setup, cluster upgrades, etcd backup and restore	11

Everyday-example glossary

If you forget a concept, remember its everyday example first - the definition follows easily.

Kubernetes term	Remember it as
Cluster / control plane / node	A restaurant chain / its head office / its branches
etcd	The head office's master register book
Scheduler	HR deciding which branch a new cook joins
kubelet	The branch manager reporting to head office
Reconciliation loop	An AC thermostat holding 24 degrees
Pod	A flat - one address, shared kitchen
Labels and selectors	Supermarket stickers and the manager's instruction
Deployment	A manager always keeping 3 cooks on shift
Rolling update	Changing a bus's tyres one at a time while it runs
Service	The restaurant's fixed phone number
Ingress	A mall's main entrance and directory board
Namespace	Floors in one office building
ConfigMap / Secret	The notice board / a locker
PV / PVC / StorageClass	A gym locker / the request slip / the locker factory
StatefulSet	Numbered train seats with your bag underneath
DaemonSet / Job / CronJob	A guard on every floor / a one-time cleaning / an alarm clock
Taint and toleration	An airport VIP lounge and the lounge card
Requests and limits	A table booking and the fire-safety maximum
Probes	Hospital checks: awake? heartbeat? ready for patients?
HPA	Opening extra supermarket billing counters
RBAC	Office ID cards and the access register
NetworkPolicy	Housing-society gate rules
Helm	A phone app store with update and rollback
Helm chart and values	A wedding-card design and the names filled in
Kustomize	A tailor altering a ready-made kurta
GitOps	The approved floor plan the inspector checks against
Operator	An expert employee who runs one complex system for you

Final checklist - can you do these without notes?

- Draw the Kubernetes architecture and explain every component in one sentence.
- Explain what happens, step by step, after `kubectl apply -f deploy.yaml`.
- Write Pod, Deployment and Service YAML from memory (or generate it with dry-run).
- Perform a rolling update and a rollback, and explain maxSurge and maxUnavailable.
- Expose an app with ClusterIP, NodePort, LoadBalancer and Ingress.
- Inject configuration with a ConfigMap and a Secret, and explain why base64 is not security.
- Give a database persistent storage with a PVC and explain StatefulSets.
- Place pods with nodeSelector, affinity and taints; set requests and limits.
- Add liveness, readiness and startup probes and autoscale with an HPA.
- Debug Pending, ImagePullBackOff, CrashLoopBackOff and OOMKilled pods.
- Restrict access with RBAC and traffic with a NetworkPolicy.
- Explain GitOps and deploy an app from Git with Argo CD.
- Install, upgrade and roll back a public Helm chart with your own values.
- Write a Helm chart with templates, helpers, per-environment values and a hook, then package it.
- Create Kustomize overlays for dev and prod.
- Explain kubeadm setup, the cluster upgrade order and how to back up etcd.

Where to go next

- **Certifications** - CKAD (application developer), CKA (administrator), CKS (security). All are hands-on exams; the labs in these notes are the right kind of practice.
- **Practise daily** - rebuild the final project from scratch until it takes under 30 minutes.
- **Try a real cloud** - create a small EKS, AKS or GKE cluster with free credits and deploy your project there. Delete it afterwards to avoid charges.
- **Official docs** - kubernetes.io/docs has tutorials and the full API reference.

> **Last word from TRISHA ACADEMY**
> Kubernetes looks big, but it is one idea repeated everywhere: you declare the desired state, and a controller works forever to make reality match it. Keep practising!